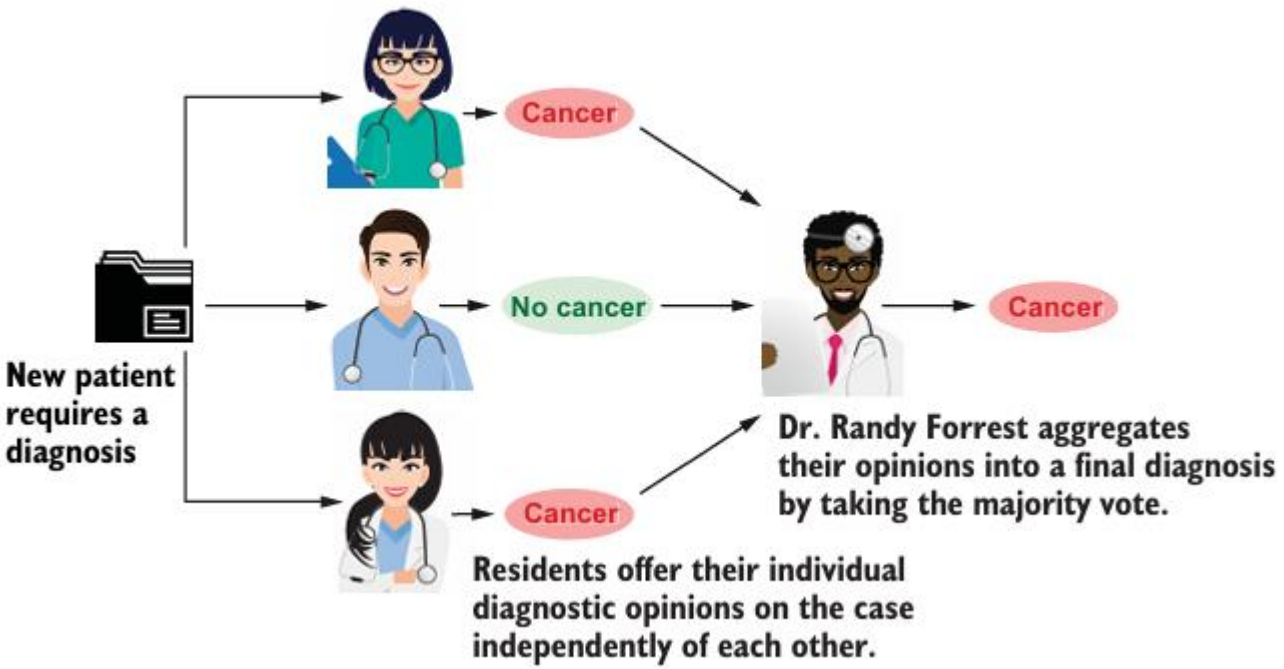
06

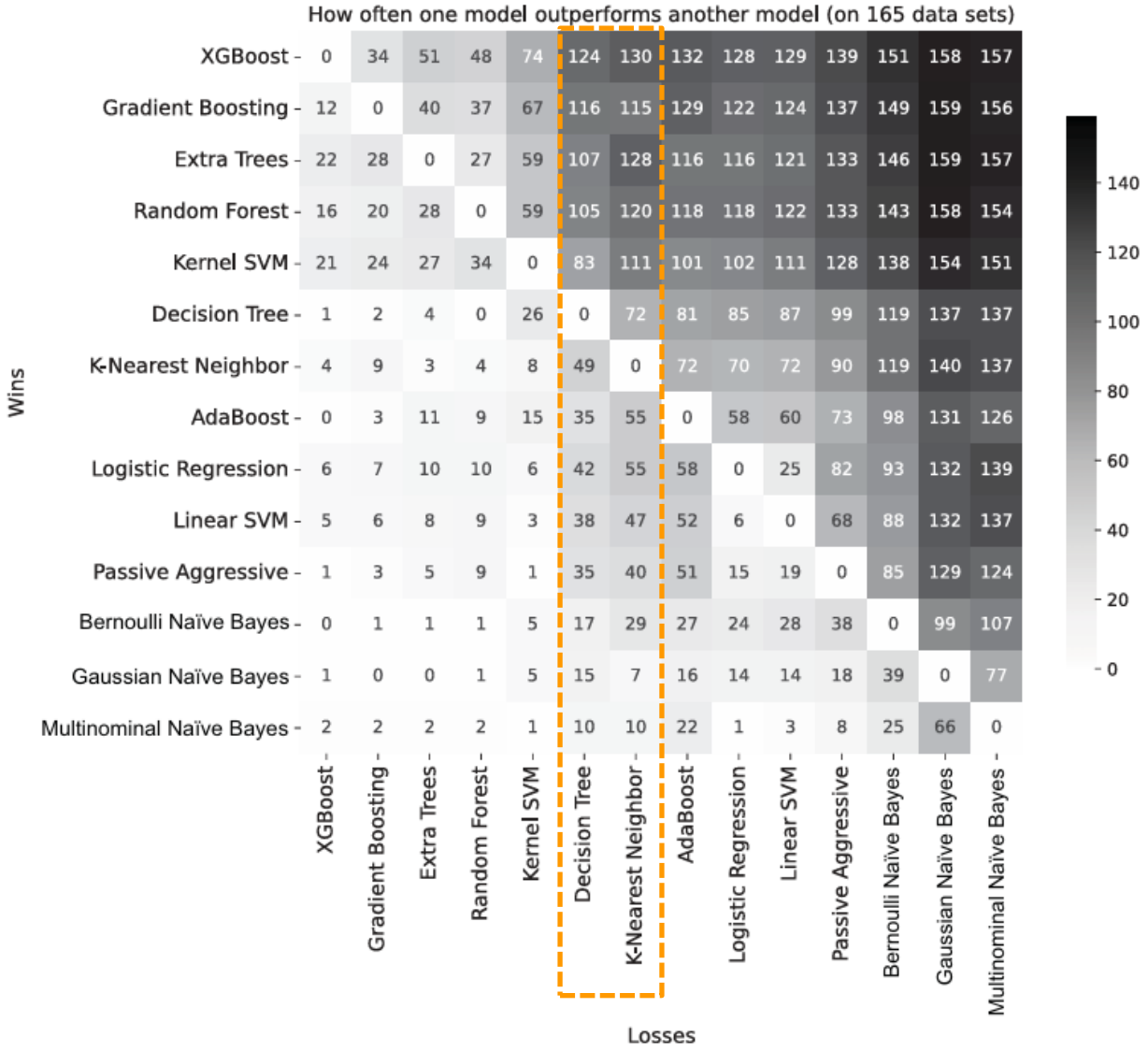
Ensemble Classifier

Ensemble methods: The wisdom of the crowds

- Ensemble diversity
  - refers to the fact that individual ensemble components, or machine-learning models, are different from each other.
- Model aggregation



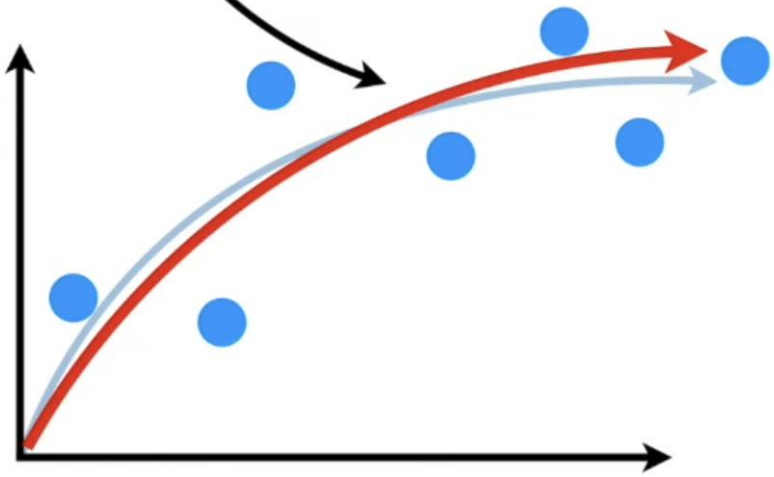
Why you should care about ensemble learning



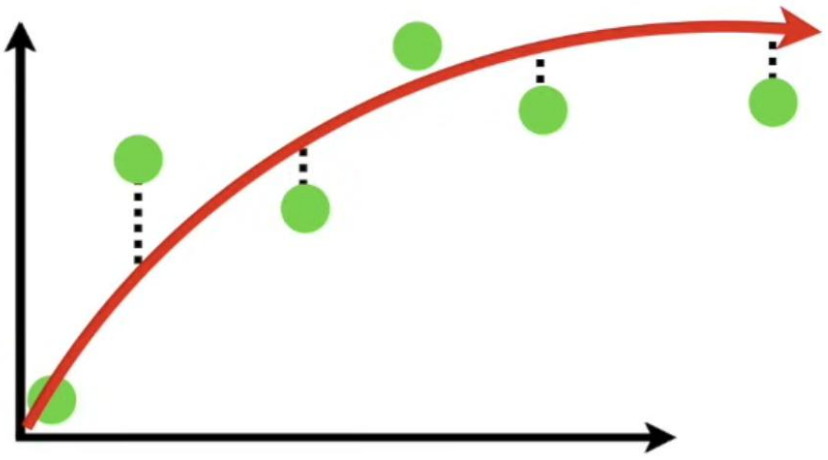
Randal S. Olson, William La Cava, Zairah Mustahsan, Akshay Varik, and Jason H. Moore, Data-driven Advice for Applying Machine Learning to Bioinformatics Problems, Pacific Symposium on Machine Learning (2018); arXiv preprint: https://arxiv.org/abs/1708.05070

Bias and Variance

In machine learning, the ideal algorithm has **low bias** and can accurately model the true relationship…

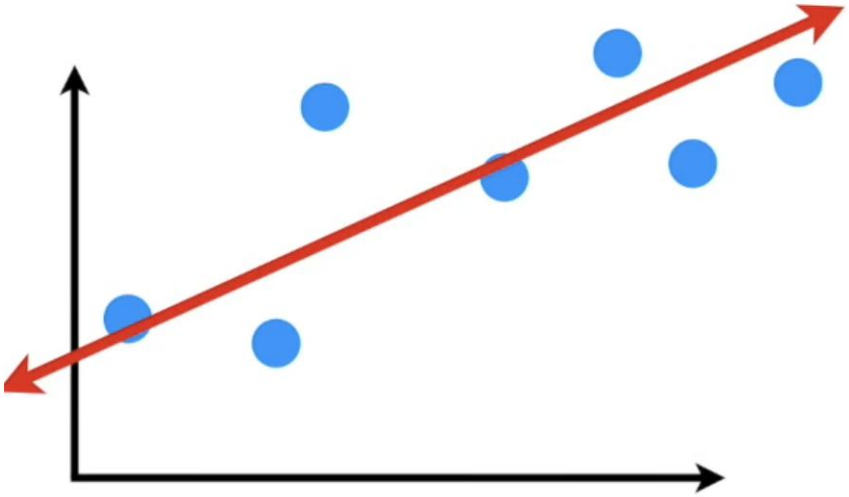


…and it has **low variability**, by producing consistent predictions across different datasets.

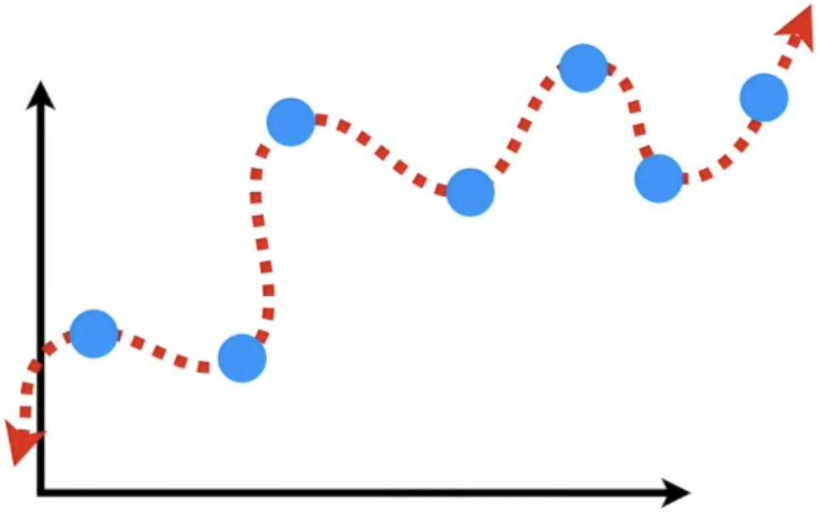


Bias and Variance

This is done by finding the sweet spot between a simple model…



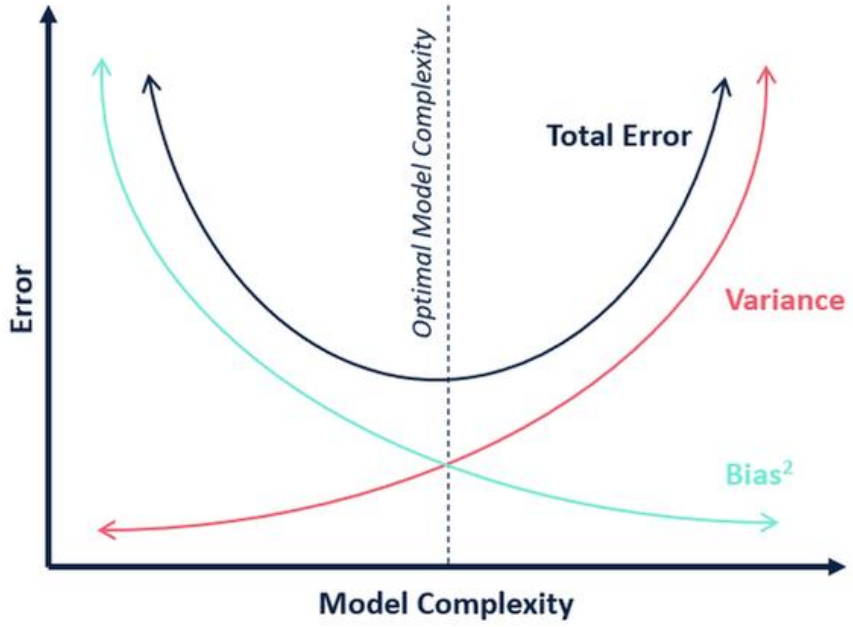
…and a complex model.



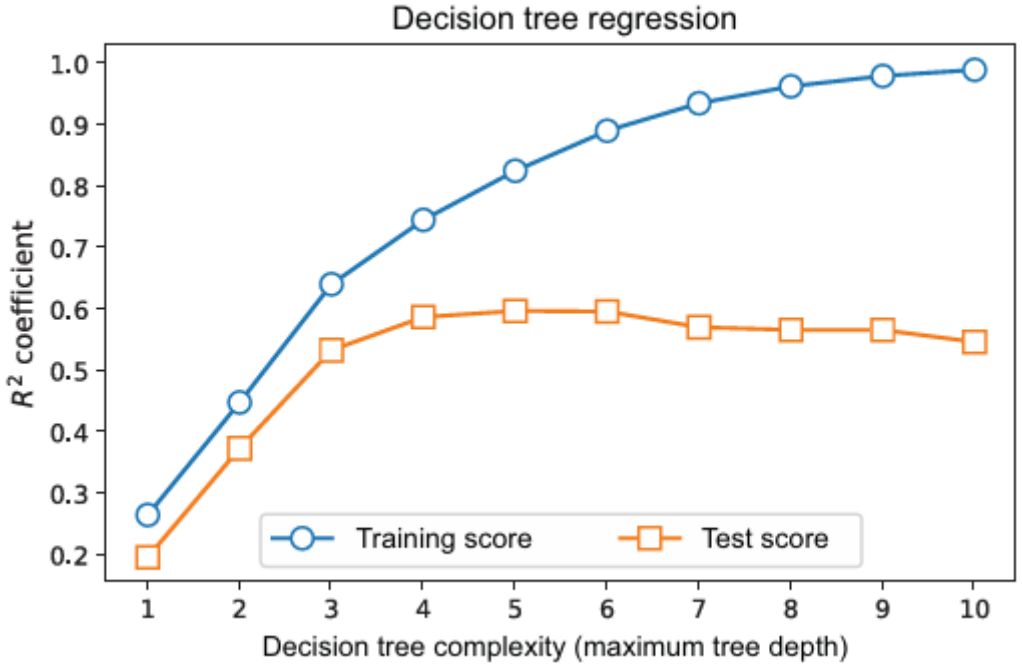
Three commonly used methods for finding the sweet spot between simple and complicated models are: **regularization**, **boosting** and **bagging**.

The bias-variance tradeoff

- The bias of a model is the error arising from the effect of modeling assumptions (such as a preference for simpler models).
- The variance of a model is the error arising from sensitivity to small variations in the data set.
- Highly complex models (low bias) will overfit the data and be more sensitive to noise (high variance).
- Simpler models (high bias) will underfit the data and be less sensitive to noise (low variance).
- Ensemble methods seek to overcome this problem by combining several low-bias models to reduce their variance or combining several low-variance models to reduce their bias



Fit vs. Complexity in individual models



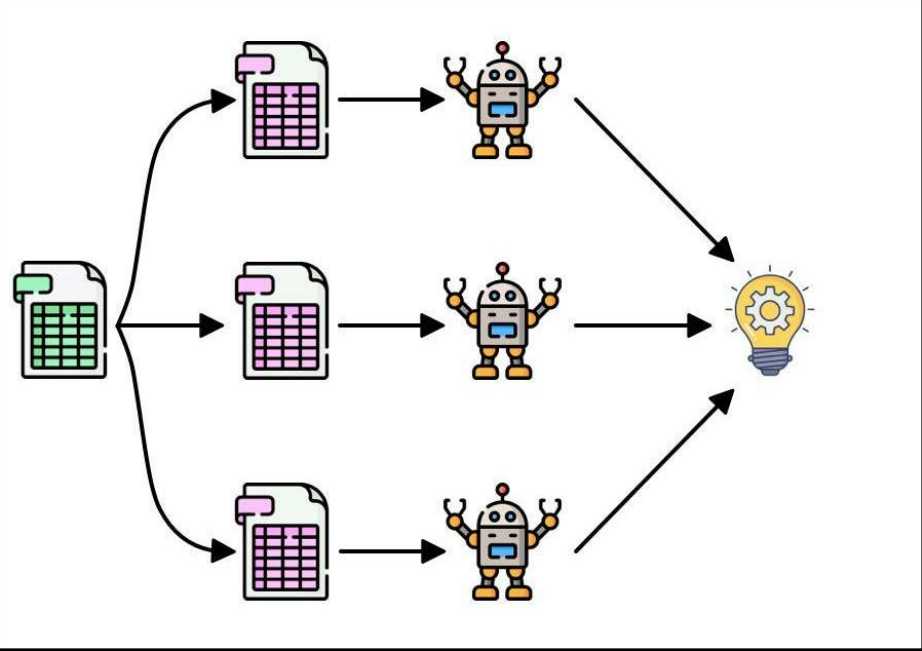
Higher R^2 scores mean that the model achieves lower error and fits the data better. An R^2 score close to 1 means that the model achieves nearly zero error. It's possible to fit the training data nearly perfectly with very deep decision trees, but such overly complex models actually overfit the training data and don't generalize well to future data, as evidenced by the test scores.

- Overly simple models tend to not fit the training data properly and tend to generalize poorly on future data; **a model that is performing poorly on training and test data is** underfitting.
- Overly complex models can achieve very low training errors but tend to generalize poorly on future data too; **a model that is performing very well on training data, but poorly on test data is** overfitting**.**
- **The best models trade off between complexity and fit**, sacrificing a little bit of each during training so that they can generalize most effectively when deployed.

Terminology and taxonomy for ensemble methods

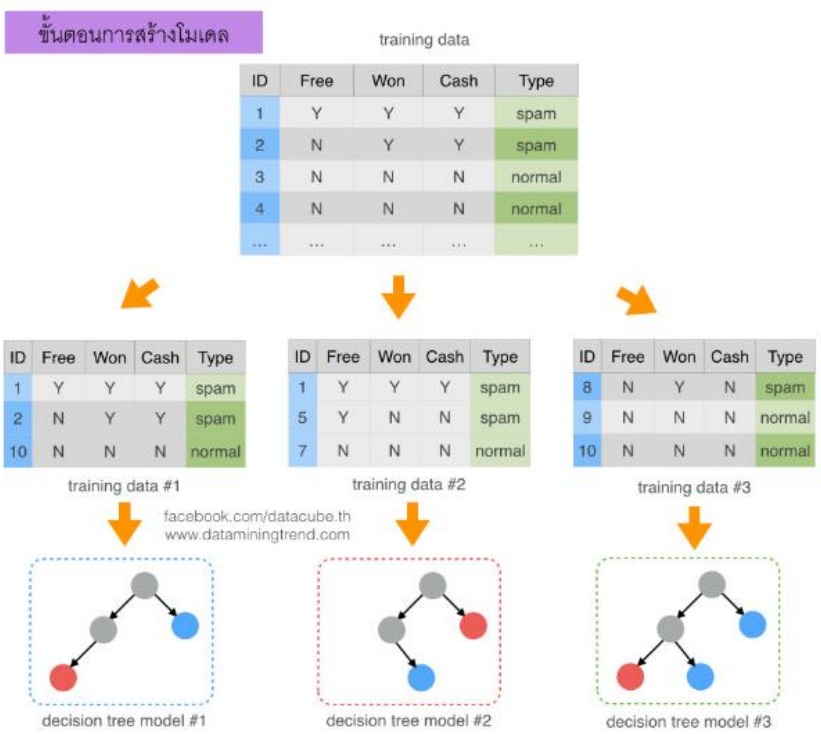
- All ensembles are composed of individual machine-learning models called **base models, base learners, or** base estimators and are trained using base machine-learning algorithms
- Ensemble methods can be classified into two types depending on how they are trained: parallel and sequential ensembles.
- **Parallel ensemble methods**, as the name suggests, train each component base model independently of the others, which means that they can be trained in parallel.
  - Homogeneous parallel ensembles - All the base learners are of the same type and trained using the same base-learning algorithm.
  - Heterogeneous parallel ensembles—The base learners are trained using different base-learning algorithms.
- **Sequential ensemble methods** exploit the dependence of base learners. During training, sequential ensembles train a new base learner in such a manner that it minimizes mistakes made by the base learner trained in the previous step.

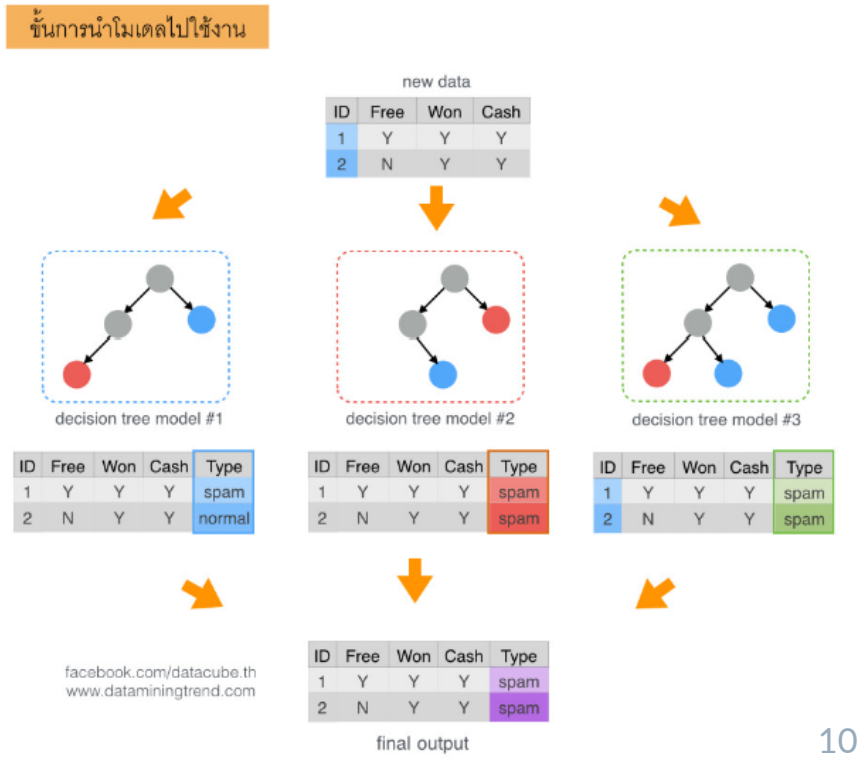
Parallel ensembles



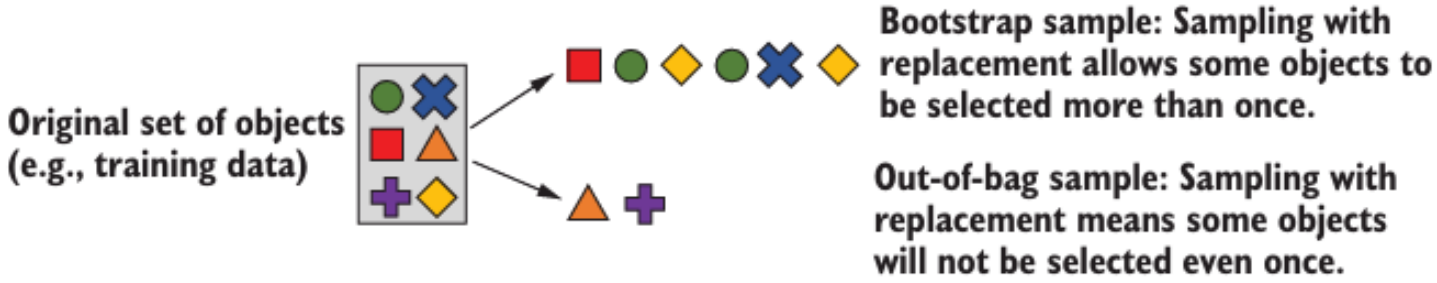
Bagging: Bootstrap aggregating

- Bagging uses the same base machine-learning algorithm to train base estimators and training base estimators on replicates of the data set.
- During training, bootstrap sampling, or sampling with replacement, is used to generate replicates of the training data set. This ensures that base learners trained on each of the replicates are also different from each other.
- During prediction, model aggregation is used to combine the predictions of the individual base learners into one ensemble prediction. For classification tasks, we can combine individual predictions using majority voting. For regression tasks, we can combine individual predictions using simple averaging.





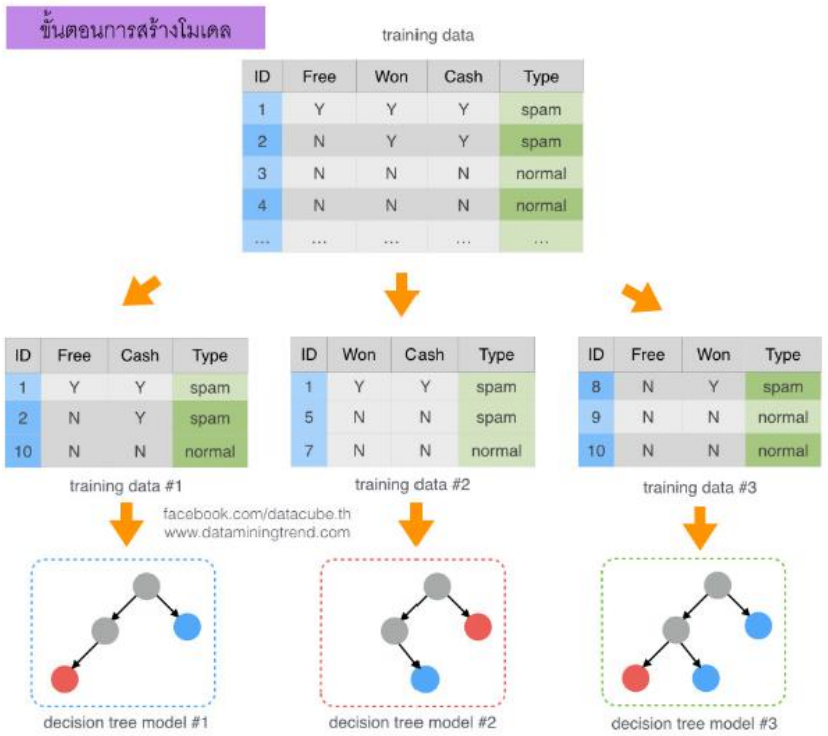
THE OUT-OF-BAG SAMPLE

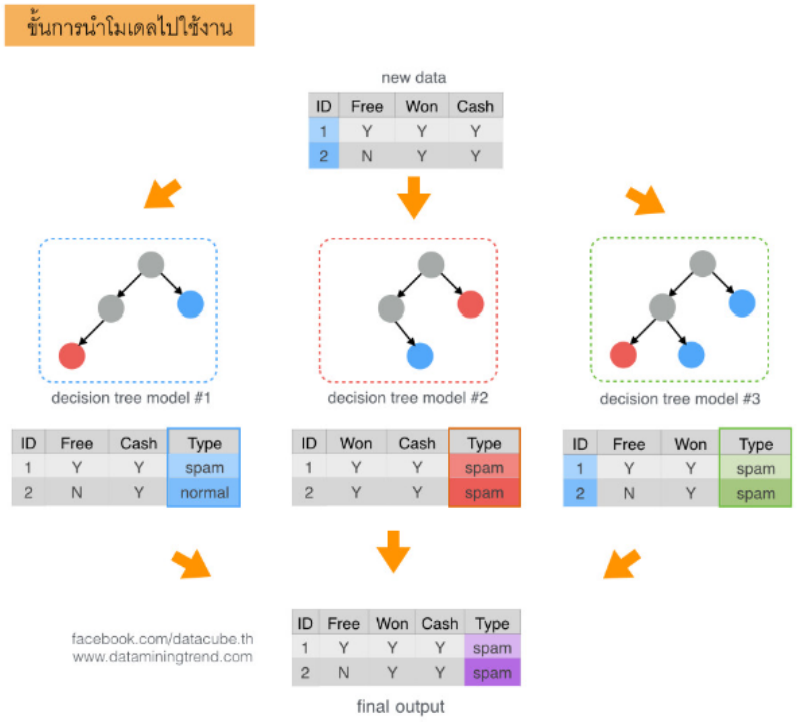


- The OOB sample is effectively a held-out test set and can be used to evaluate the ensemble without the need for a separate validation set or even a cross-validation procedure. This is great because it allows us to utilize data more efficiently during training.
- The error estimate computed using OOB instances is called the OOB error or the OOB score.
- To summarize: after one round of bootstrap sampling, we get one bootstrap sample (for training a base estimator) and a corresponding OOB sample (to evaluate that base estimator).
- The averaged OOB error is a good estimate of the performance of the overall ensemble.

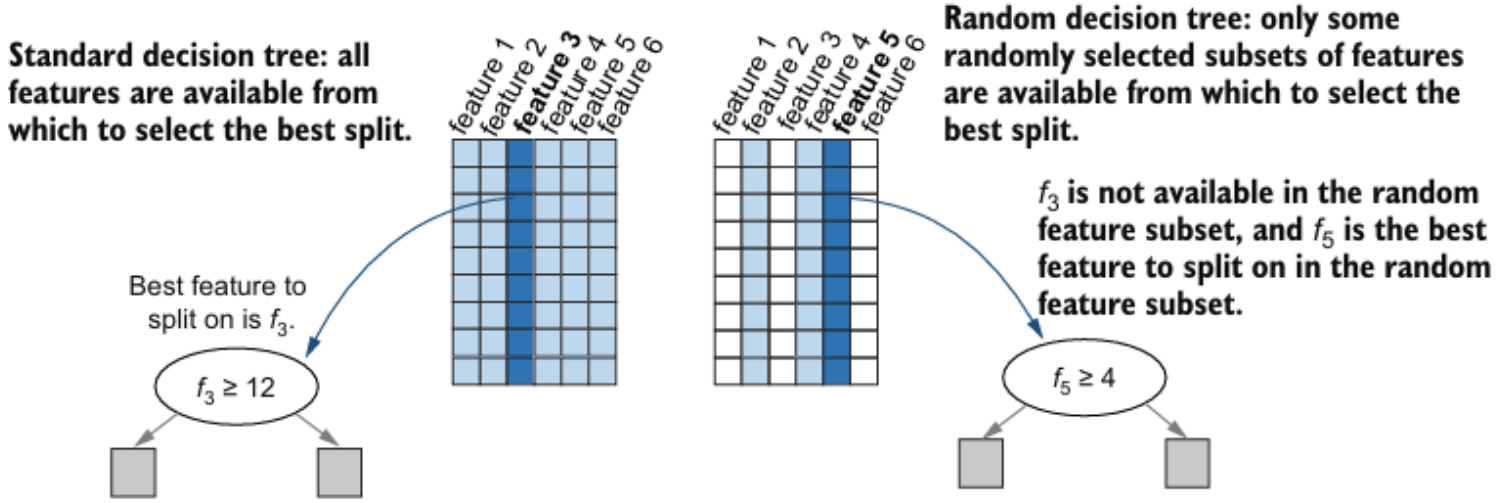
Random Forests

- Random forest specifically refers to an ensemble of randomized decision trees constructed using bagging.
- Random forests perform bootstrap sampling to generate a training subset (exactly like bagging), and then use randomized decision trees as base estimators.

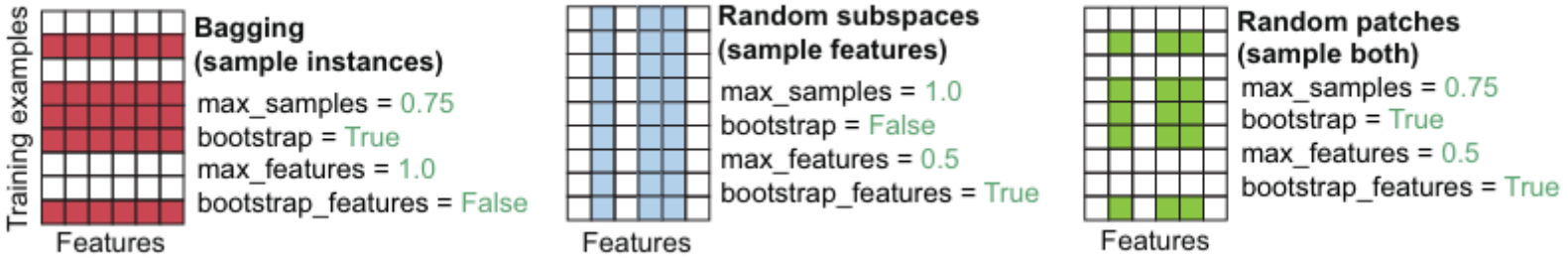




Random Forests



- This randomization occurs every time a decision node is constructed. Thus, even if we use the same data set, we'll obtain a different randomized tree each time we train.
- When randomized tree learning (with a random sampling of features) is combined with bootstrap sampling (with a random sampling of training examples), we obtain an ensemble of randomized decision trees, known as a random decision forest or simply random forest.



Extra Trees - Extremely randomized trees

- Like Random Forests, Extra Trees builds multiple decision trees and combines their predictions.
- Extra Trees take the idea of randomized decision trees to the extreme by selecting not just the splitting variable from a random subset of features but also the splitting threshold!
- For each chosen feature, a random threshold (split point) is selected instead of searching for the optimal one.
- During training, the ET will construct trees over every observation in the dataset (without bootstrap sampling) but with different subsets of features.

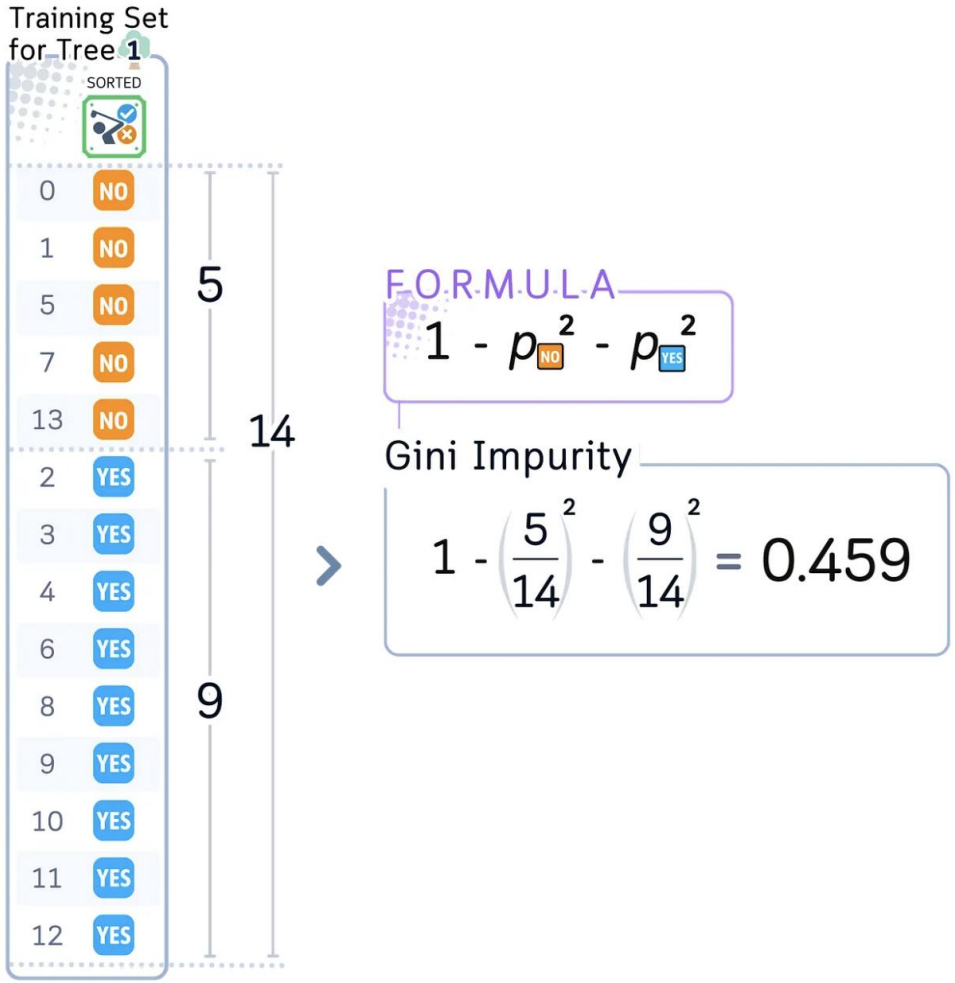
Extra Trees - Extremely randomized trees

Day	Outlook	Temperature	Humidity	Windy	Play Tennis Decision
1	sunny	hot	high	false	no
2	sunny	hot	high	true	no
3	overcast	hot	high	false	yes
4	rain	mild	high	false	yes

Day	Outlook - sunny	Outlook - rain	Outlook - overcast
1	1	0	0
2	1	0	0
3	0	0	1
4	0	1	0

Extra Trees - Extremely randomized trees

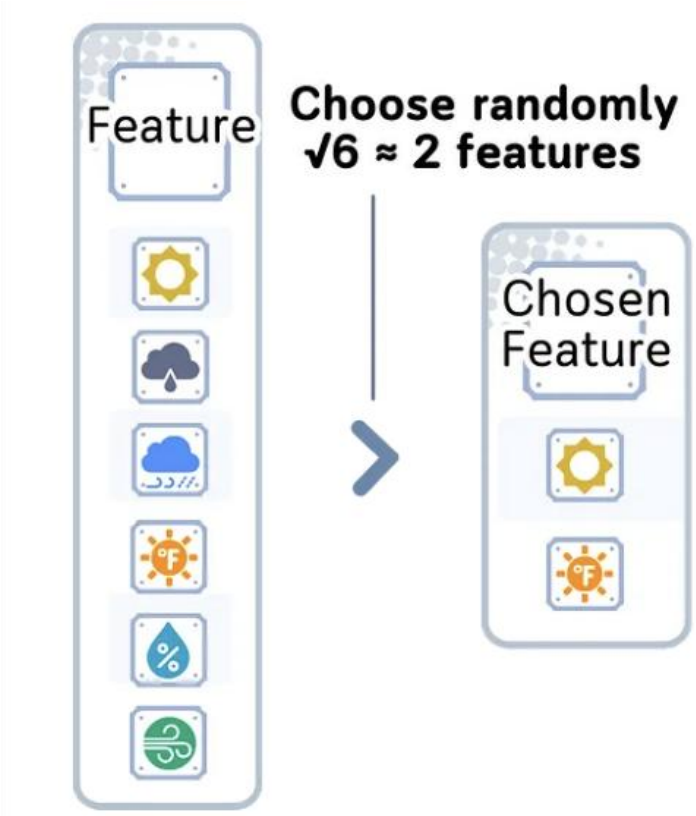
- Start at root node with complete dataset
- Calculate initial node impurity



https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75

Extra Trees - Extremely randomized trees

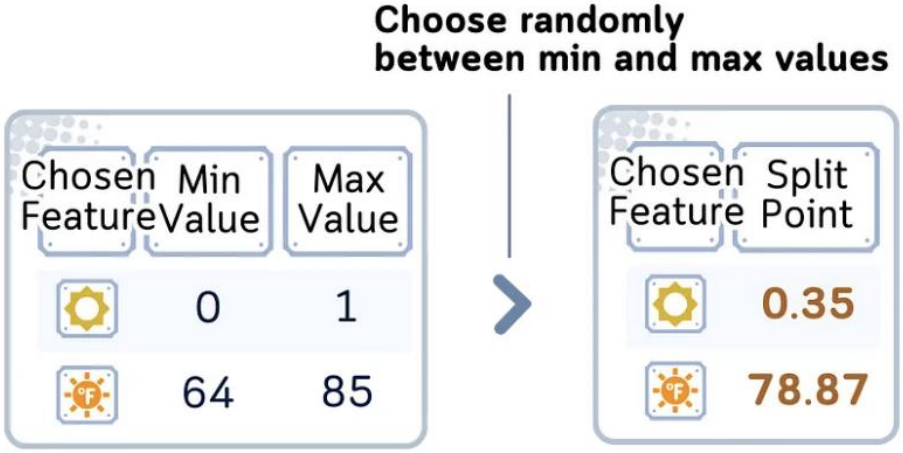
- Select random subset of features from total available features:
  - Classification: √n_features
  - Regression: n_features/3



https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75

Extra Trees - Extremely randomized trees

- For each selected feature randomly generate split points.

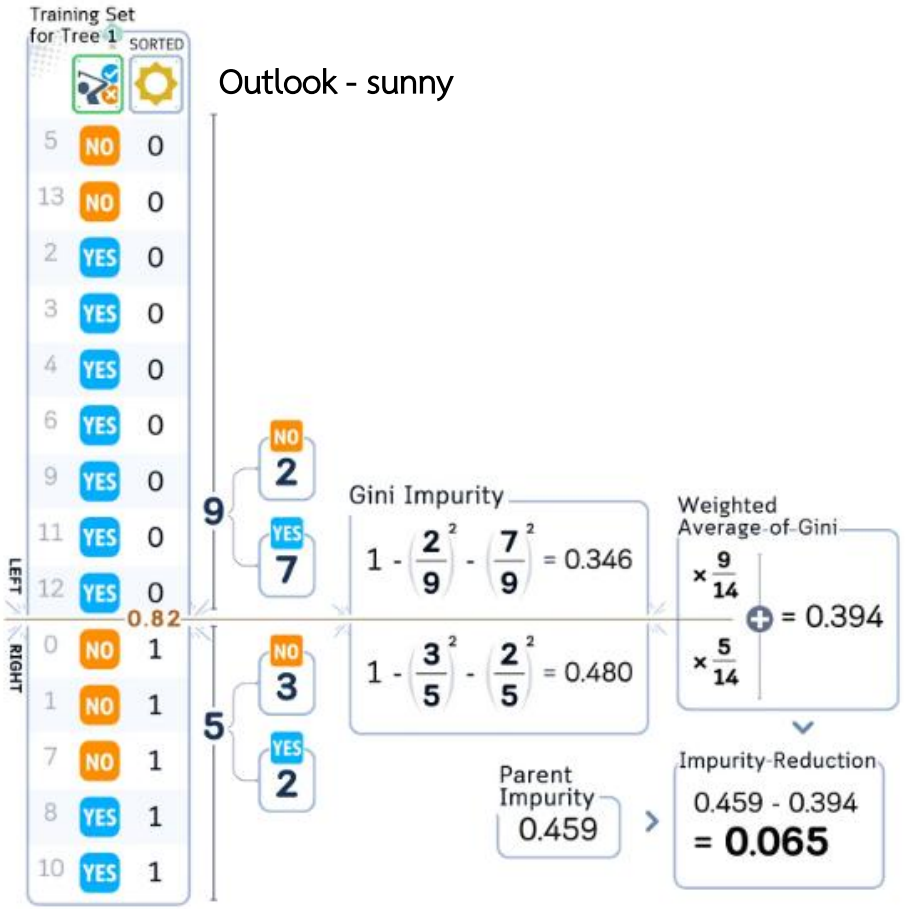


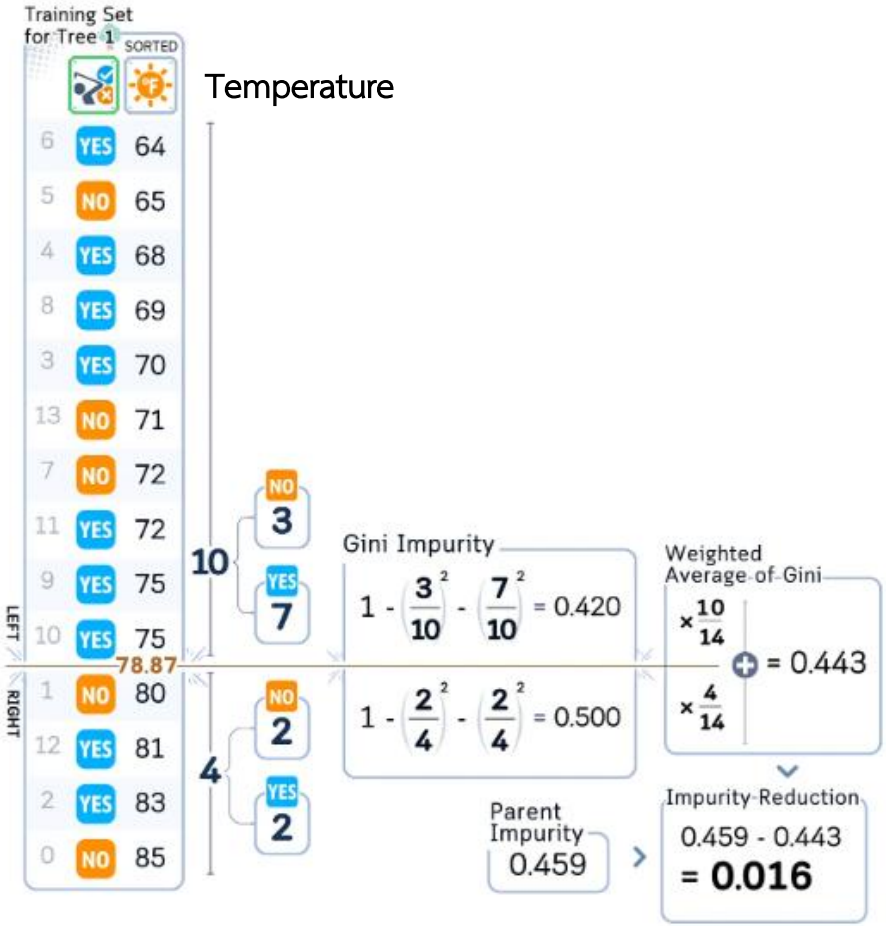
Extra Trees picks random split points between feature ranges (like 0.35 between 0–1) instead of testing all possible splits like Random Forest does. This unique random splitting is what makes Extra Trees ‘extremely’ randomized.

https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75

Extra Trees - Extremely randomized trees

- For each random split point, calculate the usual impurity reduction

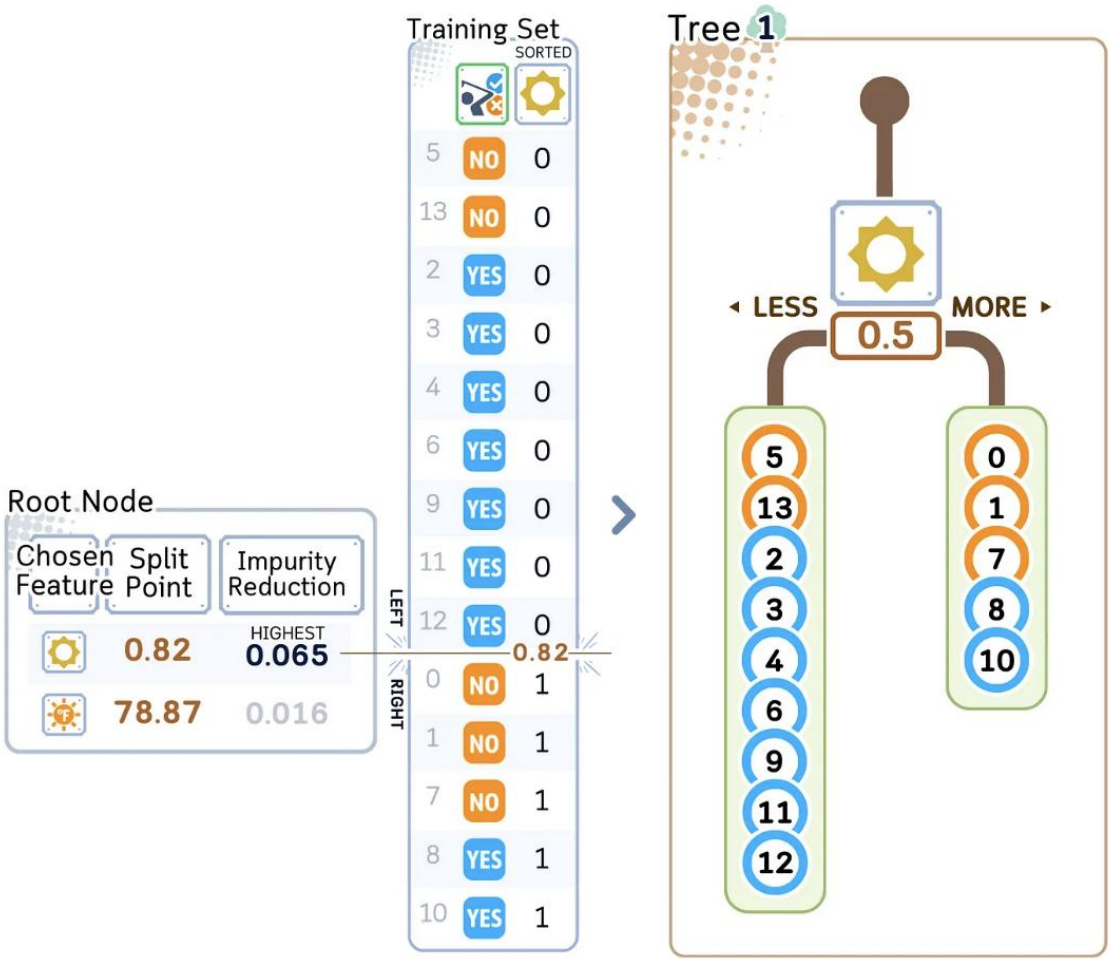




https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75

Extra Trees - Extremely randomized trees

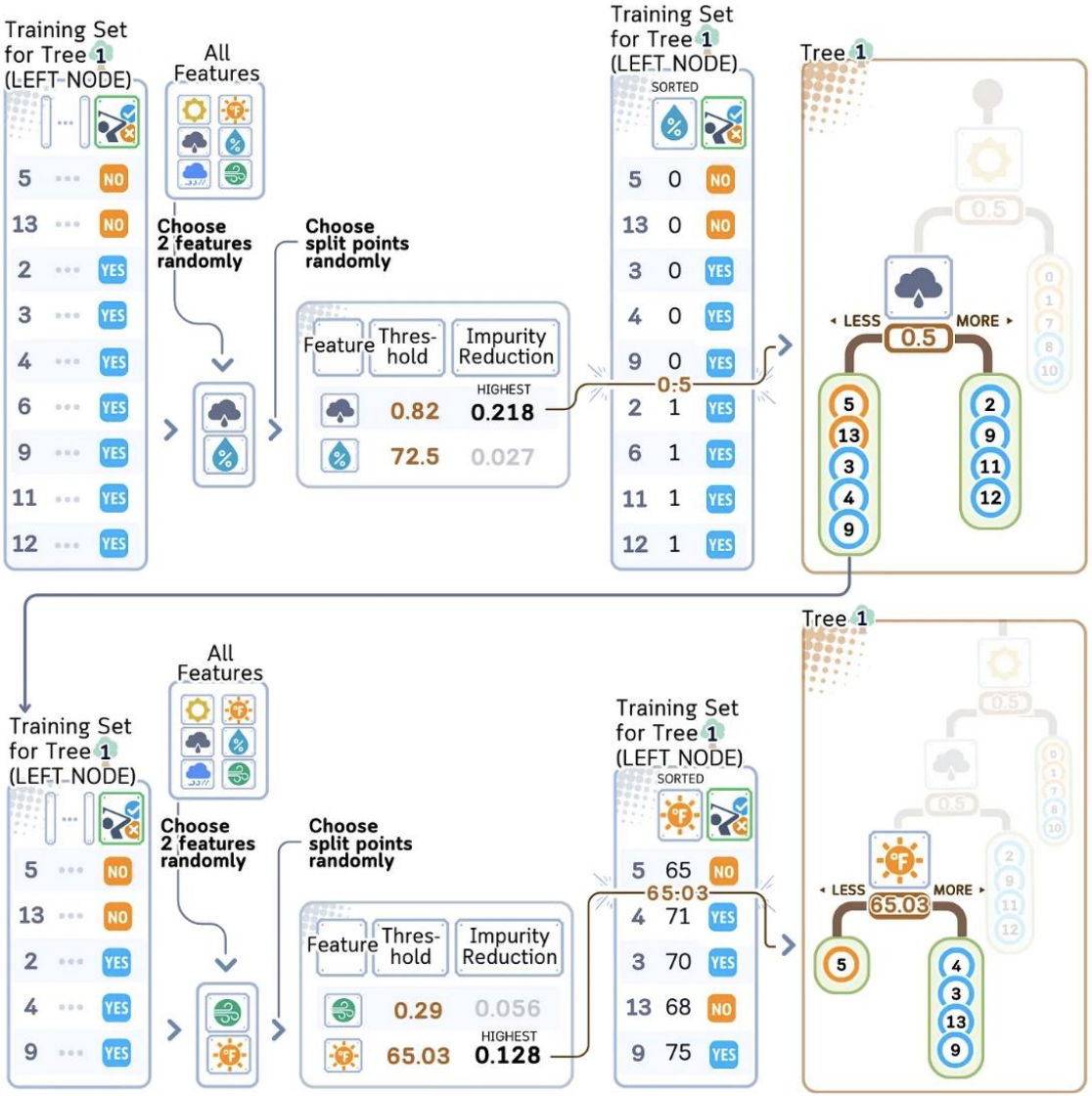
- Split the current node data using the feature and split point that gives the highest impurity reduction among the random splits



https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75

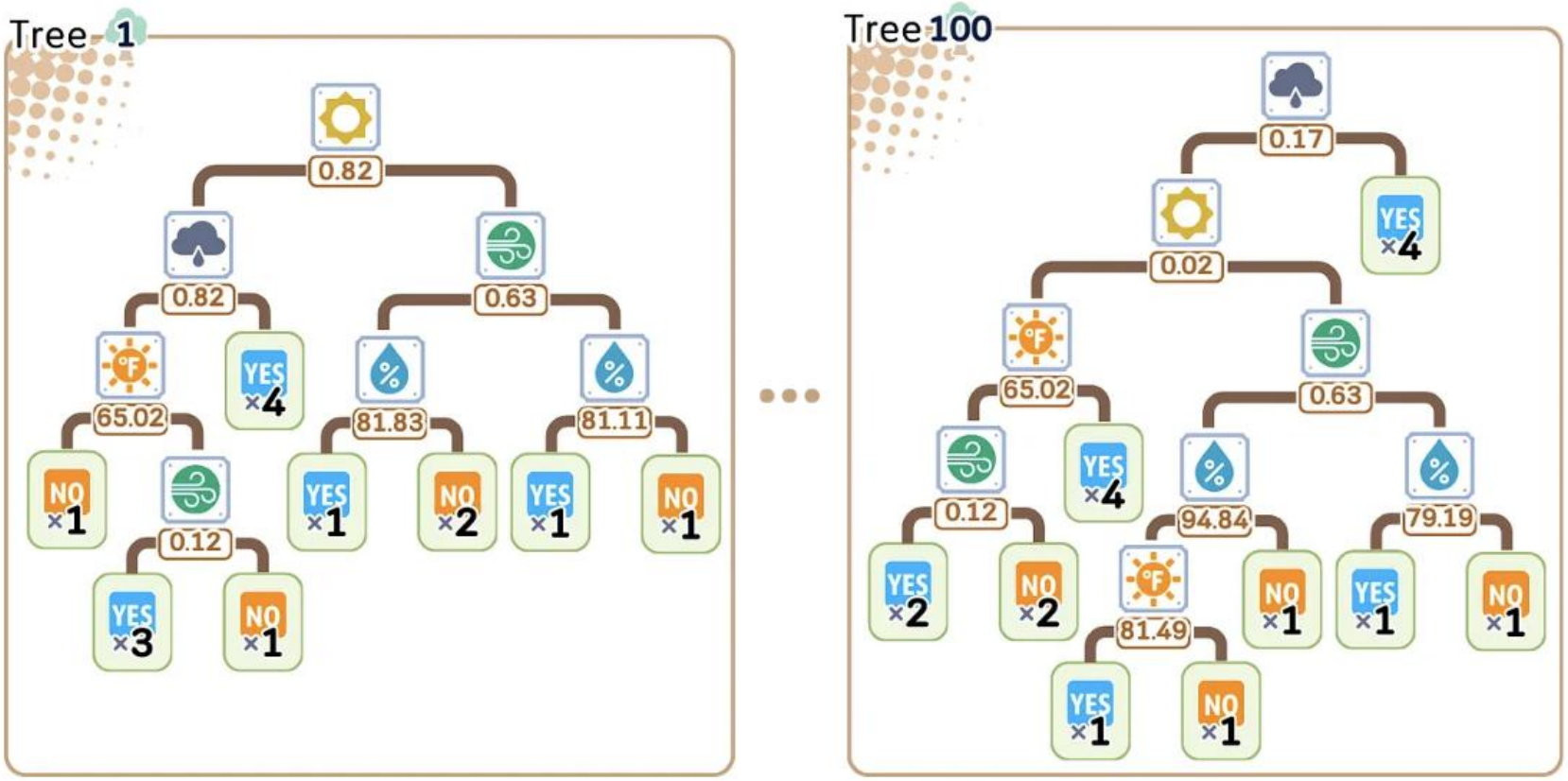
Extra Trees - Extremely randomized trees

- For each child node, repeat the process



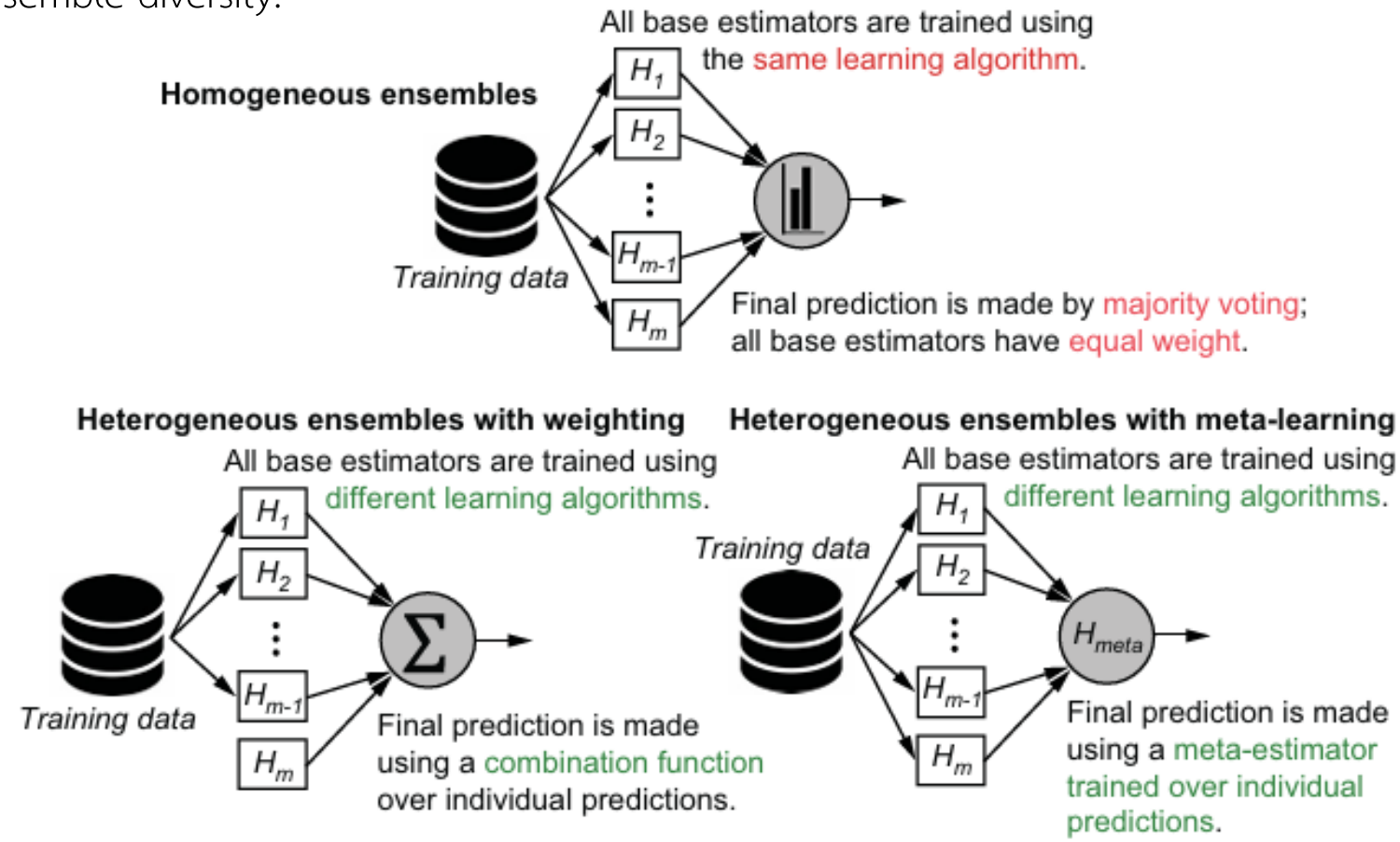
Extra Trees - Extremely randomized trees

- Tree Construction



Heterogeneous ensemble

- Heterogeneous ensemble methods use different base-learning algorithms to directly ensure ensemble diversity.

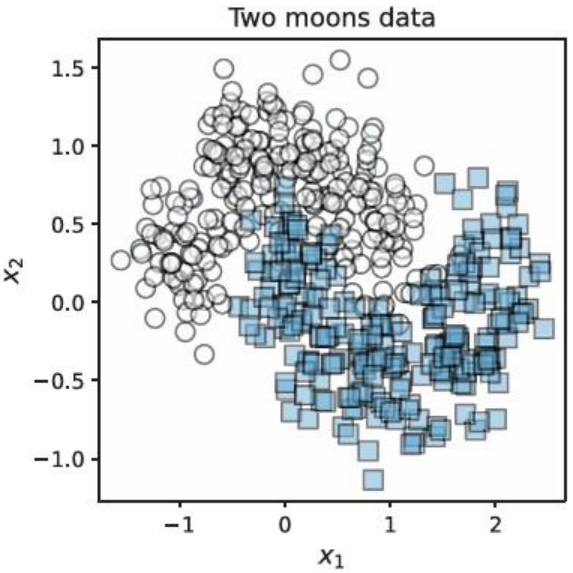


Heterogeneous ensembles come in two flavors,

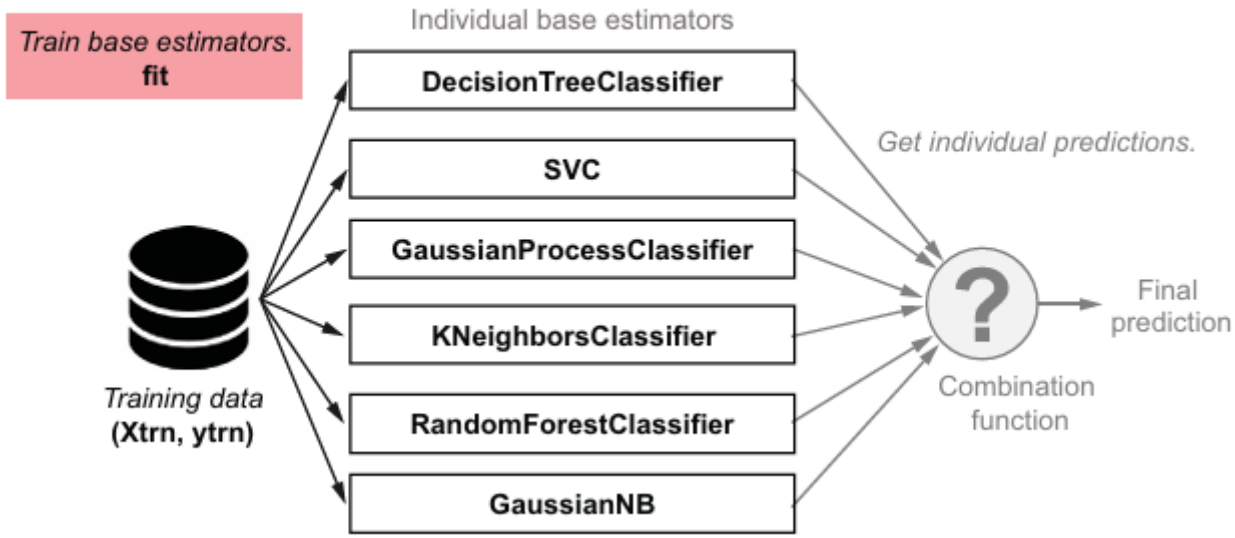
depending on how they combine individual base-estimator predictions into a final prediction

Base estimators for heterogeneous ensembles

- The key is to ensure that we choose learning algorithms that are different enough to produce a diverse collection of estimators. The more diverse our set of base estimators, the better the resulting ensemble will be.
- Example:



600 synthetic training examples equally distributed into two classes: Class 0 (circles) and Class 1 (squares)



Base estimators for heterogeneous ensembles

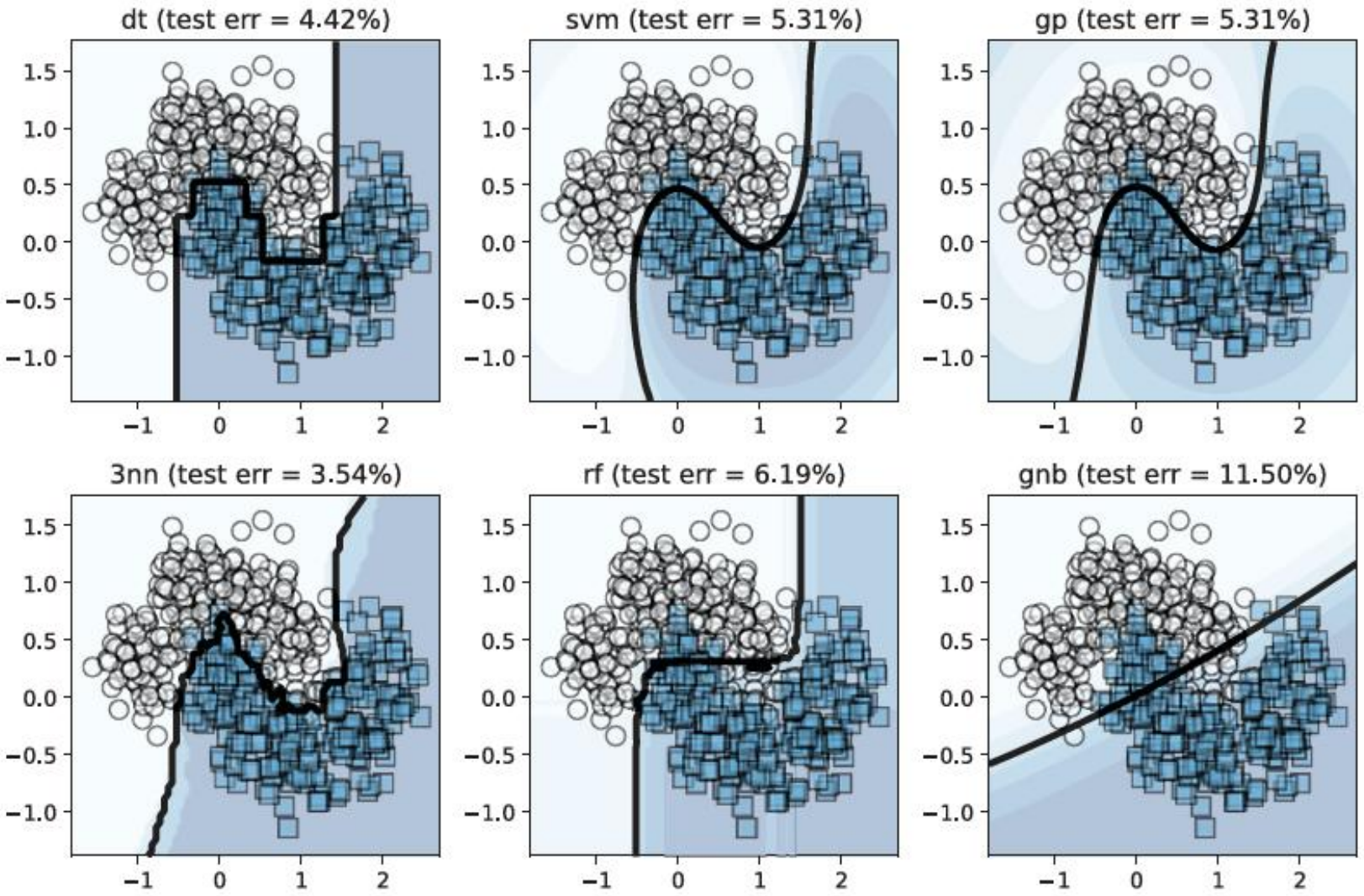
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB

estimators = [
    ('dt', DecisionTreeClassifier (max_depth=5)),
    ('svm', SVC(gamma=1.0, C=1.0, probability=True)),
    ('gp', GaussianProcessClassifier(RBF(1.0))),
    ('3nn', KNeighborsClassifier(n_neighbors=3)),
    ('rf',RandomForestClassifier(max_depth=3, n_estimators=25)),
    ('gnb', GaussianNB())]

def fit(estimators, X, y):
    for model, estimator in estimators:
        estimator.fit(X, y)
    return estimators
```

Initializes several base-learning algorithms

Fits base estimators on the training data using these different learning algorithms

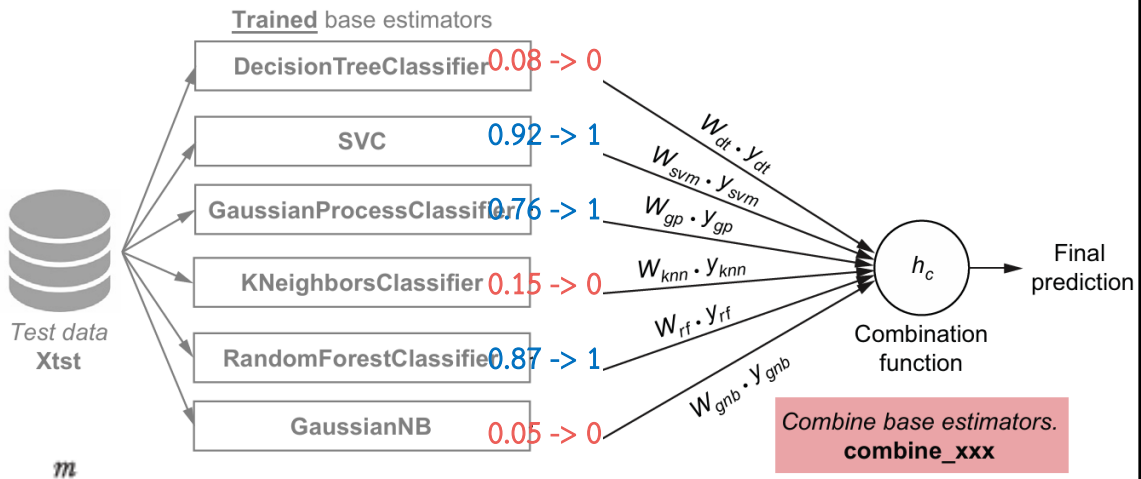
Base estimators for heterogeneous ensembles

- Individual predictions of base estimators



Combining predictions by weighting

- Accuracy weighting



$$y_{final} = w_1 \cdot y_1 + w_2 \cdot y_2 + \cdots + w_m \cdot y_m = \sum_{t=1}^{m} w_t \cdot y_t$$

$$w_i = \frac{\text{accuracy}_i}{\sum \text{accuracy}_i}$$

The test set accuracy of dt is a_{dt} = 1 – 0.0442 = 0.9558 — w_{dt} = 0.171

The test set accuracy of svm is a_{svm} = 1 – 0.0531 = 0.9469 — w_{svm} = 0.169

The test set accuracy of gp is a_{gp} = 1 – 0.0531 = 0.9469 — w_{gp} = 0.169

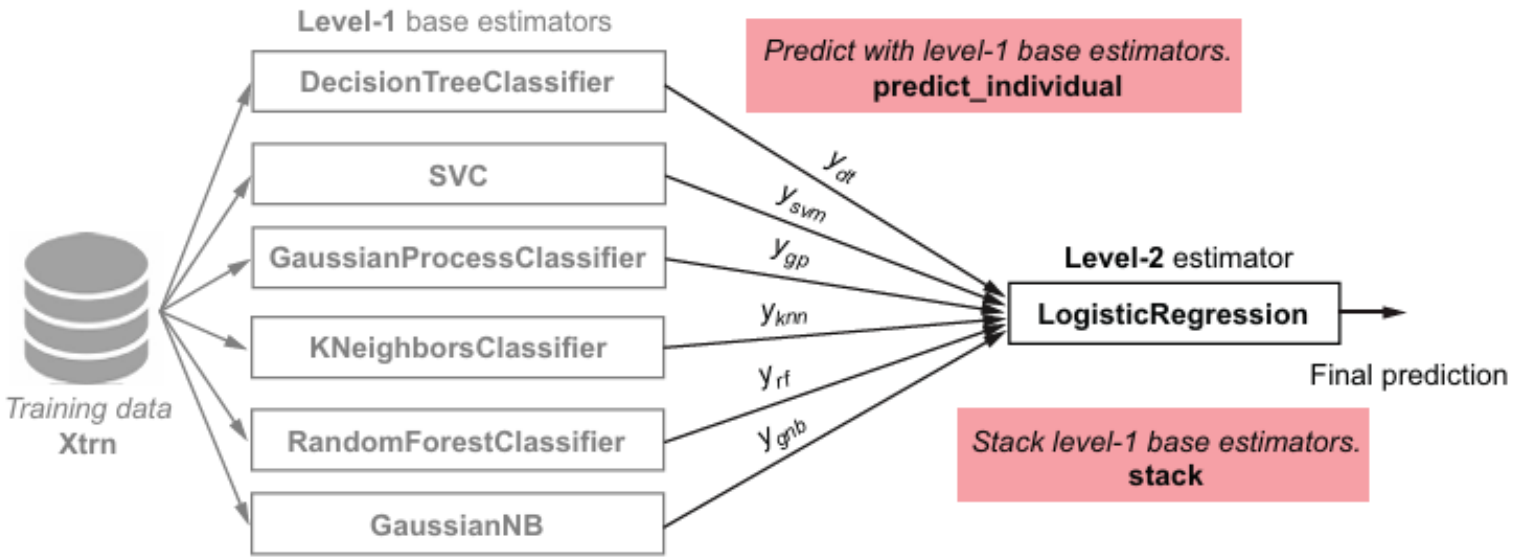
The test set accuracy of 3nn is a_{3nn} = 1 – 0.0354 = 0.9646 — w_{3nn} = 0.172

The test set accuracy of rf is a_{rf} = 1 – 0.0619 = 0.8939 — w_{rf} = 0.160

The test set accuracy of gnb is a_{gnb} = 1 – 0.155 = 0.885 — w_{gnb} = 0.158

Combining predictions by meta-learning

- The predictions of the base estimators are given as inputs to a second-level learning algorithm.
- Stacking is the most common meta-learning method and gets its name because it stacks a second classifier on top of its base estimators.
- The general stacking procedure has two steps:
  - Level 1: Fit base estimators on the training data. (this step is the same as before)
  - Level 2: Construct a new data set from the predictions of the base classifiers, which become meta-features. Meta-features can either be the predictions or the probability of predictions.

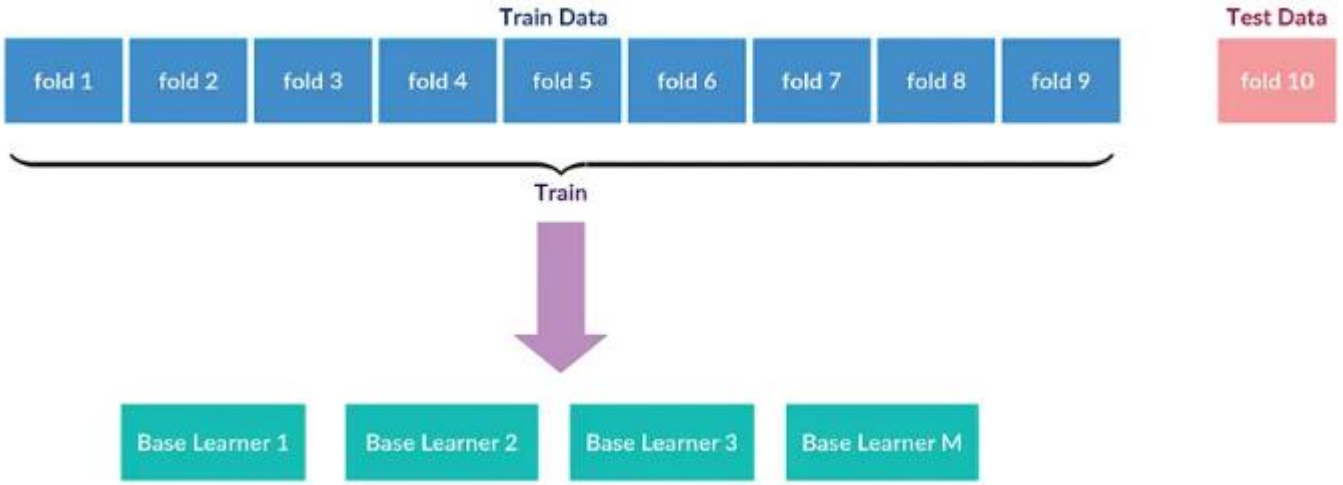


The level-2 estimator here can be trained using any base-learning algorithm

Combining predictions by meta-learning

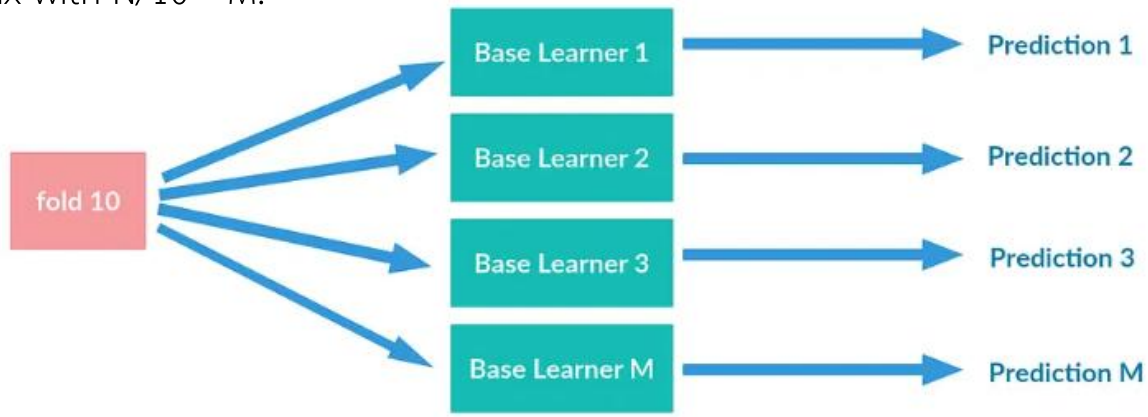
- Stacking with cross validation
- CV (cross validation) is a model validation and evaluation procedure that is commonly employed to simulate out-of-sample testing, tune model hyperparameters, and test the effectiveness of machine-learning models.
- Divide the Train set into k-number of folds.

- Keep one of the folds aside, and train the base models, using the remaining folds. The kept-aside fold will be treated as the testing data for this step.



Combining predictions by meta-learning

- Stacking with cross validation
- Then, predict the valued for the remaining fold (10th fold), using all the M models trained. So this will result in M number of predictions for each data point in the 10th fold. Now we have N/10 data points sets (prediction sets), each with M number of fields (predictions coming from the M number of models). i.e: matrix with N/10 * M.



Combining predictions by meta-learning

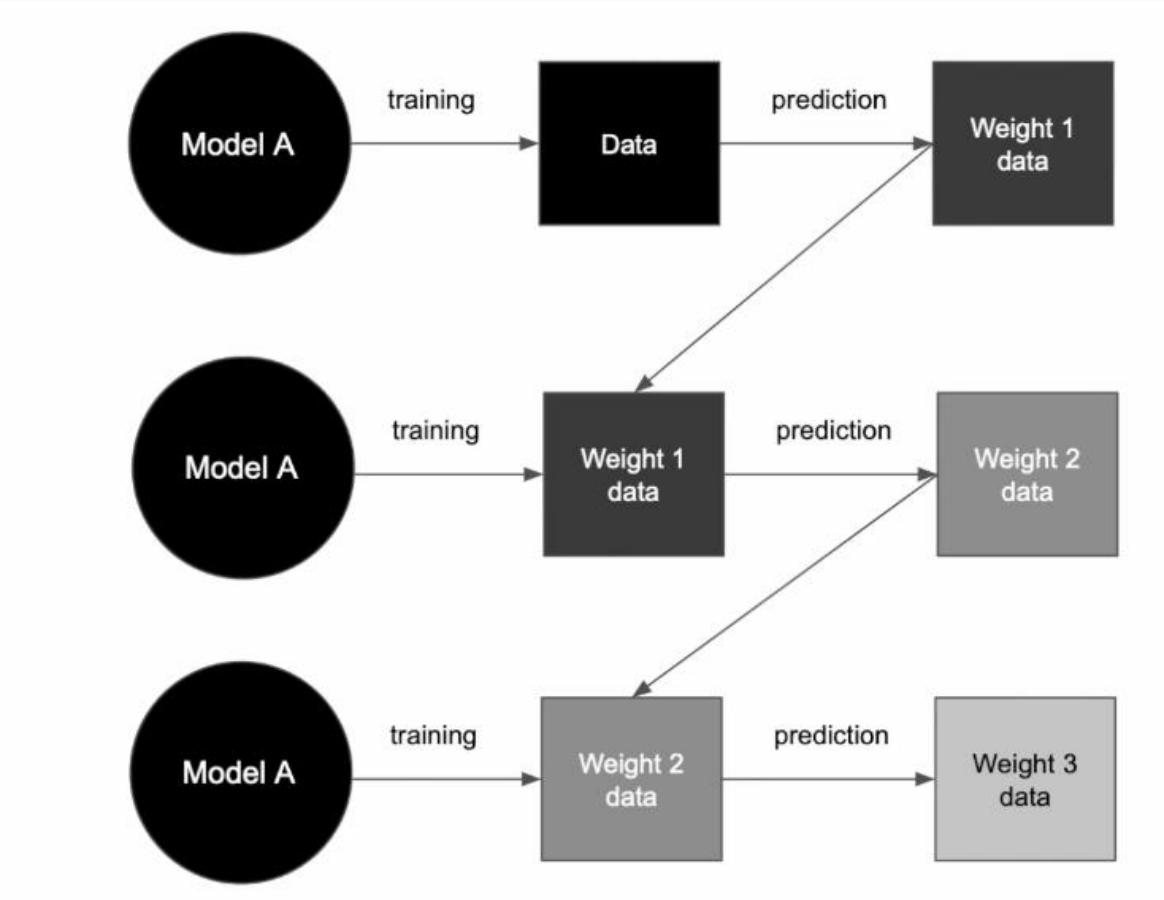
- Stacking with cross validation
- Iterate the previous process by changing the kept-out fold (from 1 to N). At the end of all the iterations, we would be having N number of prediction results sets, which corresponds to each data point in the original training set, along with the actual value of the field we predict.
- This will be the input data set to our meta-learner

Data Point #	prediction from base learner 1	prediction from base learner 2	prediction from base learner 3	prediction from base learner M	actual
1	$y_{1\hat{1}}$	$y_{1\hat{2}}$	$y_{1\hat{3}}$	$y_{1\hat{M}}$	y_1
2	$y_{2\hat{1}}$	$y_{2\hat{2}}$	$y_{2\hat{3}}$	$y_{2\hat{M}}$	y_2
...	...	...	...	...	...
N	$y_{N\hat{1}}$	$y_{N\hat{2}}$	$y_{N\hat{3}}$	$y_{N\hat{M}}$	y_N

Sequential ensembles

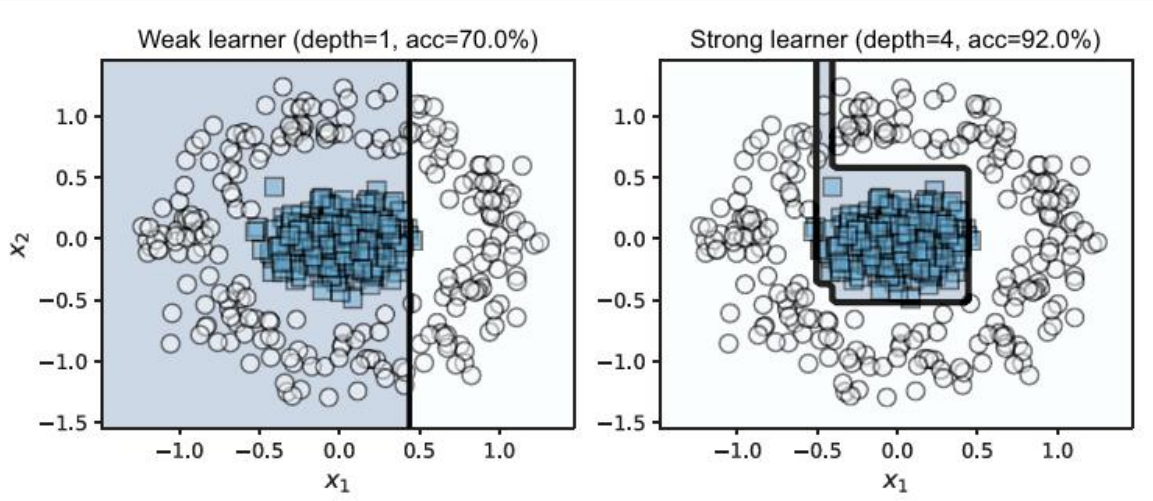
Sequential ensembles: Adaptive boosting

- Sequential ensembles exploit the dependence of base estimators
- During learning, sequential ensembles train a new base estimator in such a manner that it minimizes mistakes made by the base estimator trained in the previous step.



Sequential ensembles: Adaptive boosting

- Base estimators in parallel ensembles are typically strong learners, while in sequential ensembles, they are weak learners. Sequential ensembles aim to combine several weak learners into one strong learner.
- Weak Learners
  - A strong learner is a good model (or estimator). In contrast, a weak learner is a very simple model that doesn't perform that well. The only requirement of a weak learner (for binary classification) is that it performs better than random guessing.

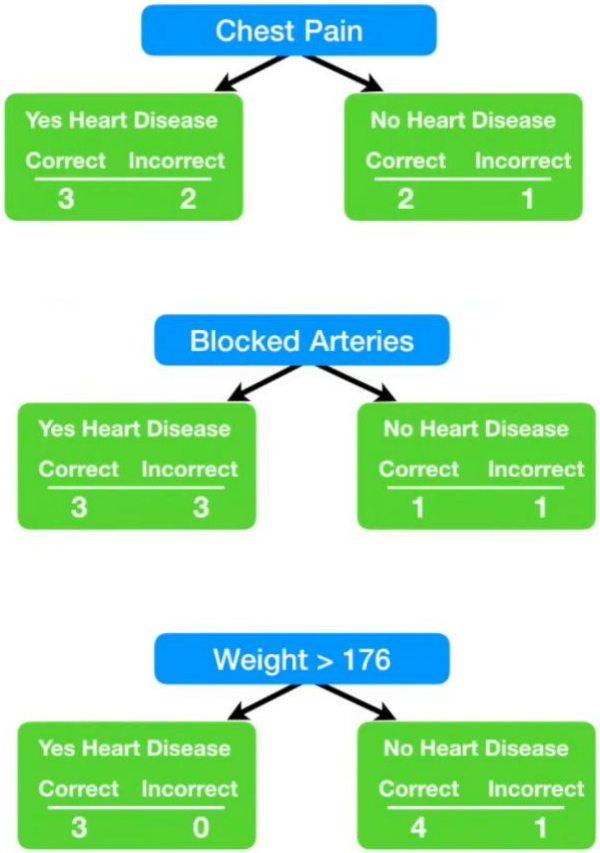


Decision stumps (trees of depth 1, left) are commonly used as weak learners in sequential ensemble methods such as boosting. As tree depth increases, a decision stump grows into a decision tree, becoming a stronger classifier, and its performance improves.

Sequential ensembles: Adaptive boosting

- Decision stumps (trees of depth 1)

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes



Sequential ensembles: Adaptive boosting

- AdaBoost: Adaptive boosting
- AdaBoost is an adaptive algorithm: at every iteration, it trains a new base estimator that fixes the mistakes made by the previous base estimator.
- It needs some way to ensure that the base-learning algorithm prioritizes misclassified training examples.
- AdaBoost does this by maintaining weights over individual training examples.
- Misclassified examples have higher weights, while correctly classified examples have lower weights.

Sequential ensembles: Adaptive boosting

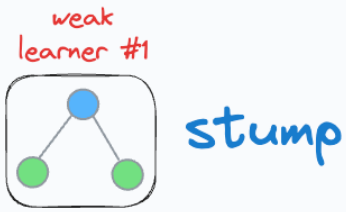
- AdaBoost: Adaptive boosting

feature 1	feature 2	feature 3	target	weight
1.2	2.3	True	class A	0.2
-0.3	0.6	False	class B	0.2
0.7	1.8	False	class A	0.2
4.5	-3.5	True	class B	0.2
-0.4	0.4	True	class A	0.2

$$w_i^{(1)} = \frac{1}{m}$$

Step 1: Train a weak learner

- In AdaBoost, every decision tree has a unit depth, and they are also called stumps.

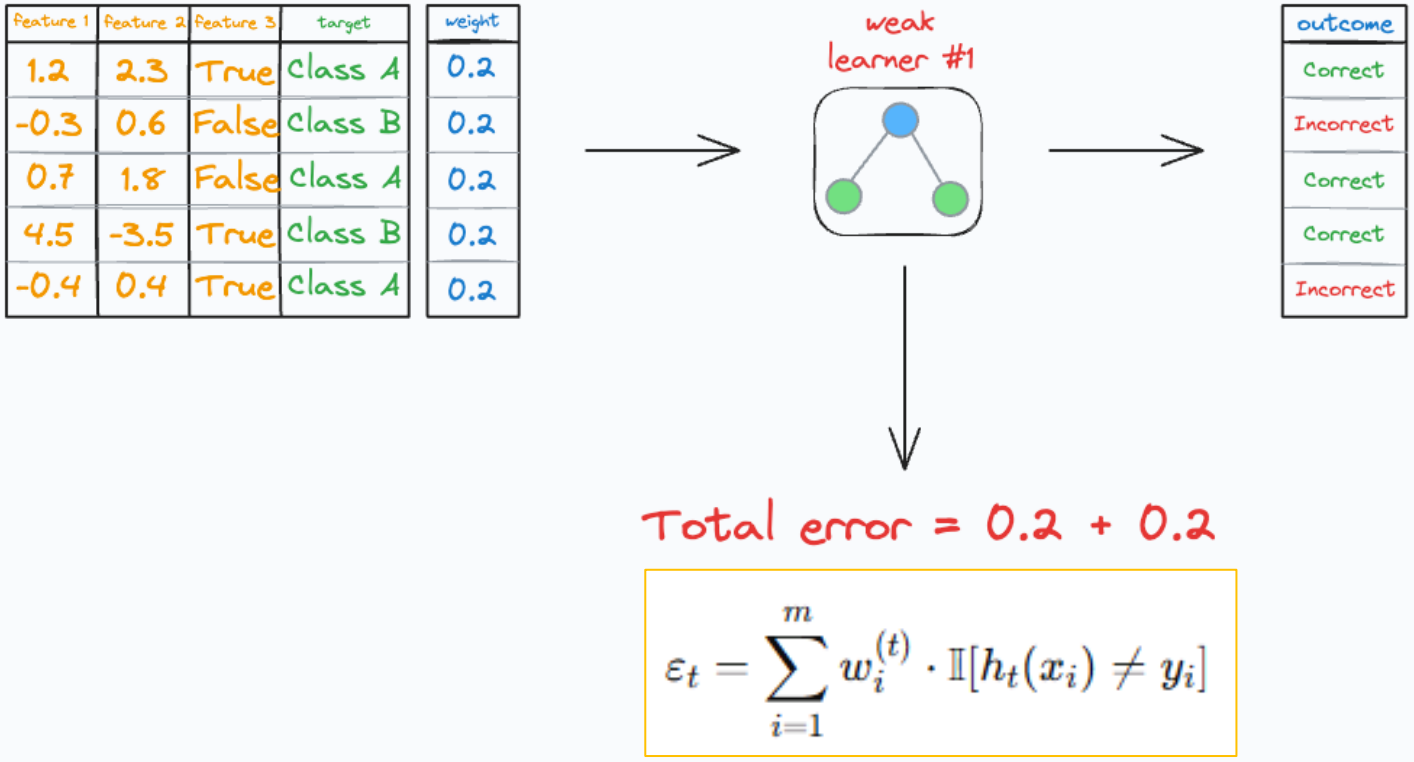


Sequential ensembles: Adaptive boosting

- AdaBoost: Adaptive boosting

Step 2: Calculate the learner's cost

- The total cost (or error/loss) of this specific weak learner is the sum of the weights of the incorrect predictions.



$$\varepsilon_t = \sum_{i=1}^{m} w_i^{(t)} \cdot \mathbb{I}[h_t(x_i) \neq y_i]$$

Sequential ensembles: Adaptive boosting

- AdaBoost: Adaptive boosting

Step 3: Calculate the learner's importance or voting power

- If the weak learner has a high error, it must have a lower importance.
- If the weak learner has a low error, it must have a higher importance.

$$\text{Importance} = \frac{1}{2} \cdot log(\frac{1 - \text{Total error}}{\text{Total error}})$$

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

- This function is only defined between [0,1]

Sequential ensembles: Adaptive boosting

- AdaBoost: Adaptive boosting

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i))$$

Step 4: Reweigh the training instances

- All the correct predictions are weighed down as follows:

$$w := w * e^{\text{-Importance}}$$

- All the incorrect predictions are weighed up as follows:

$$w := w * e^{\text{Importance}}$$

- Normalize weight เพื่อให้รวมแล้วเป็น 1 (เหมือน probability distribution)

feature 1	feature 2	feature 3	target	weight	outcome
1.2	2.3	True	Class A	0.2	Correct
-0.3	0.6	False	Class B	0.2	Incorrect
0.7	1.8	False	Class A	0.2	Correct
4.5	-3.5	True	Class B	0.2	Correct
-0.4	0.4	True	Class A	0.2	Incorrect

reweigh and normalize →

$$w_i^{(t+1)} = \frac{w_i^{(t+1)}}{\sum_{j=1}^{m} w_j^{(t+1)}}$$

weight
0.13
0.3
0.13
0.13
0.3

Sequential ensembles: Adaptive boosting

- AdaBoost: Adaptive boosting

Step 5: Sample from reweighed dataset.

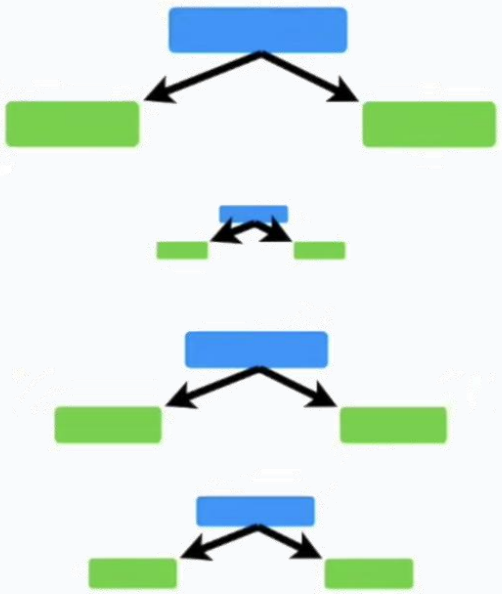
feature 1	feature 2	feature 3	target	weight
1.2	2.3	True	Class A	0.13
-0.3	0.6	False	Class B	0.3
0.7	1.8	False	Class A	0.13
4.5	-3.5	True	Class B	0.13
-0.4	0.4	True	Class A	0.3

Next, go back to step 1 — Train the next weak learner.

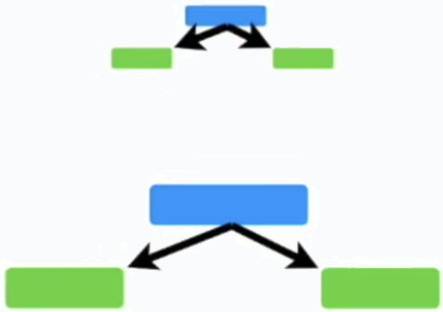
feature 1	feature 2	feature 3	target	weight
1.2	2.3	True	Class A	0.13
-0.3	0.6	False	Class B	0.3
-0.3	0.6	False	Class B	0.3
-0.4	0.4	True	Class A	0.3
-0.4	0.4	True	Class A	0.3

Sequential ensembles: Adaptive boosting

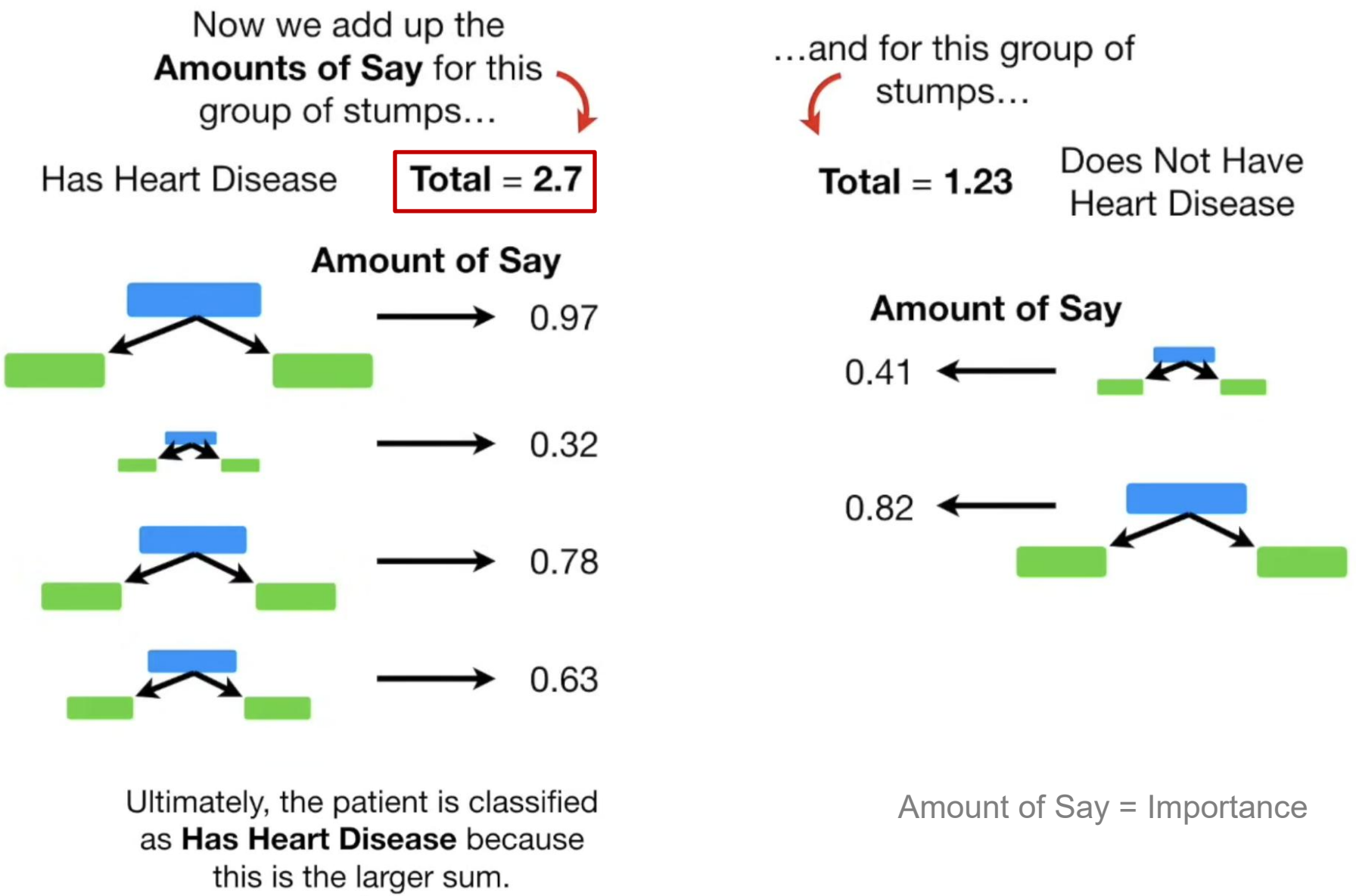
Imagine that these stumps classified a patient as **Has Heart Disease**…



…and these stumps classified the patient as **Does Not Have Heart Disease**.



Sequential ensembles: Adaptive boosting

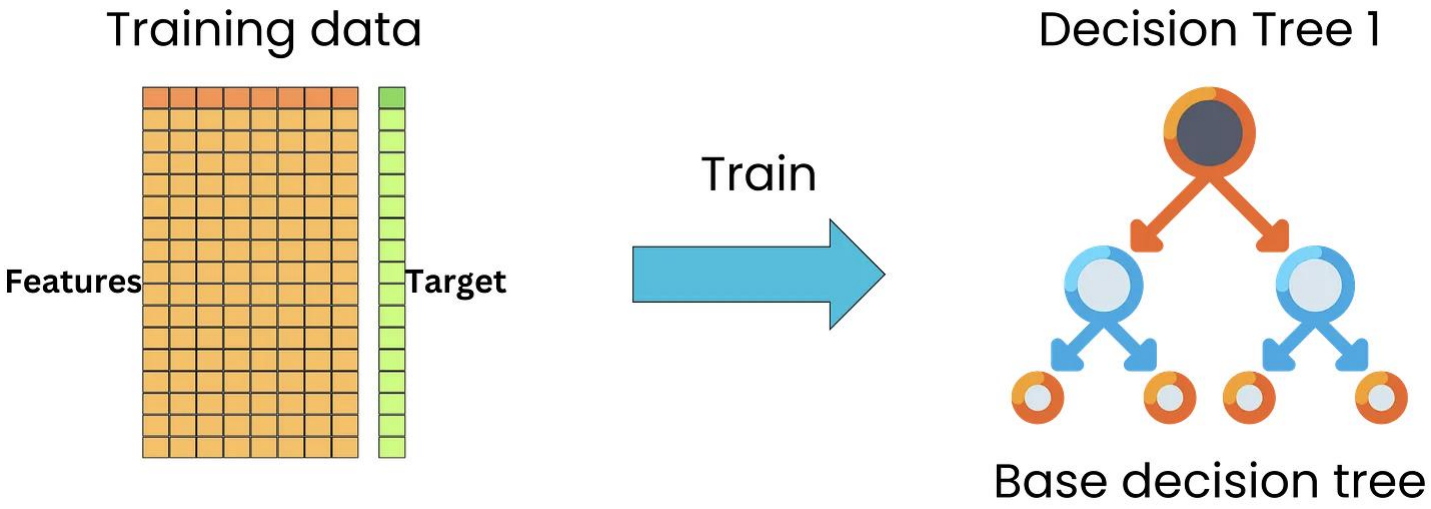


SEQUENTIAL ENSEMBLES:

GRADIENT BOOSTING

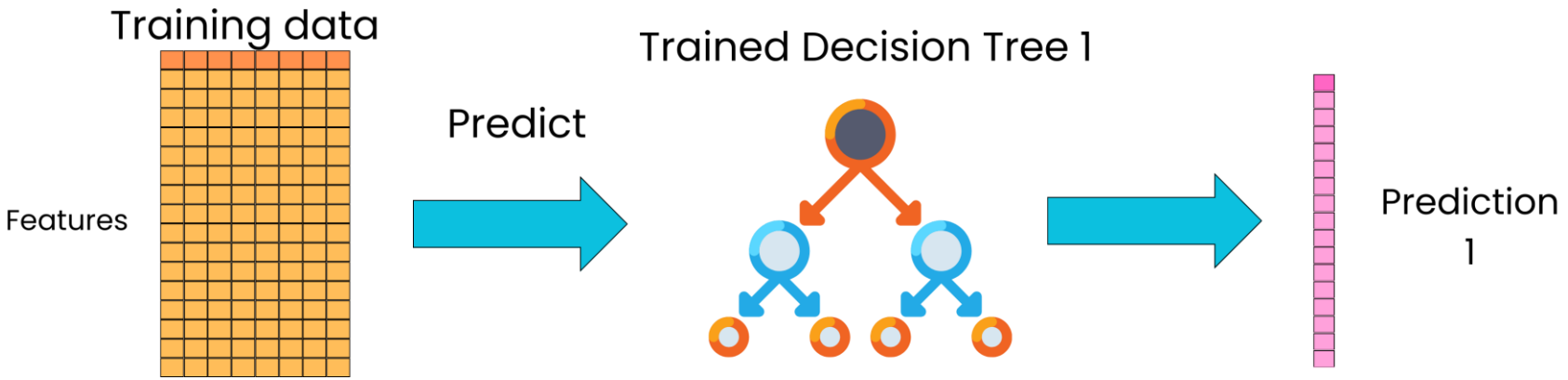
How the Gradient Boosting Algorithm Works

- The gradient-boosted trees algorithm is an iterative algorithm. In the first iteration, we need to construct an initial simple learning on the training data. This gives our base tree.



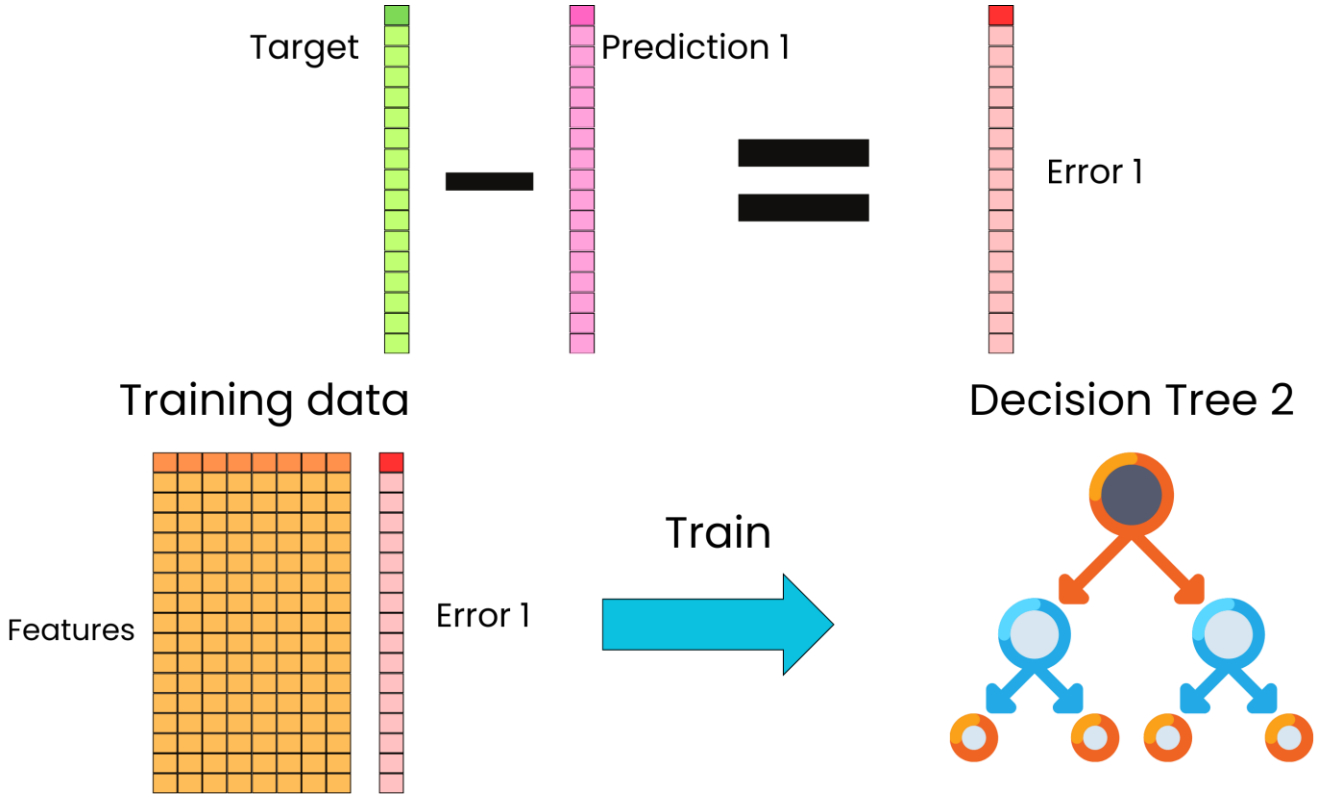
How the Gradient Boosting Algorithm Works

- By using that base tree, we can then use it to predict on the same training data, which will lead to predictions from that training data.



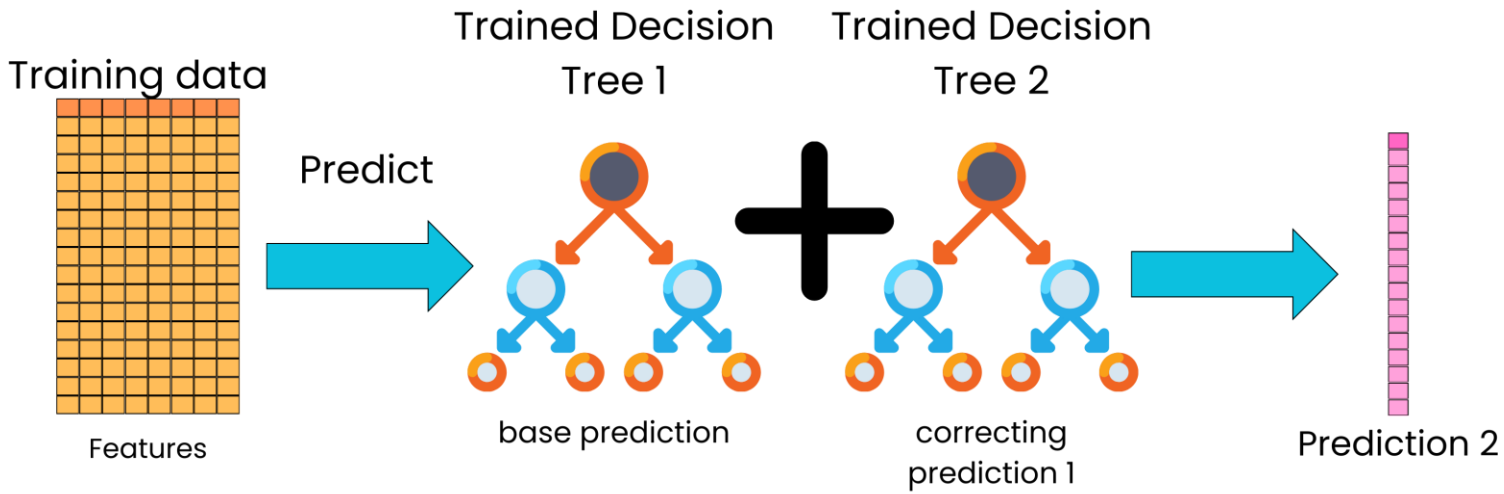
How the Gradient Boosting Algorithm Works

- Now, we can compare that prediction to the target. We can look at the error we are making by predicting on the training data, and this time, we are going to use the error as a new target for a new tree we are going to train.



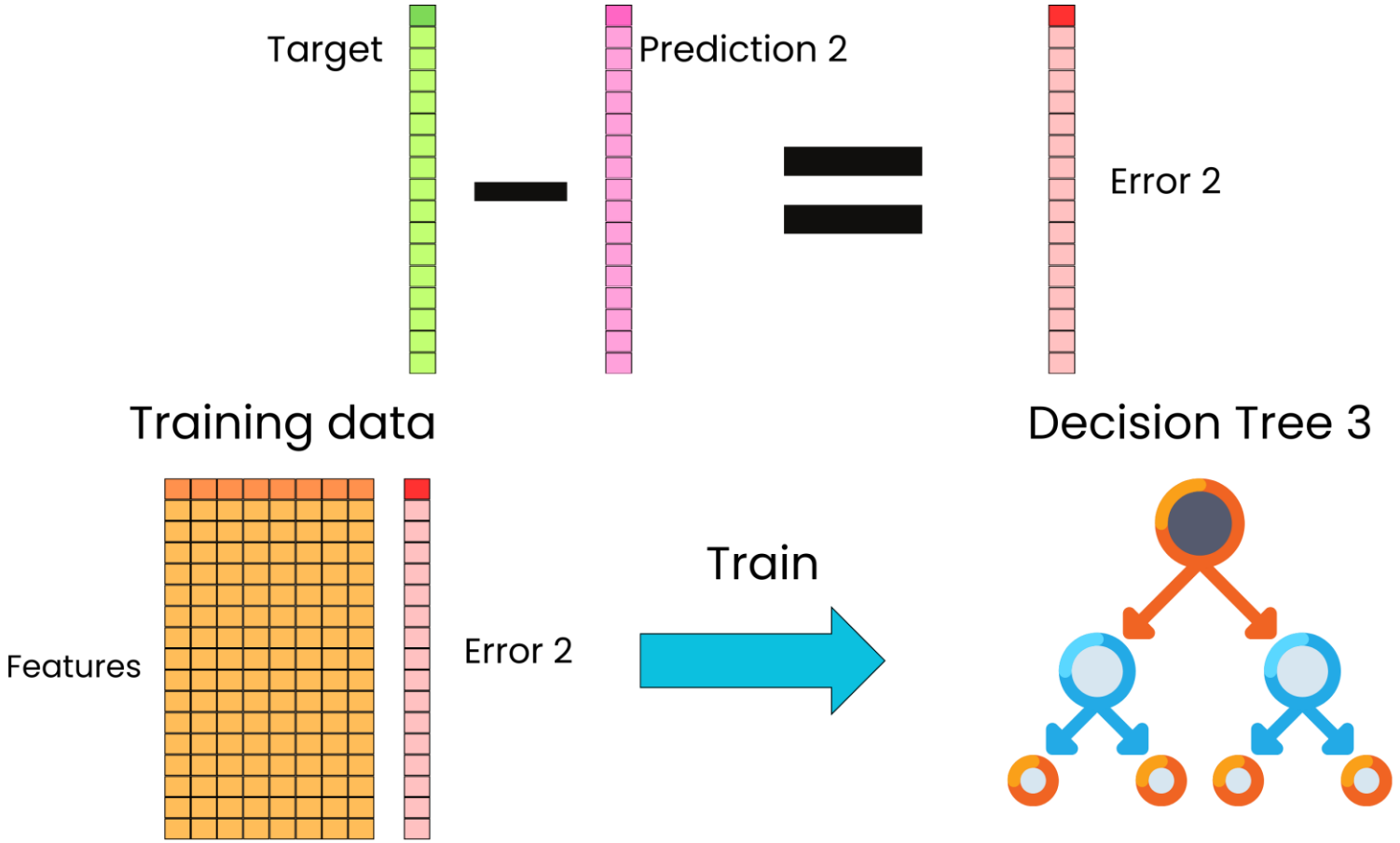
How the Gradient Boosting Algorithm Works

- The full model now becomes the additive ensemble of trees, and we can use that updated model to generate new predictions on the training data.



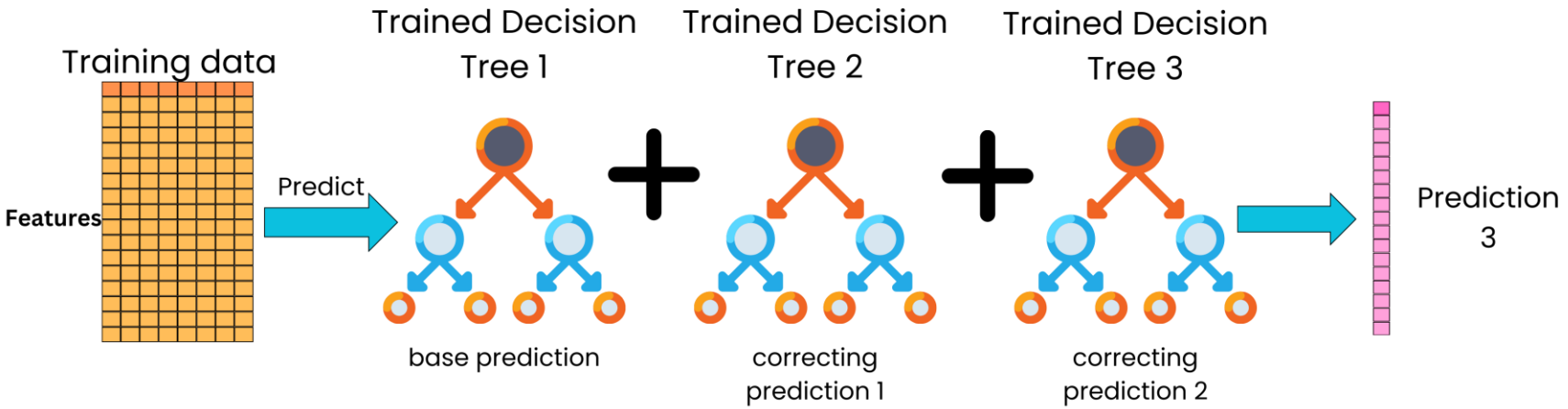
How the Gradient Boosting Algorithm Works

- Again, we can compare the predictions that the current model is making to the target and use the resulting computed error as the new target for a new correcting tree.



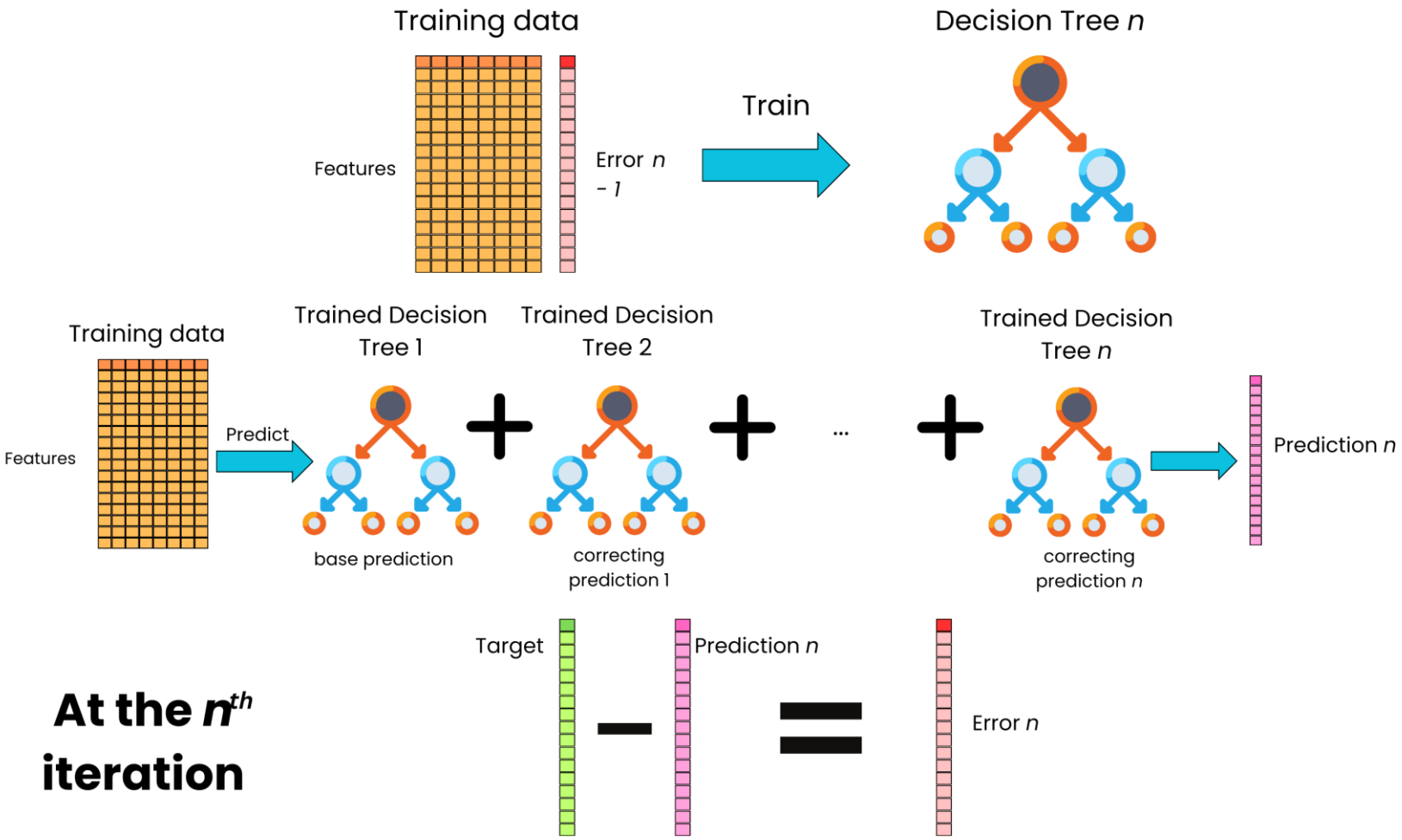
How the Gradient Boosting Algorithm Works

- And again, we update the model by adding this new correcting tree to the full ensemble of trees to generate even better predictions.



How the Gradient Boosting Algorithm Works

- We iterate this process until we start to overfit when we look at the predictive performance on a validation dataset.



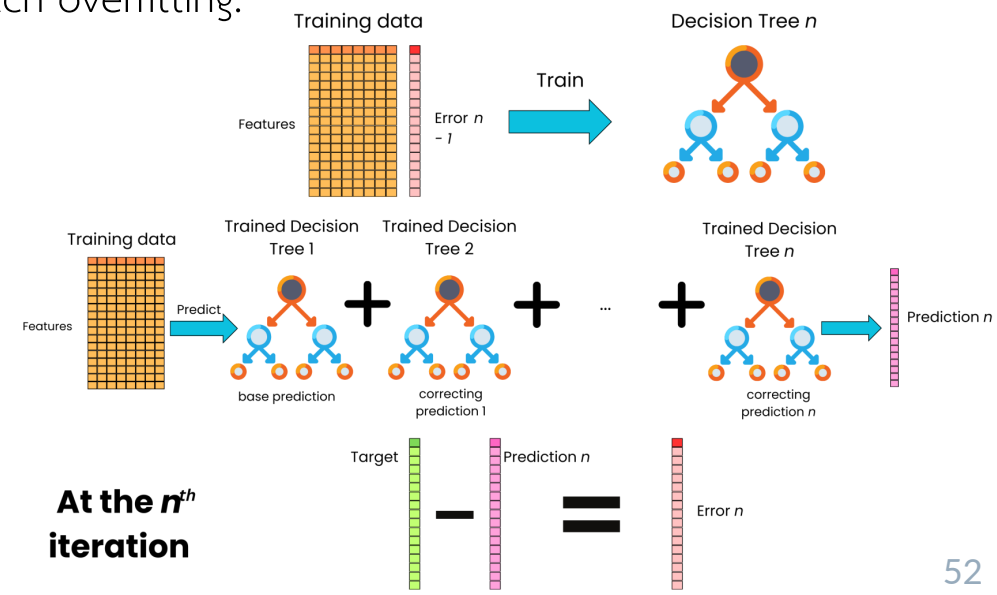
How the Gradient Boosting Algorithm Works

So, to summarize, in the gradient-boosted trees algorithm, we iterate the following:

- We train a tree on the errors made at the previous iteration
- We add the tree to the ensemble, and we predict with the new model
- We compute the errors made for this iteration.

At each iteration, the new tree tries to correct the learning done by the previous tree, and we refine little by little the predictions made by the model.

Typically, to prevent overfitting, we use the method of shrinkage. We multiply the contribution of the new tree by a small factor such that the new tree does not have too much impact on the overall prediction. This is to prevent too much overfitting.



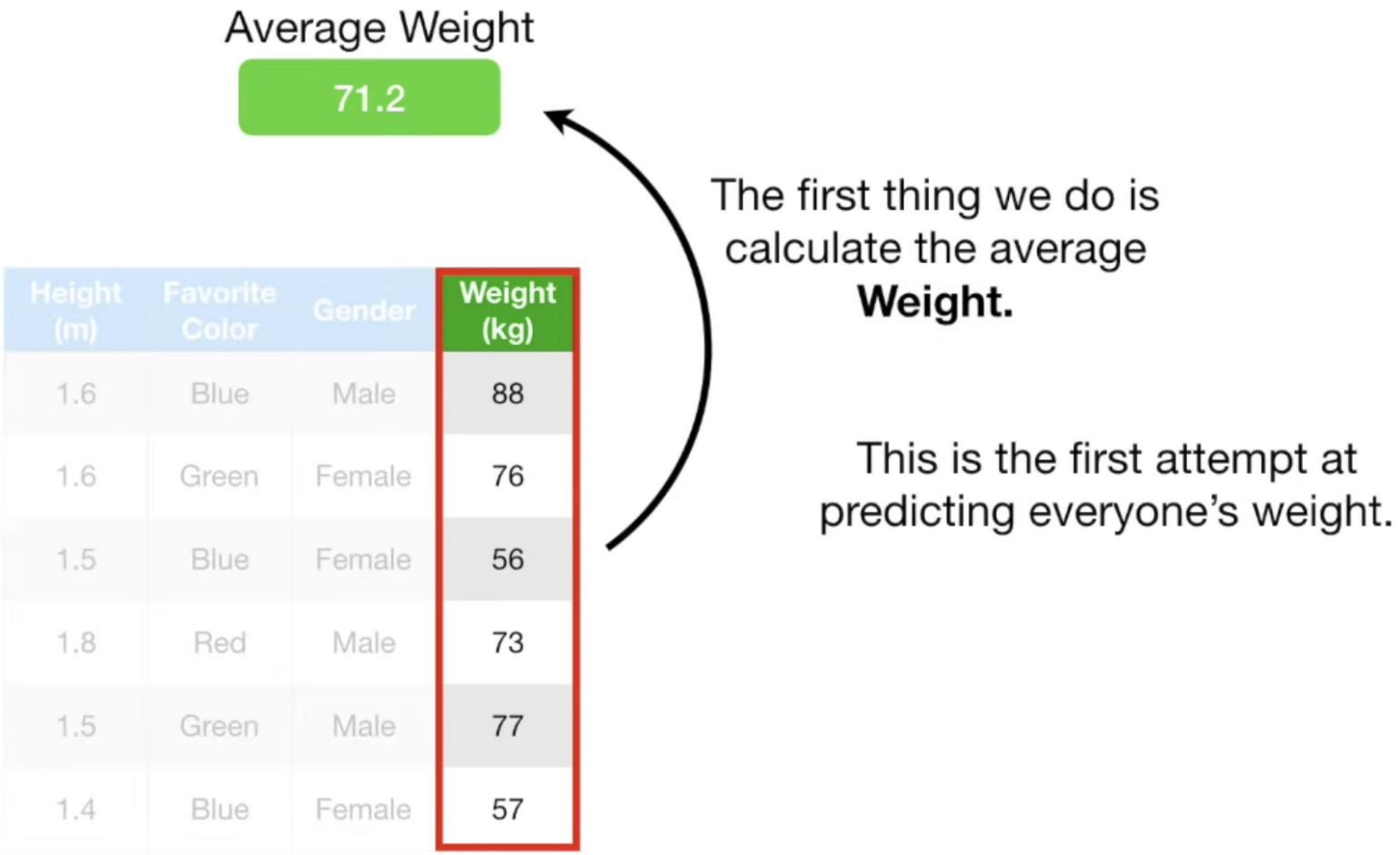
Understanding Gradient Boosting Step by Step:

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

NOTE: When **Gradient Boost** is used to **Predict** a continuous value, like **Weight**, we say that we are using **Gradient Boost** for **Regression**.

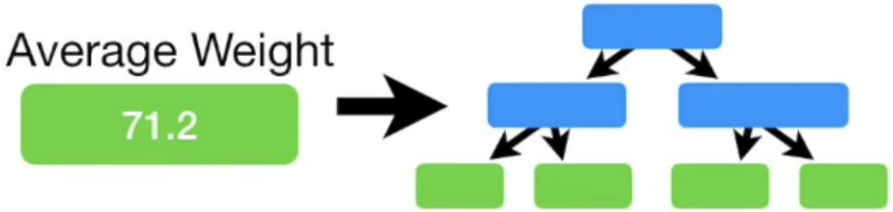
…let's see how the most common **Gradient Boost** configuration would use this **Training Data** to **Predict Weight**.

Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

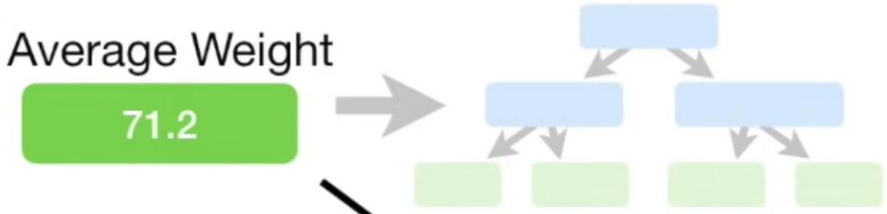
Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

The next thing we do is build a tree based on the errors from the first tree.

Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

The errors that the previous tree made are the differences between the **Observed Weights** and the **Predicted Weight, 71.2.**

(Observed Weight - Predicted Weight)

Understanding Gradient Boosting Step by Step:

Average Weight

71.2

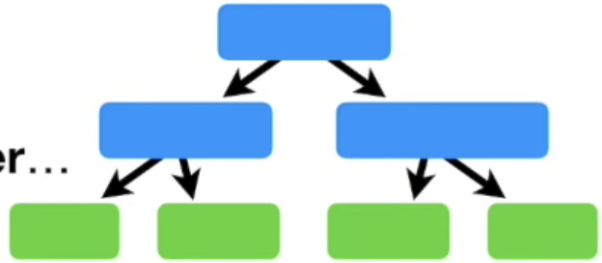
Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

(Observed Weight - Predicted Weight)

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

Understanding Gradient Boosting Step by Step:

Now we will build a **Tree**, using **Height**, **Favorite Color** and **Gender**...

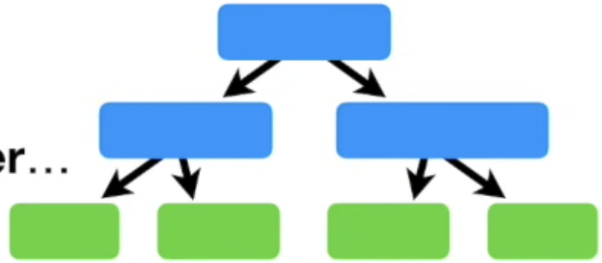


Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

...to **Predict** the **Residuals**.

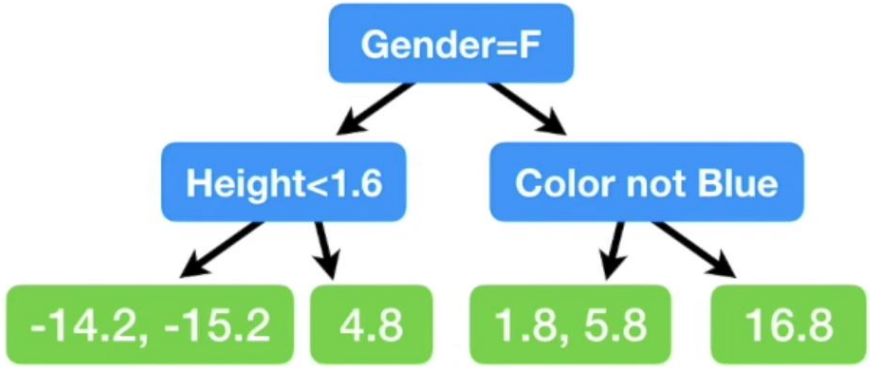
Understanding Gradient Boosting Step by Step:

Now we will build a **Tree**, using **Height**, **Favorite Color** and **Gender**...

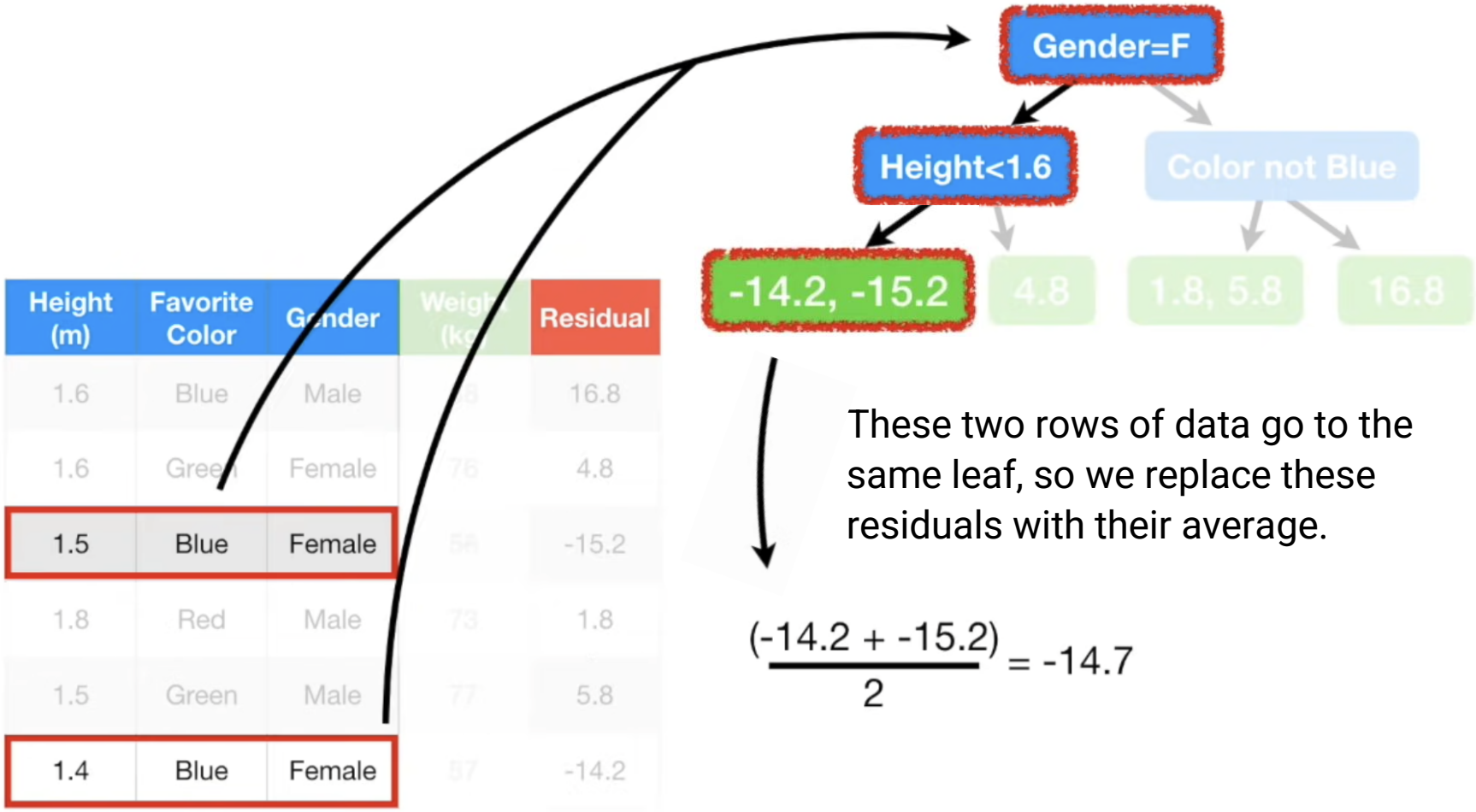


Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

...to **Predict** the **Residuals**.

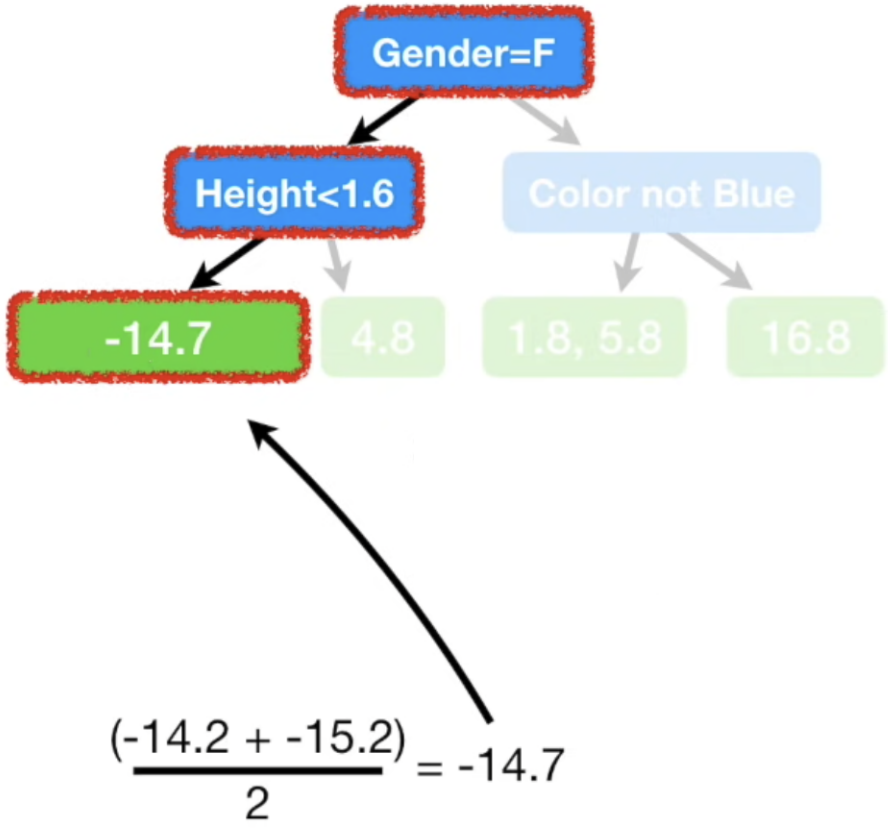


Understanding Gradient Boosting Step by Step:

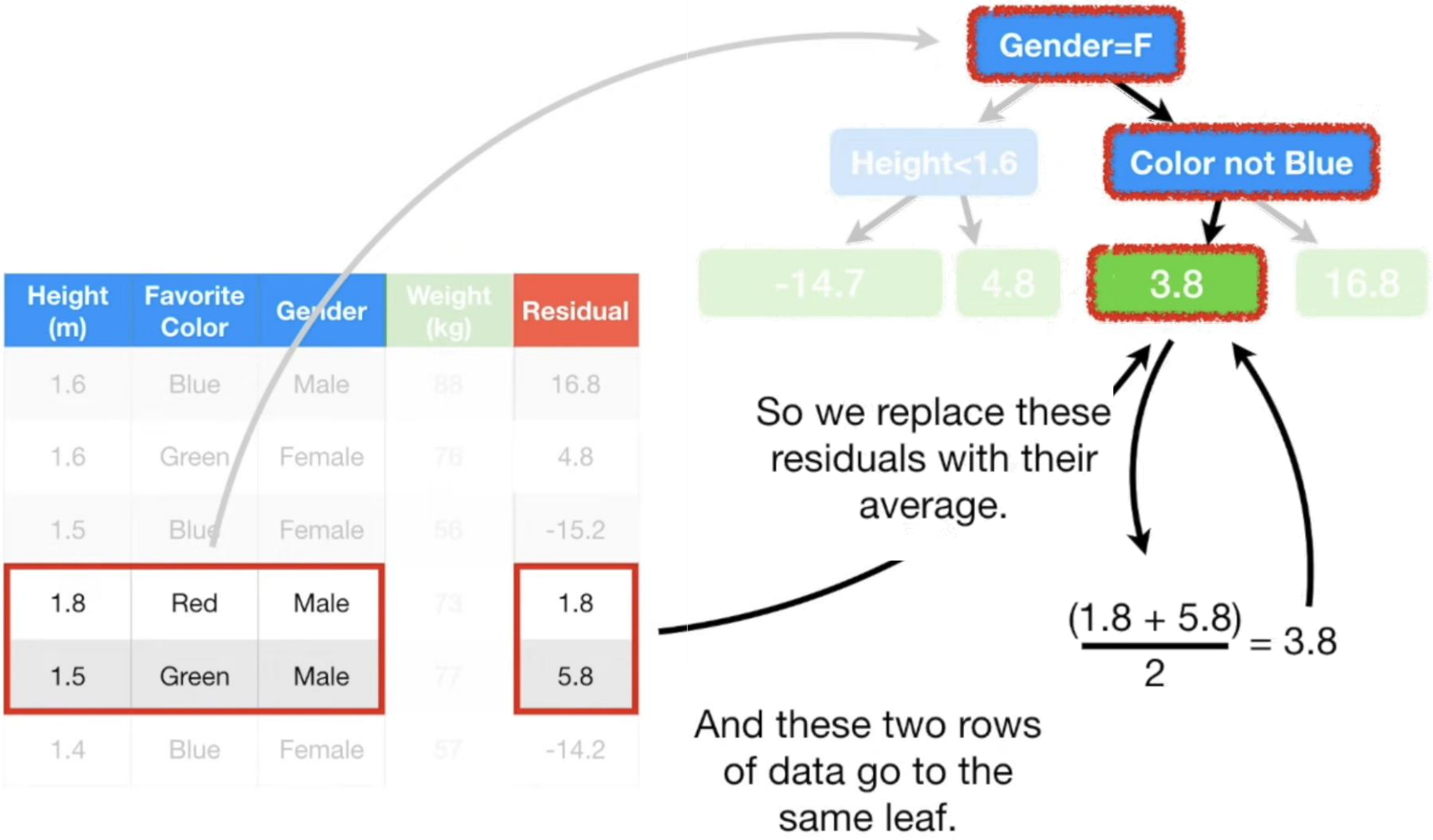


Understanding Gradient Boosting Step by Step:

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

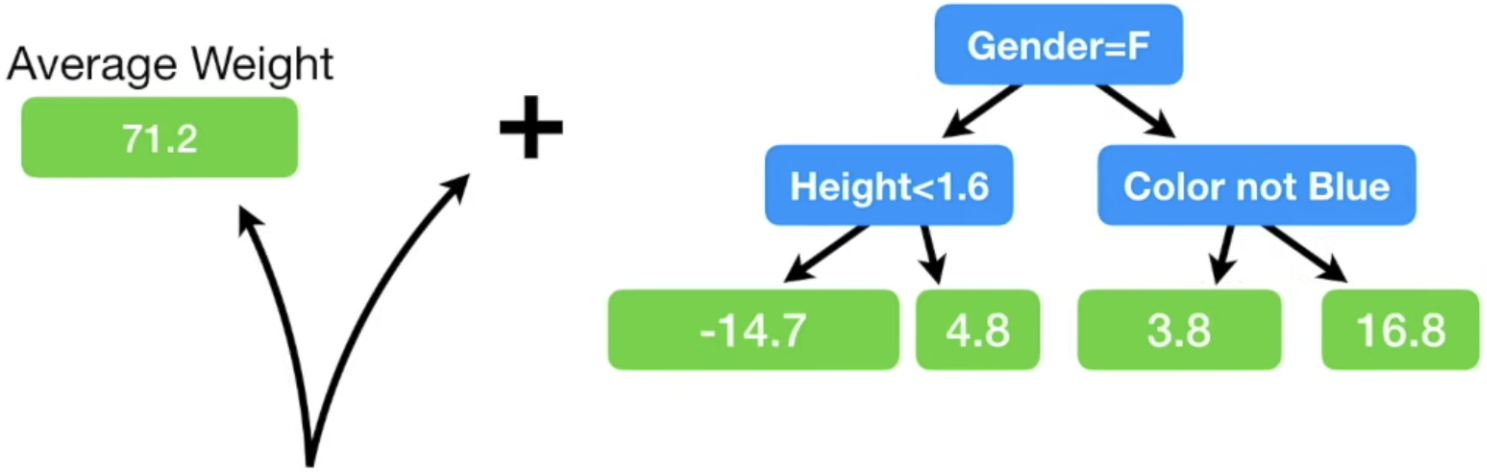


Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

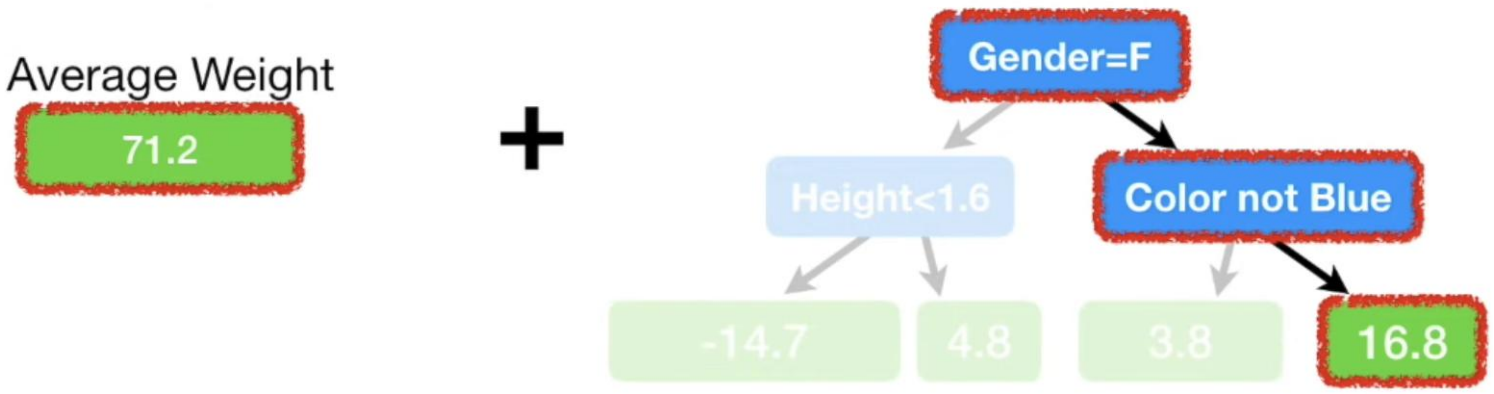
Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

…to make a new **Prediction** of an individual's **Weight** from the **Training Data**.

Understanding Gradient Boosting Step by Step:



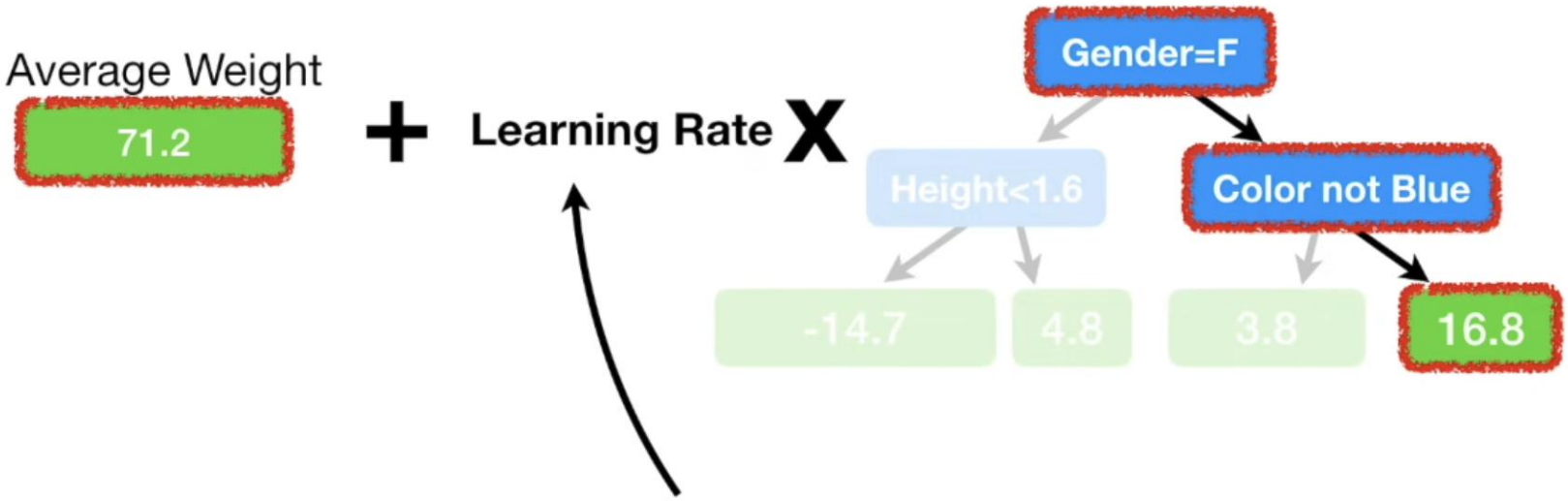
…so the **Predicted Weight** = 71.2 + 16.8 = 88

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

…which is the same as the **Observed Weight**.

The model fits the training data too well.
In other words, we have low bias but probably very high variance

Understanding Gradient Boosting Step by Step:

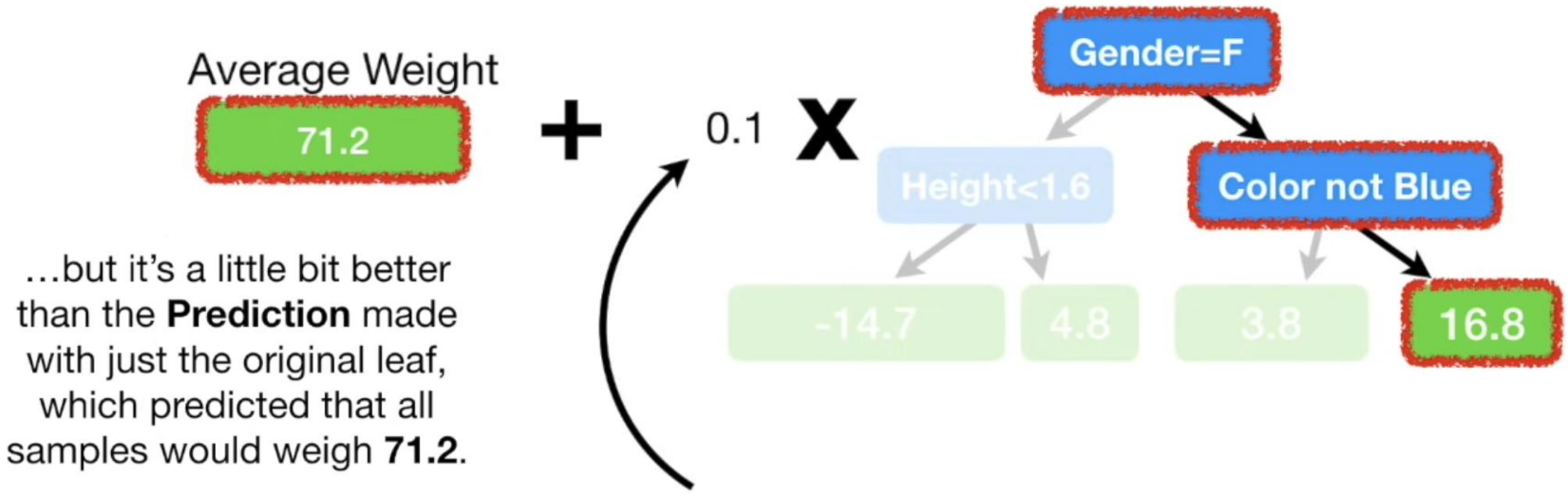


Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

Gradient Boost deals with this problem by using a **Learning Rate** to scale the contribution from the new tree.

The **Learning Rate** is a value between **0** and **1**.

Understanding Gradient Boosting Step by Step:



Predicted Weight = 71.2 + (0.1 × 16.8) = 72.9

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

With the **Learning Rate** set to **0.1**, the new **Prediction** isn't as good as as it was before…

Understanding Gradient Boosting Step by Step:

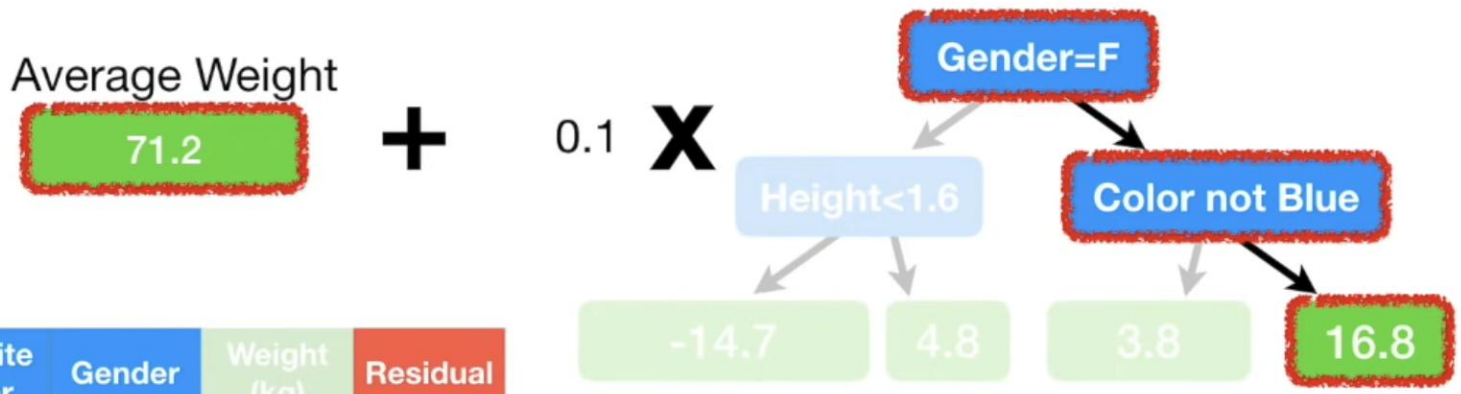
So let's build another tree so we can take another small step in the right direction.

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	
1.6	Green	Female	76	
1.5	Blue	Female	56	
1.8	Red	Male	73	
1.5	Green	Male	77	
1.4	Blue	Female	57	

Just like before, we calculate the **Pseudo Residuals**, the difference between the **Observed Weights** and our latest **Predictions**.

← **Residual** = (**Observed** - **Predicted**)

Understanding Gradient Boosting Step by Step:



Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	
1.5	Blue	Female	56	
1.8	Red	Male	73	
1.5	Green	Male	77	
1.4	Blue	Female	57	

Residual = (**88** - (71.2 + 0.1 × 16.8))

= 15.1

…and we save that in the column for **Pseudo Residuals**.

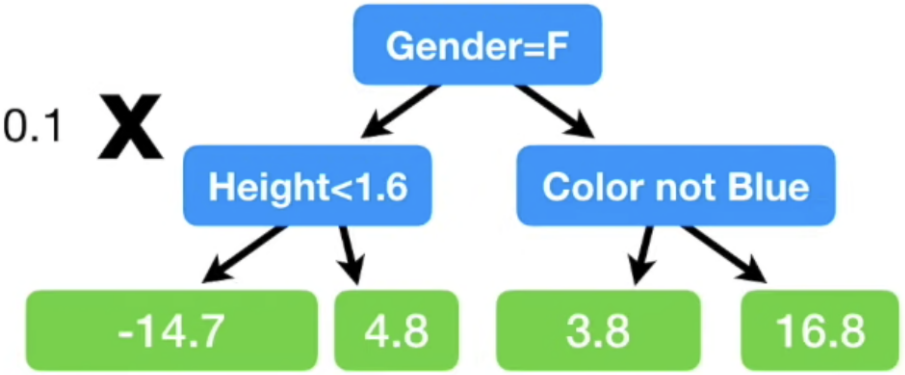
Understanding Gradient Boosting Step by Step:

Average Weight

71.2

+

0.1 X

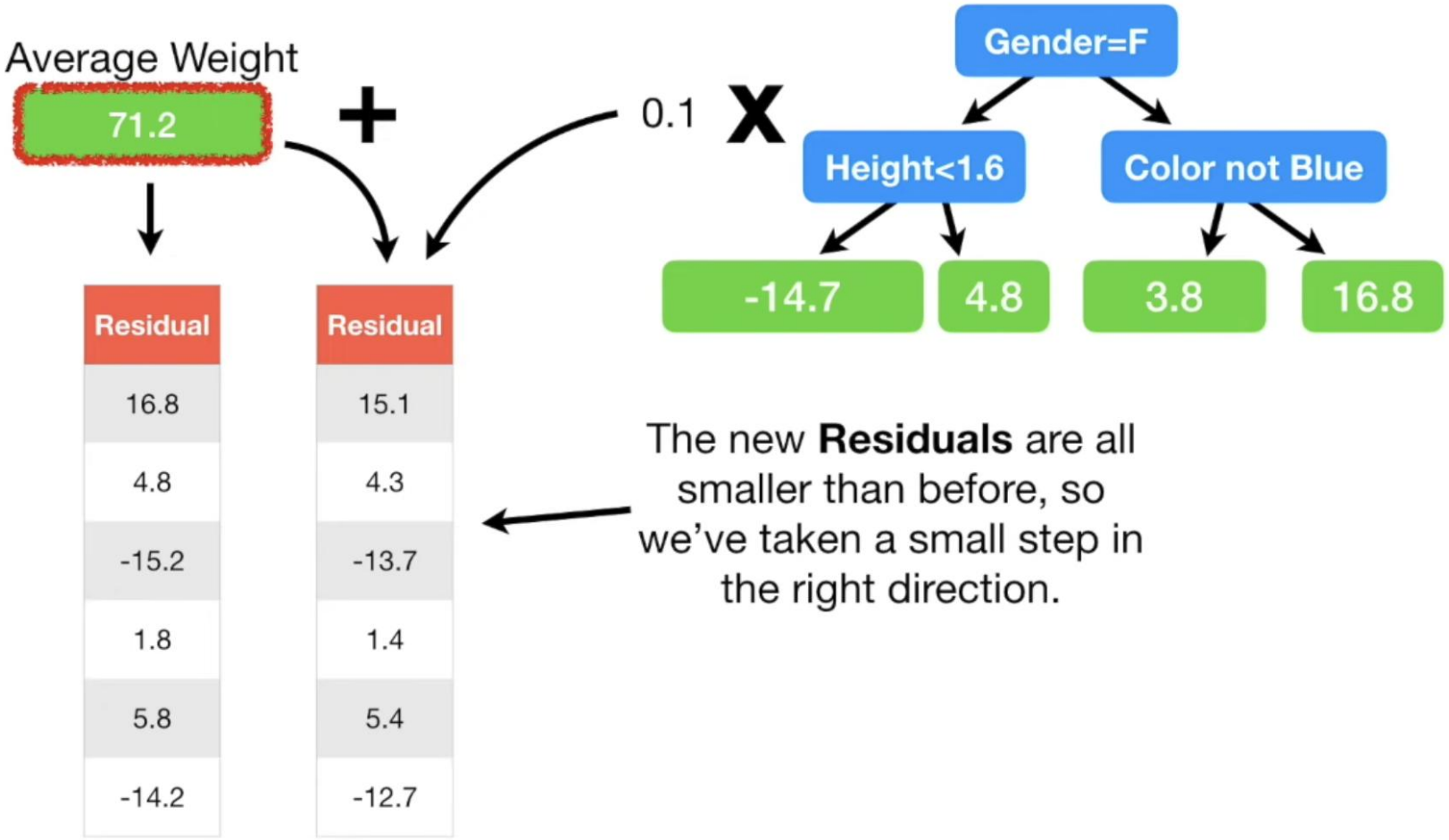


Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7

Then we repeat for the all of the other individuals in the **Training Dataset.**

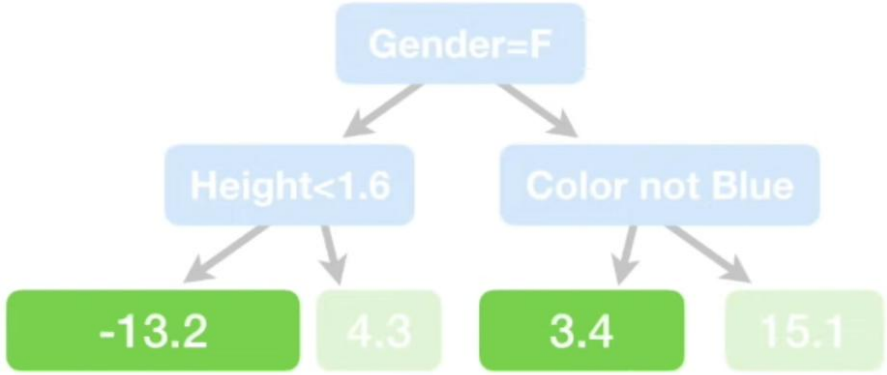
Residual = (Observed - Predicted)

Understanding Gradient Boosting Step by Step:



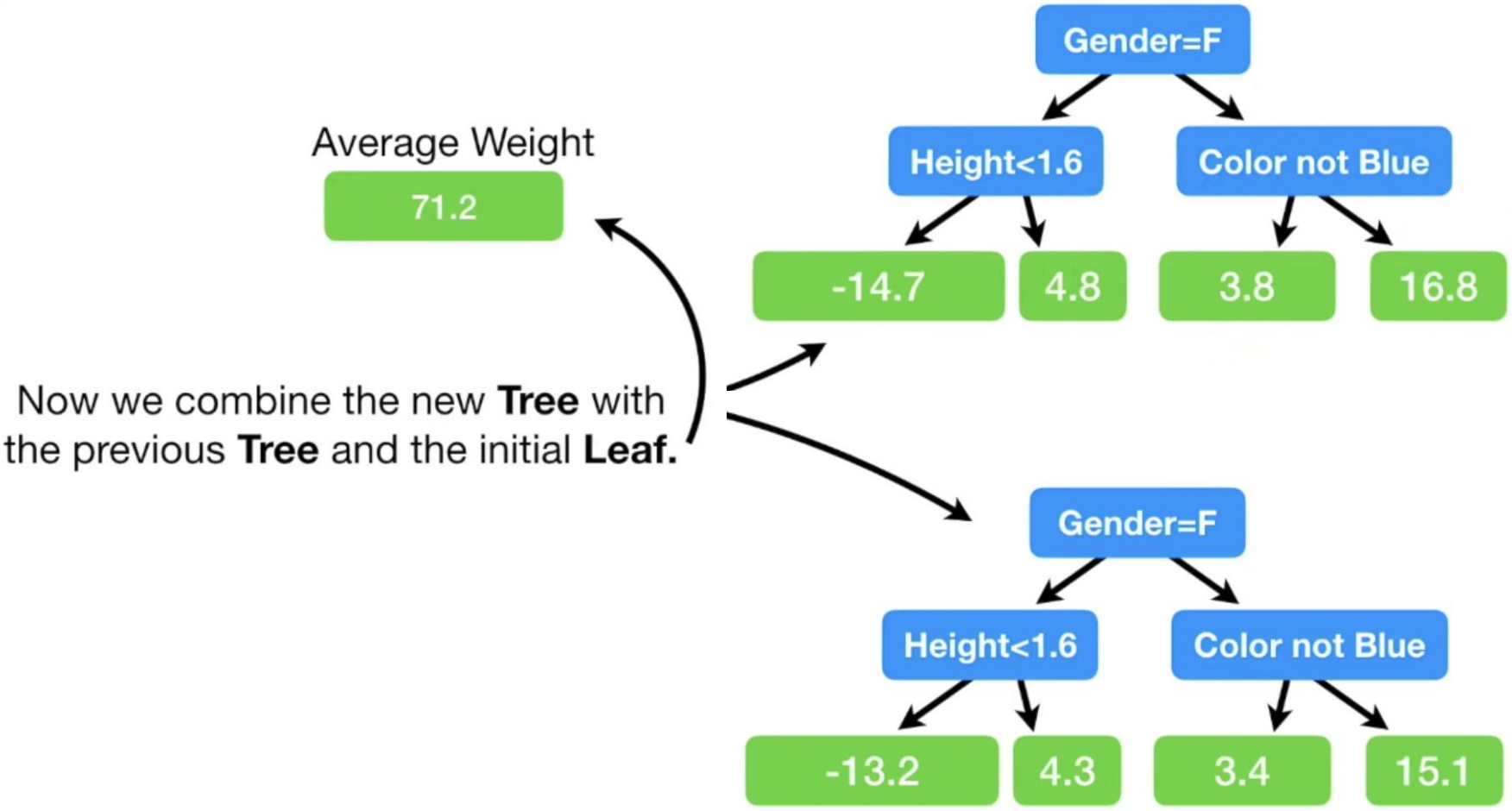
Understanding Gradient Boosting Step by Step:

NOTE: In this simple example the branches are the same as before. However, in practice, the trees can be different each time.

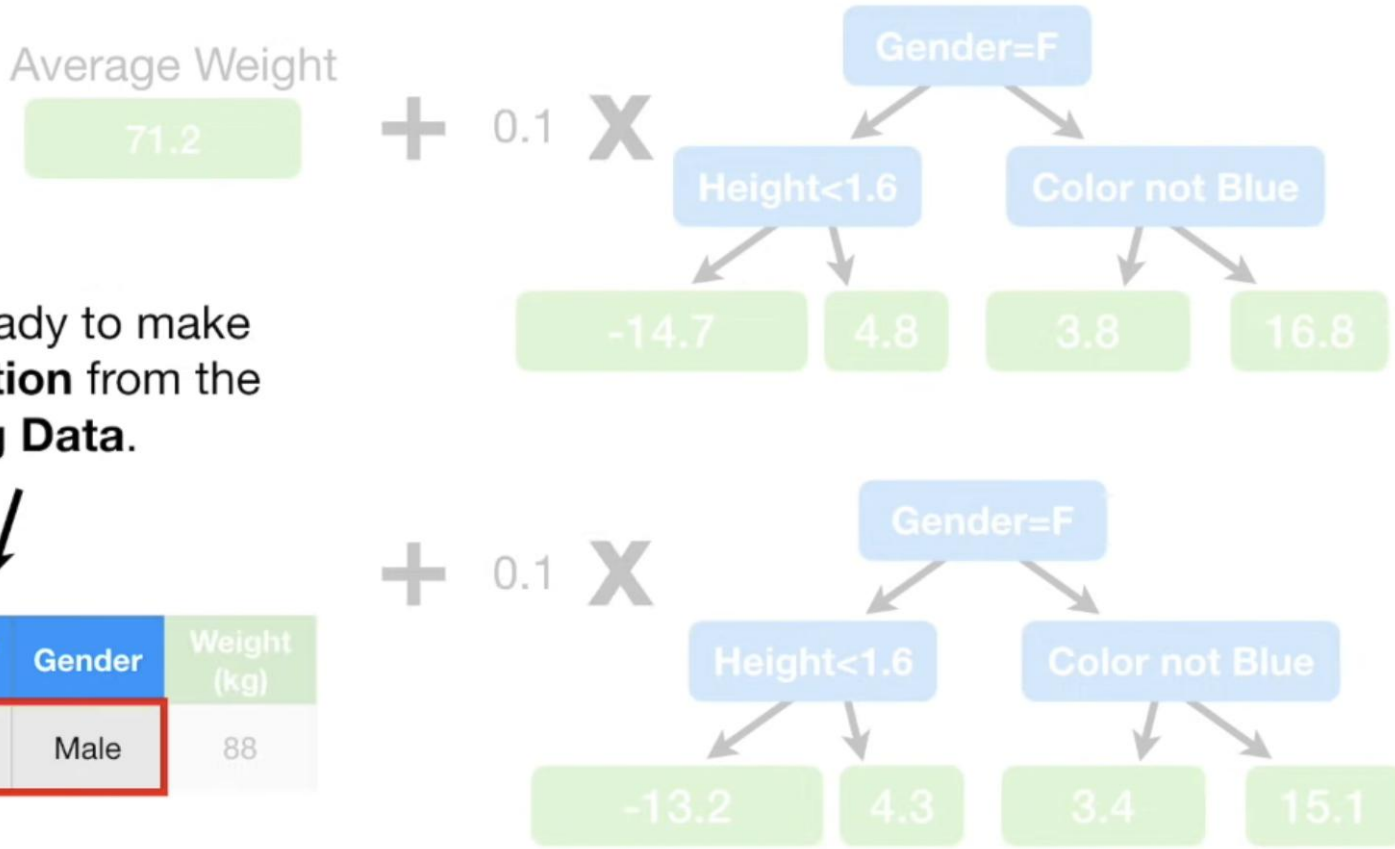


Just like before, since multiple samples ended up in these leaves, we just replace the **Residuals** with their averages.

Understanding Gradient Boosting Step by Step:



Understanding Gradient Boosting Step by Step:



Now we're ready to make a new **Prediction** from the **Training Data**.

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

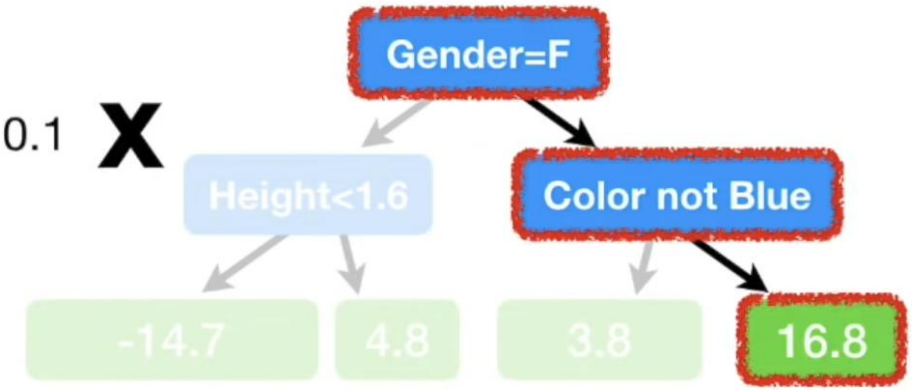
Understanding Gradient Boosting Step by Step:

Just like before, we start with the initial **Prediction**…

Average Weight

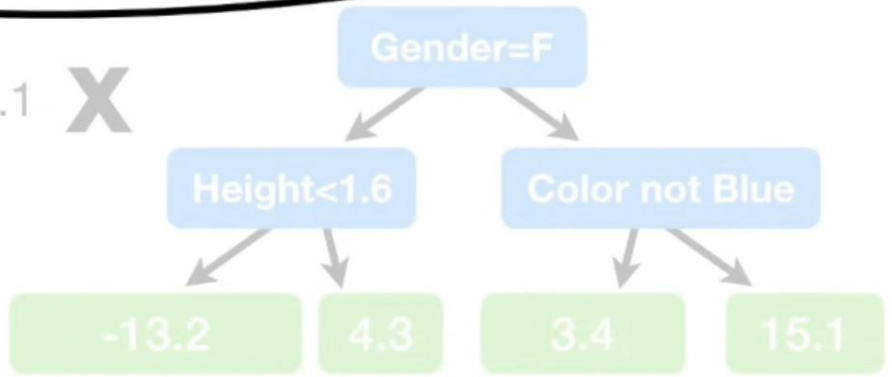
71.2

+ 0.1 X



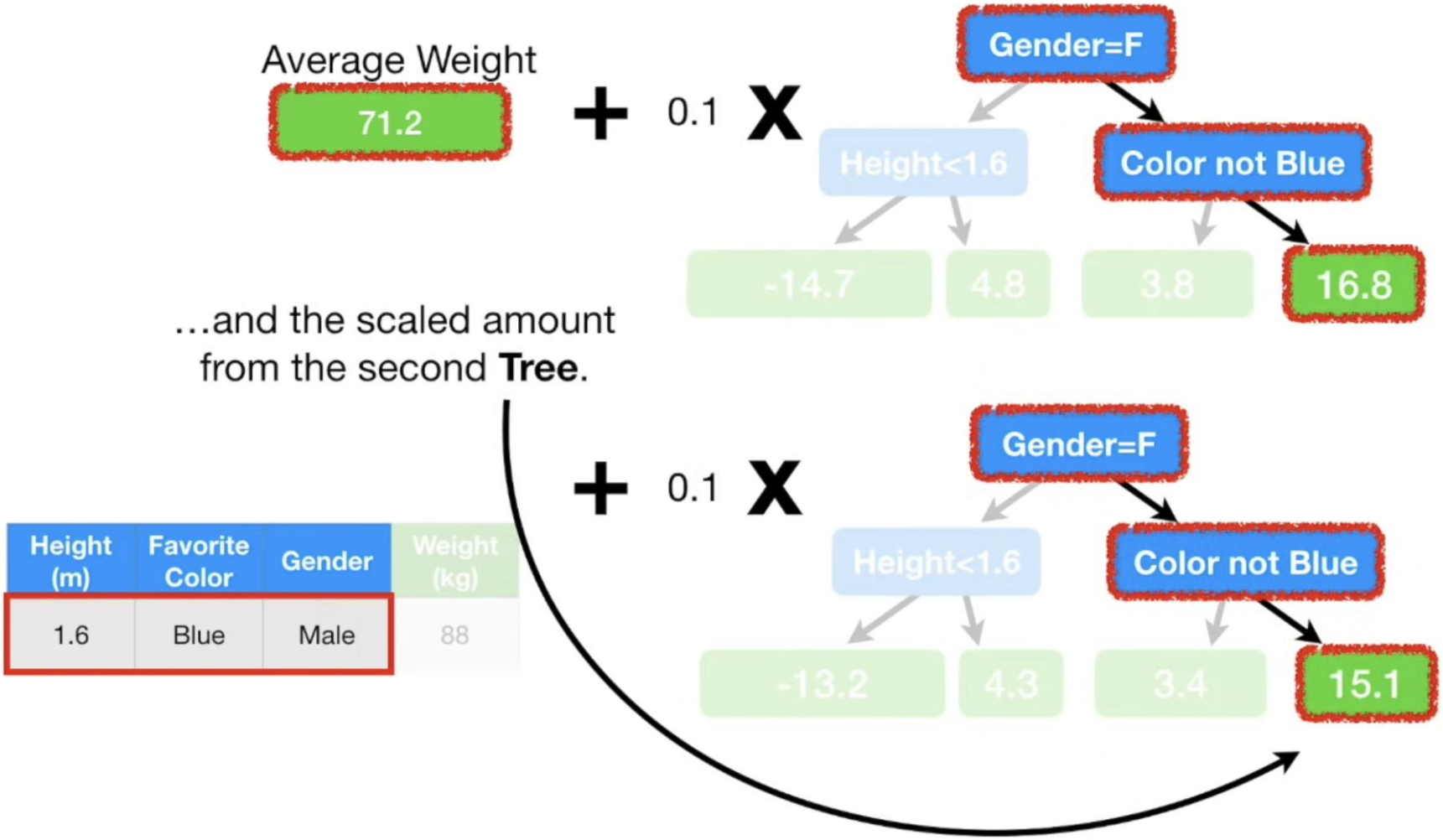
…then add the scaled amount from the first **Tree**…

+ 0.1 X



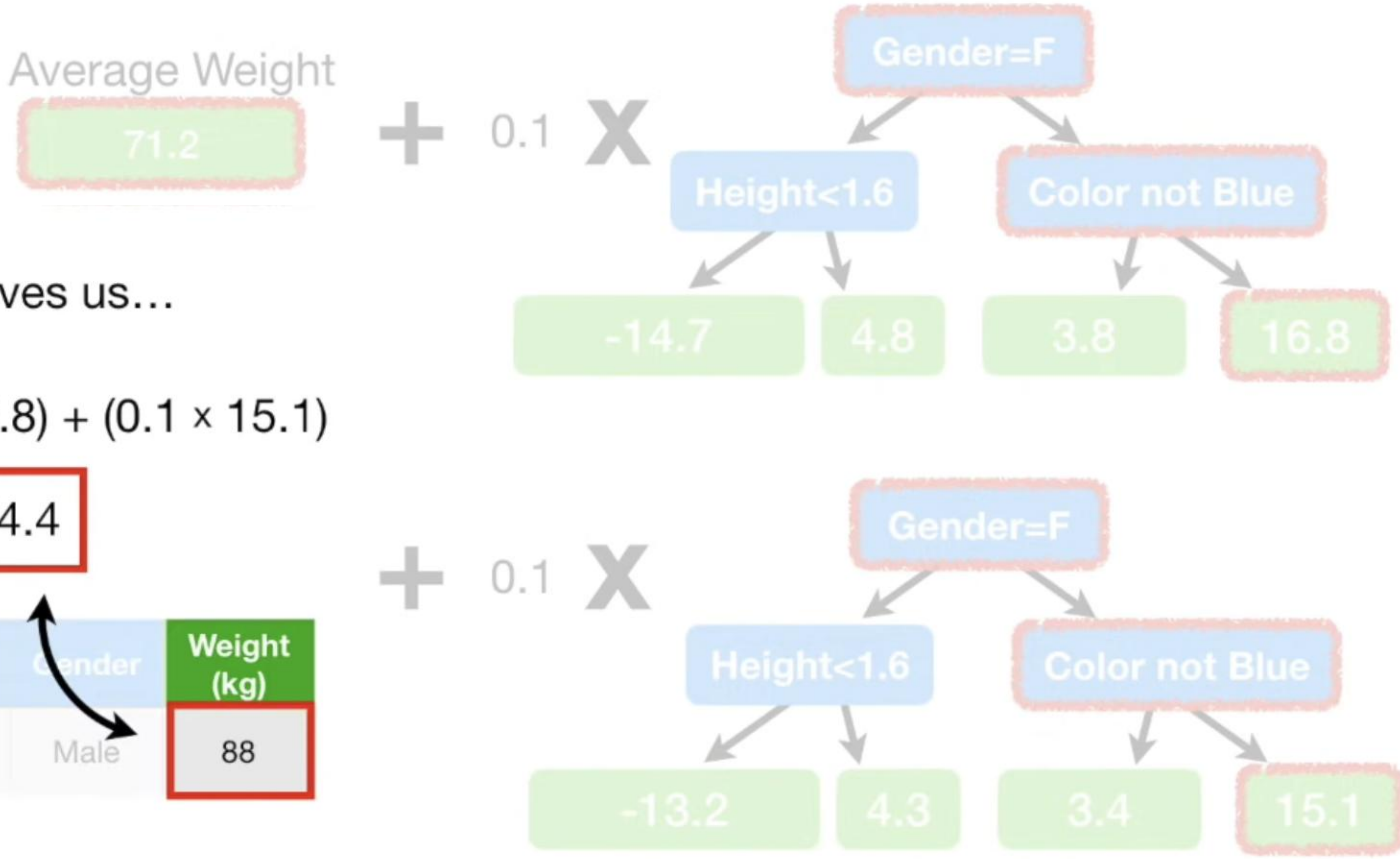
Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

Understanding Gradient Boosting Step by Step:



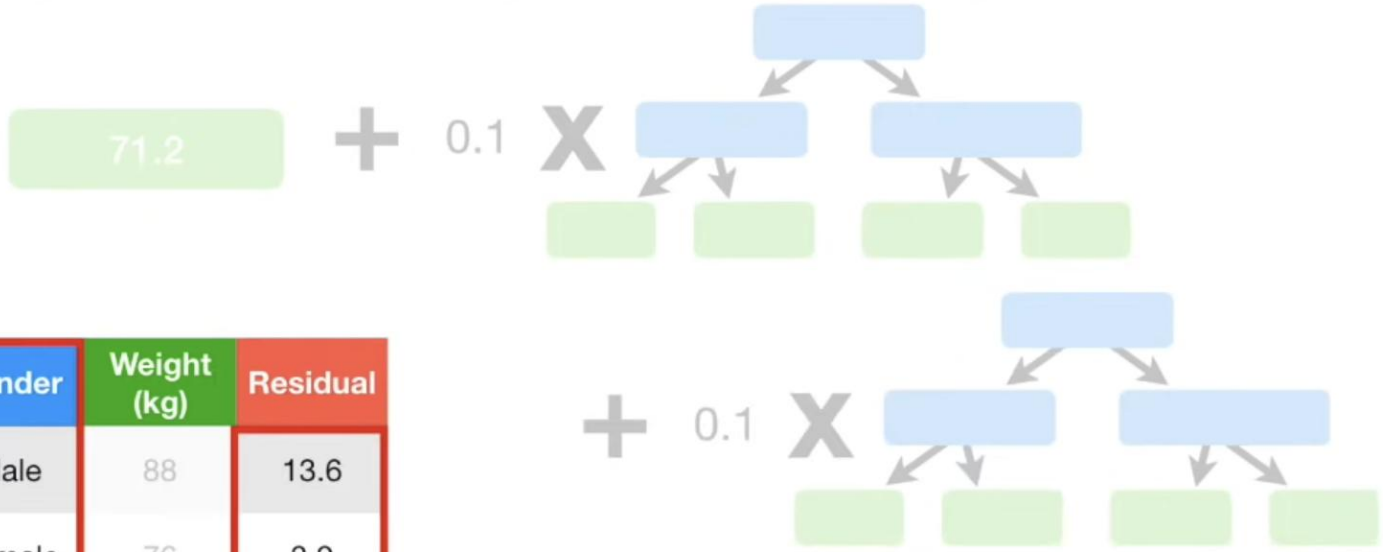
Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

Understanding Gradient Boosting Step by Step:



Which is another small step closer to the **Observed Weight**.

Understanding Gradient Boosting Step by Step:

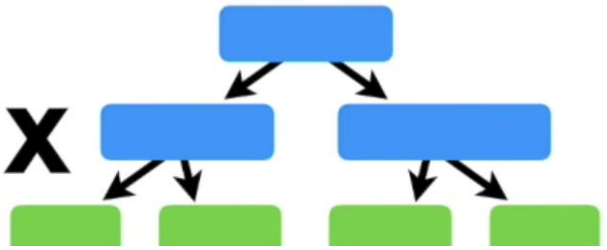


Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	13.6
1.6	Green	Female	76	3.9
1.5	Blue	Female	56	-12.4
1.8	Red	Male	73	1.1
1.5	Green	Male	77	5.1
1.4	Blue	Female	57	-11.4

…to calculate new **Residuals.**

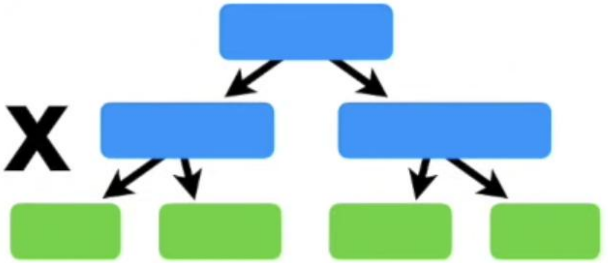
Understanding Gradient Boosting Step by Step:

Remember, these were the **Residuals** from when we just used a single leaf to **Predict Weight**…

71.2 + 0.1 X 

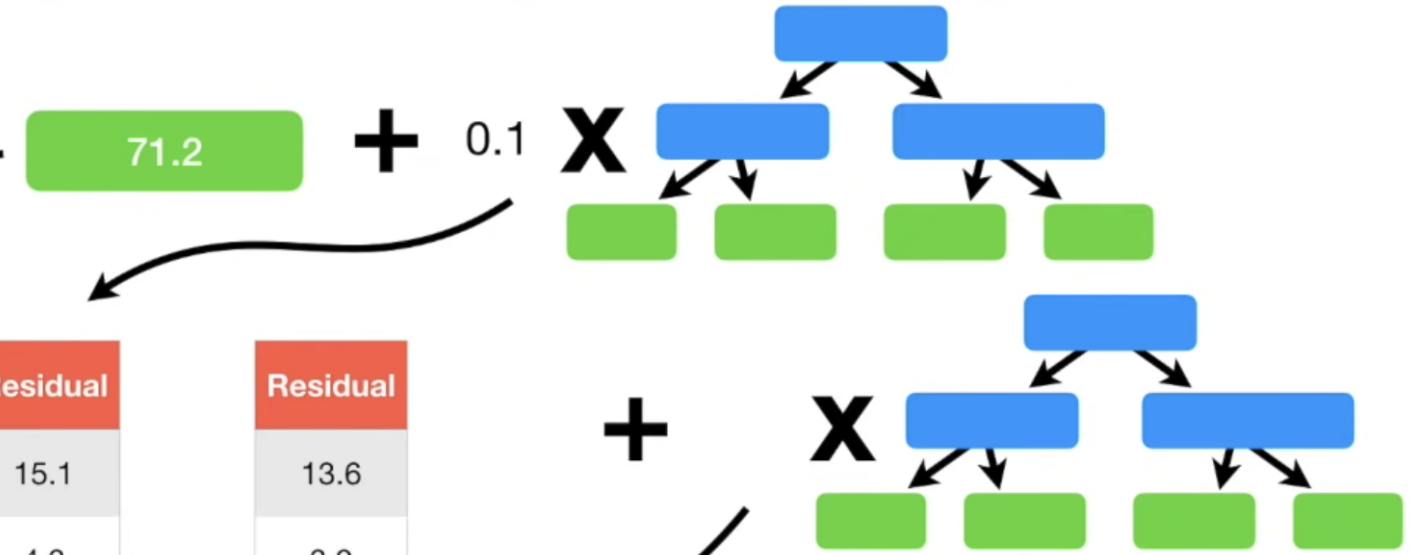
…and these were the **Residuals** after we added the first **Tree** to the **Prediction**…

Residual	Residual	Residual
16.8	15.1	13.6
4.8	4.3	3.9
-15.2	-13.7	-12.4
1.8	1.4	1.1
5.8	5.4	5.1
-14.2	-12.7	-11.4

+ X 

…and these are the **Residuals** after we added the second **Tree** to the **Prediction**…

Understanding Gradient Boosting Step by Step:



Residual
16.8
4.8
-15.2
1.8
5.8
-14.2

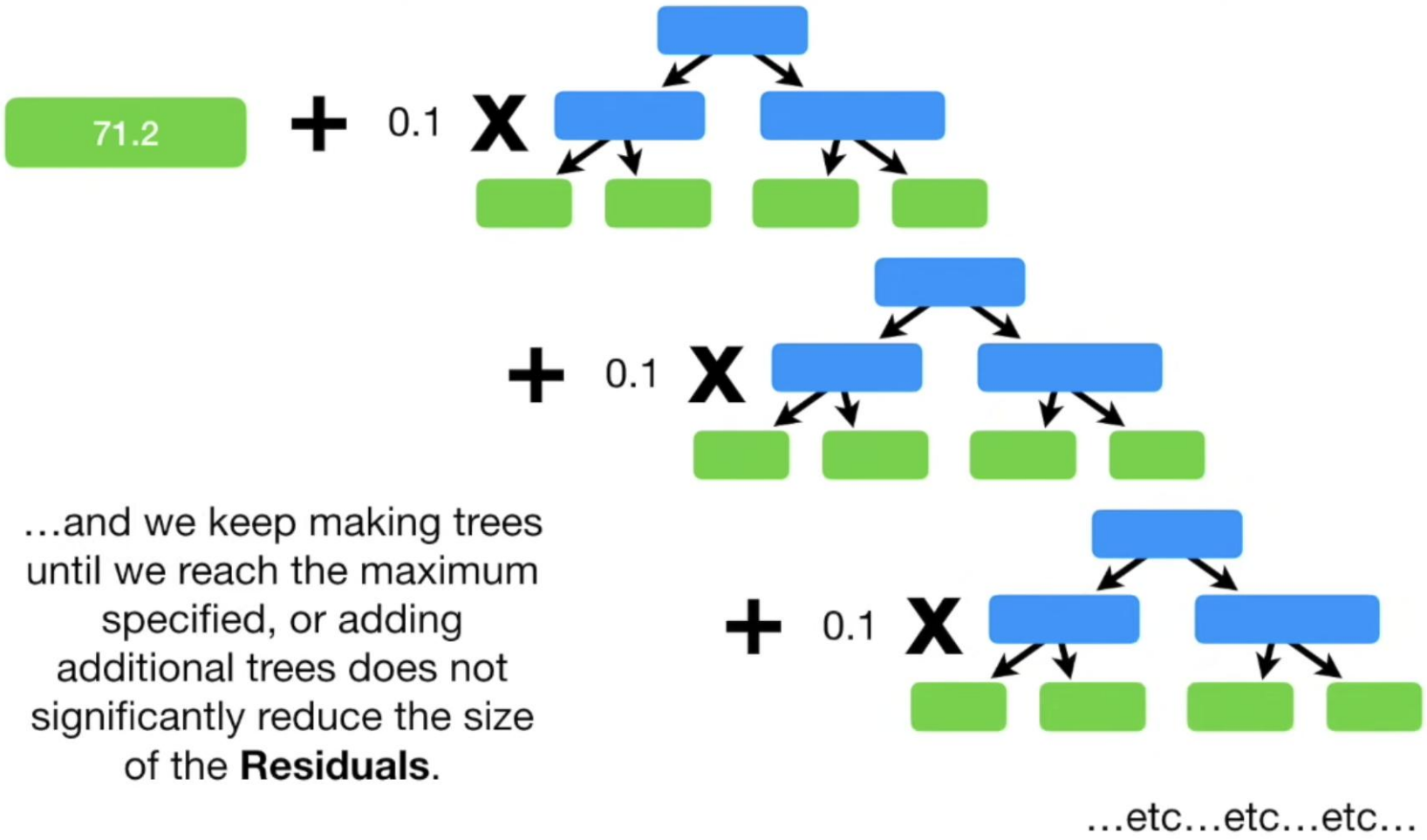
Residual
15.1
4.3
-13.7
1.4
5.4
-12.7

Residual
13.6
3.9
-12.4
1.1
5.1
-11.4

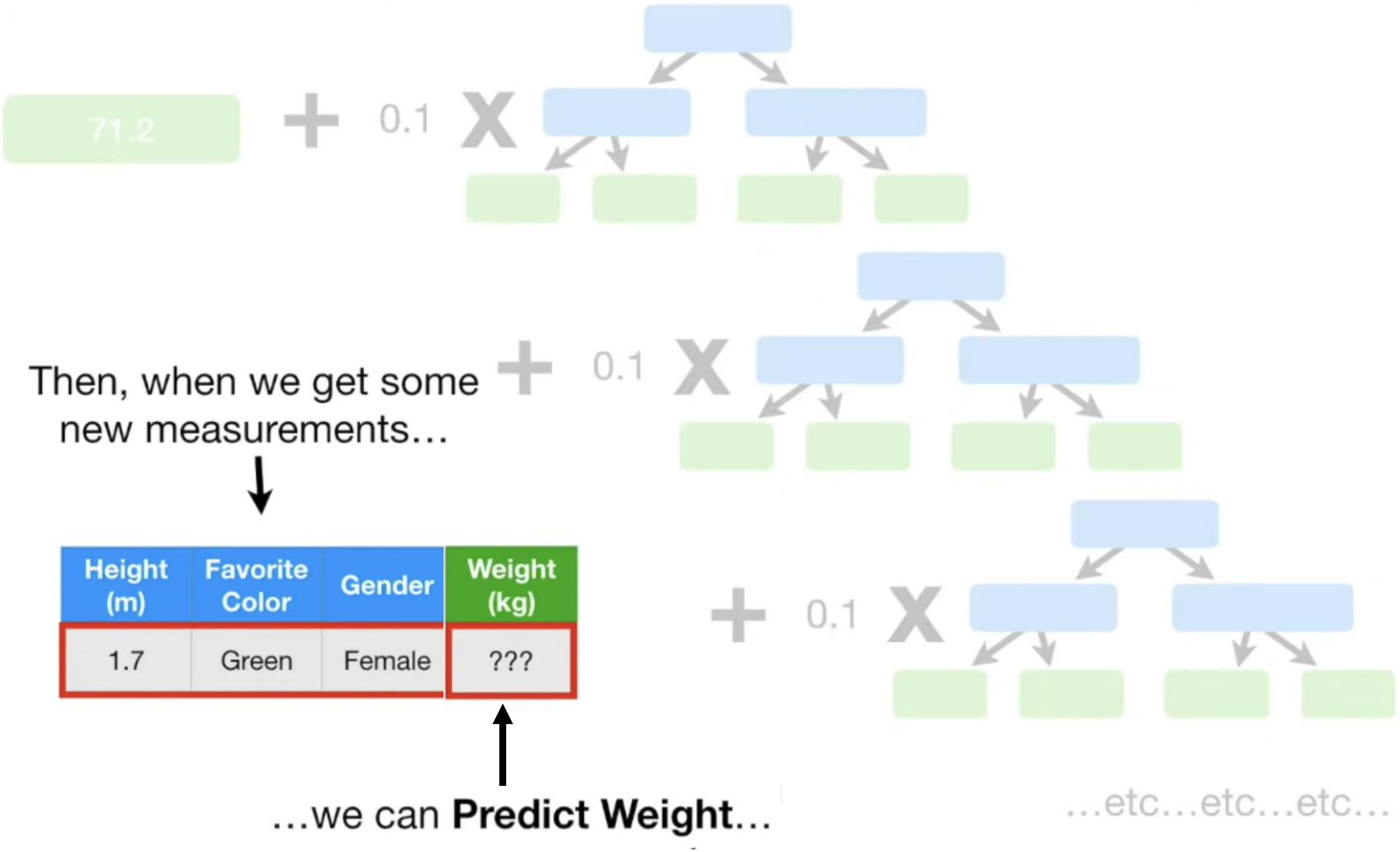
Each time we add a tree to the **Prediction**, the **Residuals** get smaller.

So we've taken another small step towards making good **Predictions**.

Understanding Gradient Boosting Step by Step:

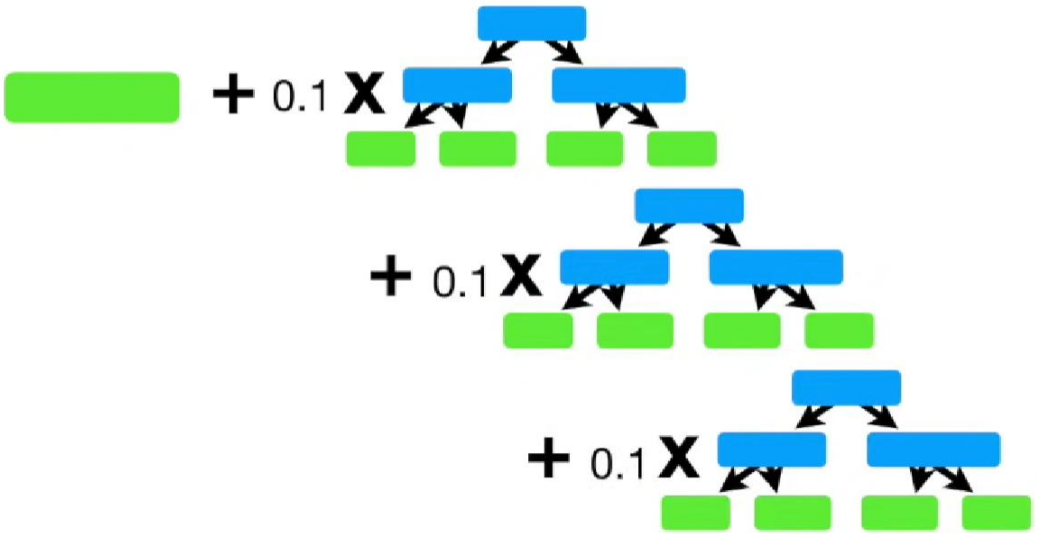


Understanding Gradient Boosting Step by Step:



Gradient Boosting for Classification

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes



Gradient Boosting for Classification

When we use **Gradient Boost for Classification**, the initial **Prediction** for every individual is the **log(odds)**.

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

So let's calculate the overall **log(odds)** that someone **Loves Troll 2**.

Gradient Boosting for Classification

…then the **log(odds)** that someone **Loves Troll 2** is…

log(4/2) = 0.7

$$\log(\frac{4}{2}) = 0.7$$

…which we will put into our initial leaf.

$$\log(\frac{4}{2}) = 0.7$$

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

Gradient Boosting for Classification

log(4/2) = 0.7

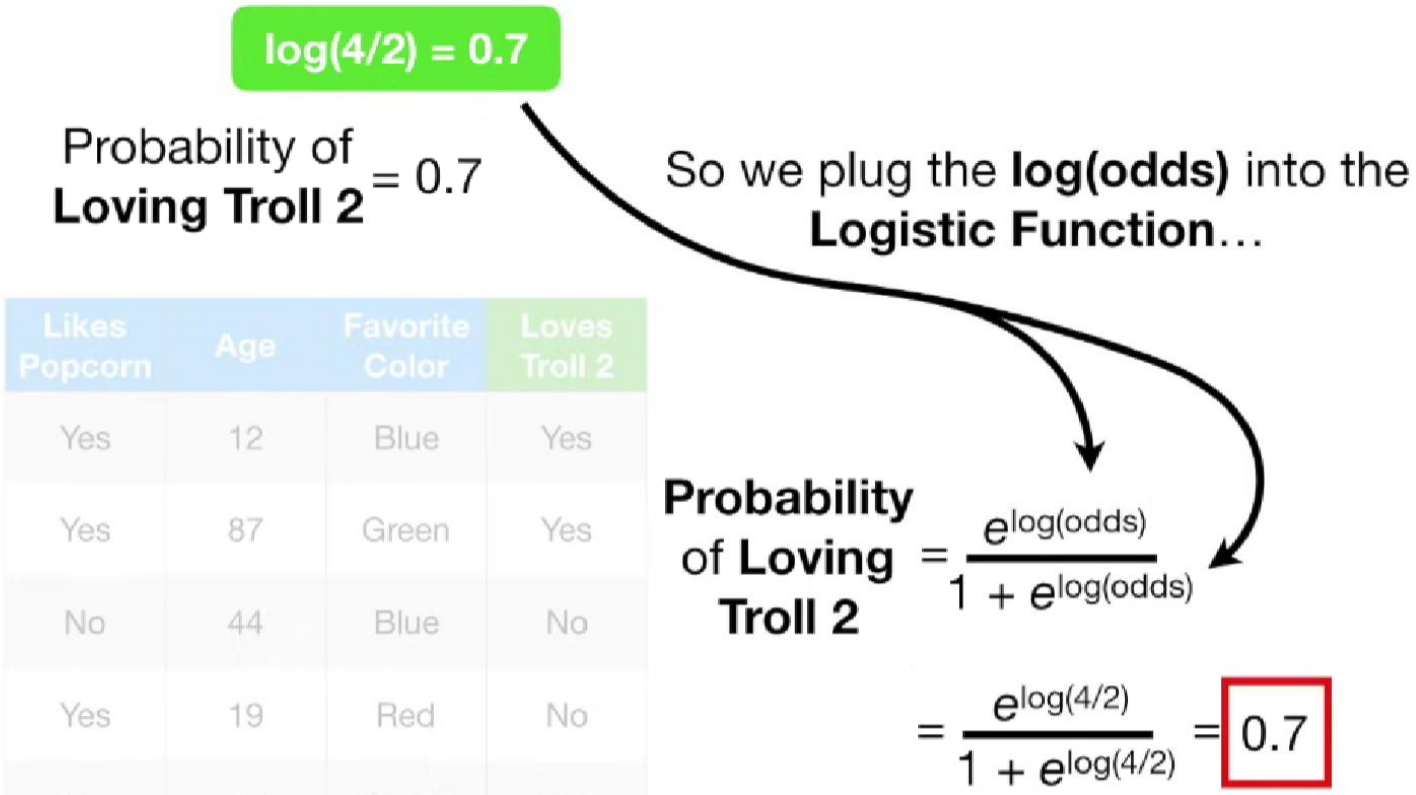
Just like with **Logistic Regression**, the easiest way to use the **log(odds)** for **Classification** is to convert it to a **Probability**...

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

...and we do that with a **Logistic Function**.

$$\text{Probability of Loving Troll 2} = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

Gradient Boosting for Classification



Gradient Boosting for Classification

log(4/2) = 0.7

Probability of **Loving Troll 2** = 0.7

NOTE: These two numbers, the **log(4/2)** and the **Probability** are the same only because I'm rounding. If I allowed **4** digits passed the decimal place…

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

$$\log(\frac{4}{2}) = 0.6931$$

$$\frac{e^{\log(4/2)}}{1 + e^{\log(4/2)}} = 0.6667$$

Gradient Boosting for Classification

log(4/2) = 0.7

Probability of **Loving Troll 2** = 0.7

Since the **Probability** of **Loving Troll 2** is greater than **0.5**, we can **Classify** everyone in the **Training Dataset** as someone who **Loves Troll 2**.

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

NOTE: While **0.5** is a very common threshold for making **Classification** decisions based on **Probability**, we could have just as easily used a different value.

Gradient Boosting for Classification

log(4/2) = 0.7

Probability of **Loving Troll 2** = 0.7

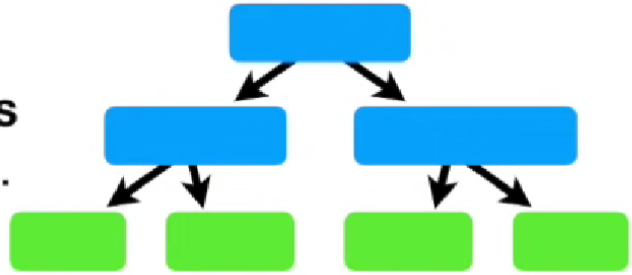
Likes Popcorn	Age	Favorite Color	Loves Troll 2	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3

We can measure how bad the initial **Prediction** is by calculating **Pseudo Residuals**, the difference between the **Observed** and the **Predicted** values.

Residual = (**Observed** - **Predicted**)

Gradient Boosting for Classification

Now we will build a **Tree**, using **Likes Popcorn**, **Age** and **Favorite Color**...

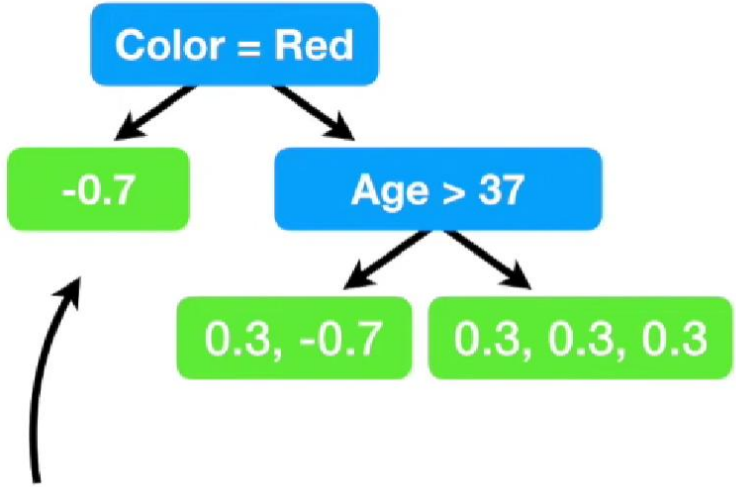


Likes Popcorn	Age	Favorite Color	Loves Troll 2	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3

...to **Predict** the **Residuals**.

Gradient Boosting for Classification

Likes Popcorn	Age	Favorite Color	Loves Troll 2	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3



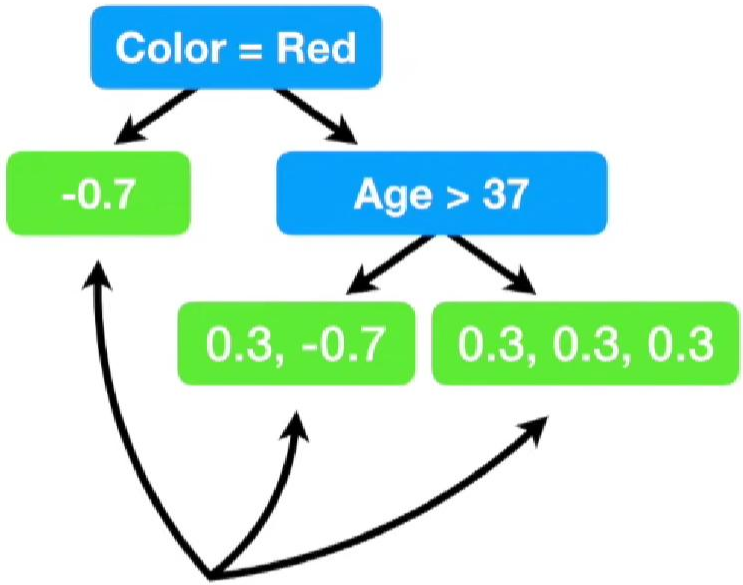
And here's the tree!

In this simple example, we are limiting the number of leaves to **3**.

In practice people often set the maximum number of leaves to be between **8** and **32**

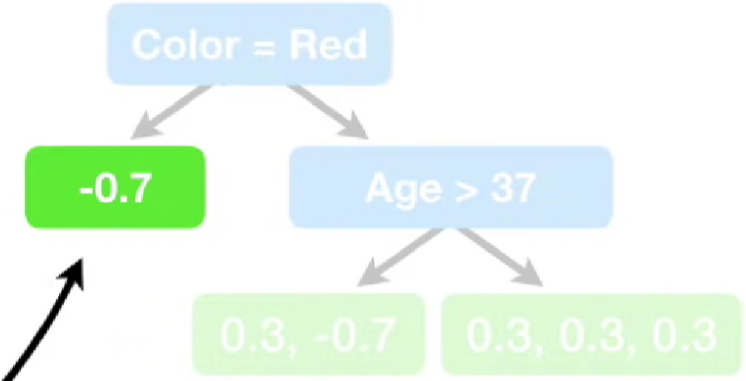
Gradient Boosting for Classification

Likes Popcorn	Age	Favorite Color	Loves Troll 2	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3



Now let's calculate the **Output Values** for the leaves.

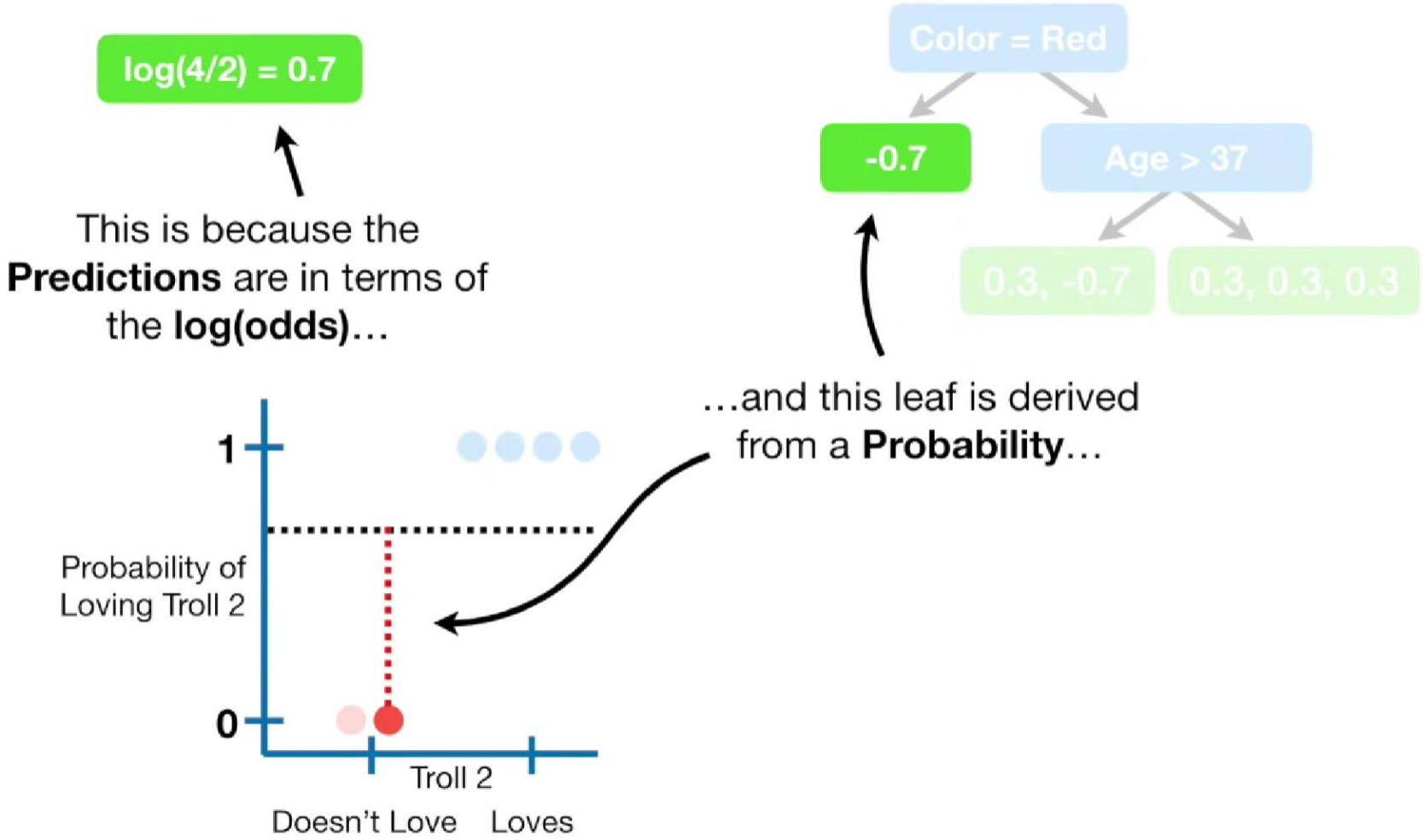
Gradient Boosting for Classification



When we used **Gradient Boost** for **Regression**, a leaf with single **Residual** had an **Output Value** equal to that **Residual**.

In contrast, when we use **Gradient Boost** for **Classification**, the situation is a little more complex.

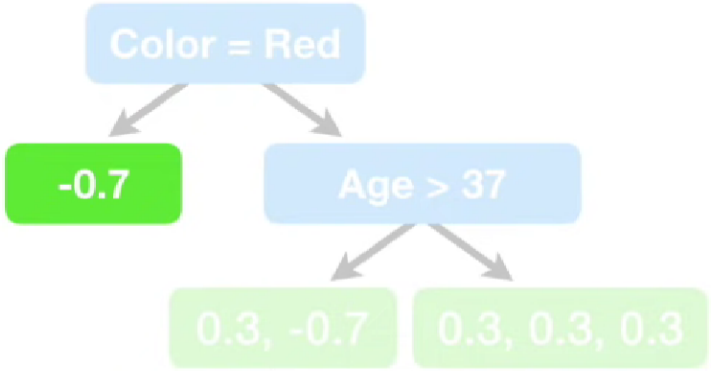
Gradient Boosting for Classification



Gradient Boosting for Classification

Gradient Boosting for Classification

When we use **Gradient Boost** for **Classification**, the most common transformation is the following formula.

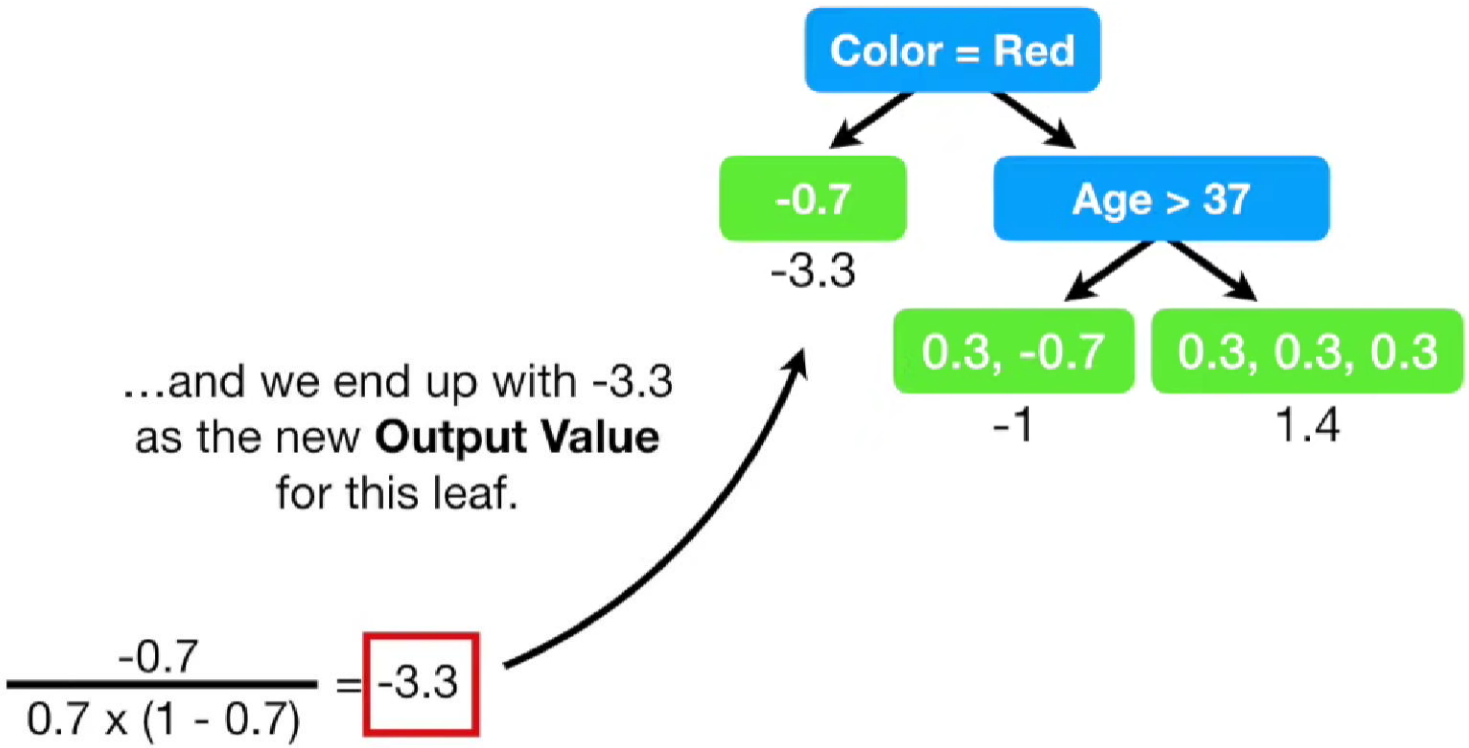


$$\frac{\sum \text{Residual}_i}{\sum \left[\text{Previous Probability}_i \times (1 - \text{Previous Probability}_i)\right]}$$

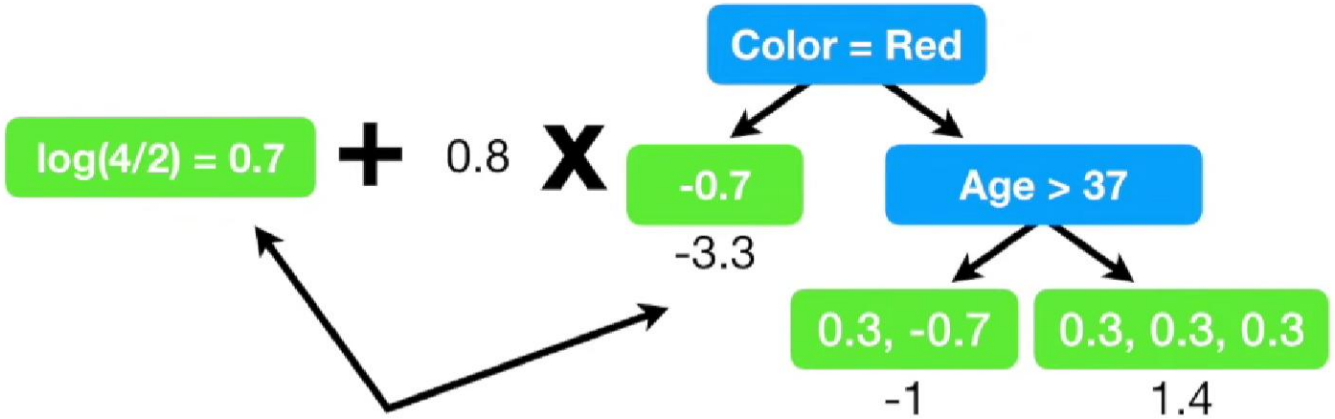
The numerator is the sum of the all of the **Residuals** in the leaf…

the denominator is the sum of the previously predicted probabilities for each **Residual** times **1** minus the same predicted probability.

Gradient Boosting for Classification



Gradient Boosting for Classification



Now we are ready to update our **Predictions** by combining the initial leaf with the new tree.

NOTE: Just like before, the new tree is scaled by a **Learning Rate**.

This example uses a relatively large **Learning Rate** for illustrative purposes. However, **0.1** is more common.

Gradient Boosting for Classification

Likes Popcorn	Age	Favorite Color	Loves Troll 2
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes

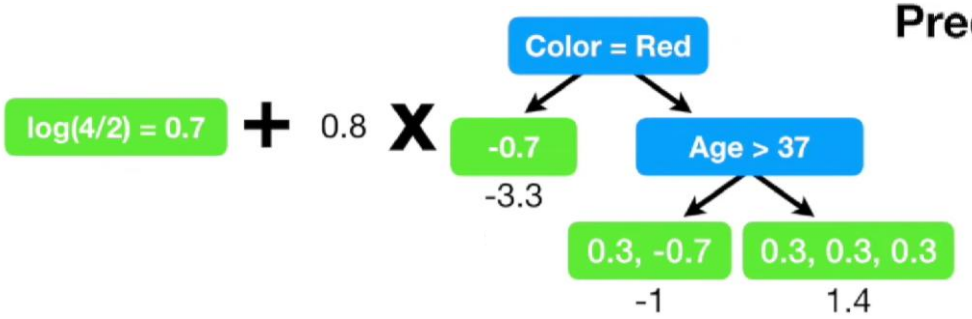
Now we convert the new **log(odds) Prediction** into a **Probability**...

$$\text{Probability} = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

$$\log(\text{odds}) \text{ Prediction} = 0.7 + (0.8 \times 1.4) = 1.8$$

$$\text{Probability} = \frac{e^{1.8}}{1 + e^{1.8}} = 0.9$$

...and the new **Predicted Probability = 0.9**...



Gradient Boosting for Classification

Likes Popcorn	Age	Favorite Color	Loves Troll 2	Predicted Prob.
Yes	12	Blue	Yes	0.9
Yes	87	Green	Yes	
No	44	Blue	No	
Yes	19	Red	No	
No	32	Green	Yes	
No	14	Blue	Yes	

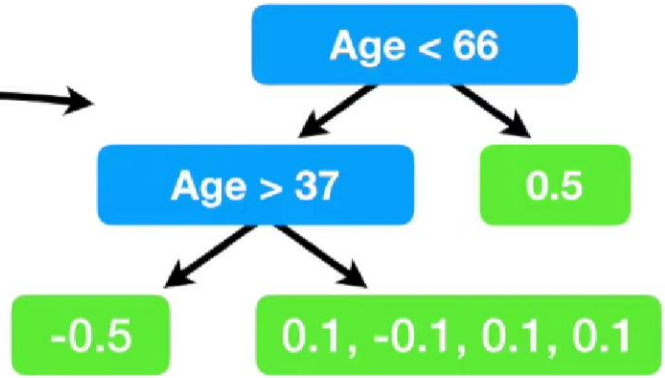
We save the new **Predicted Probability** here.

$$\text{Probability} = \frac{e^{1.8}}{1 + e^{1.8}} = 0.9$$

log(odds) Prediction = 0.7 + (0.8 × 1.4) = 1.8

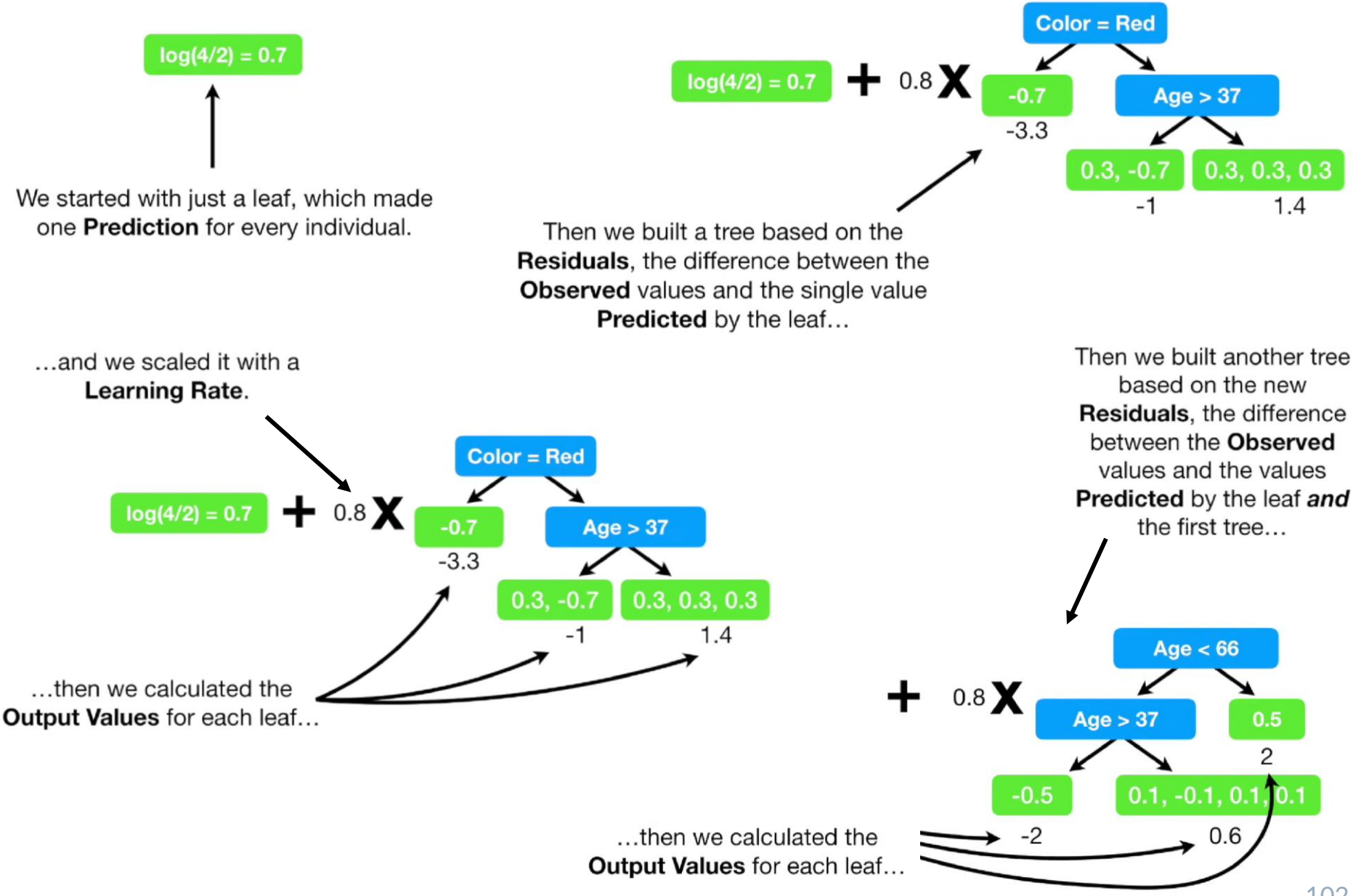
Gradient Boosting for Classification

Now that we have the **Residuals**, we can build a new tree…

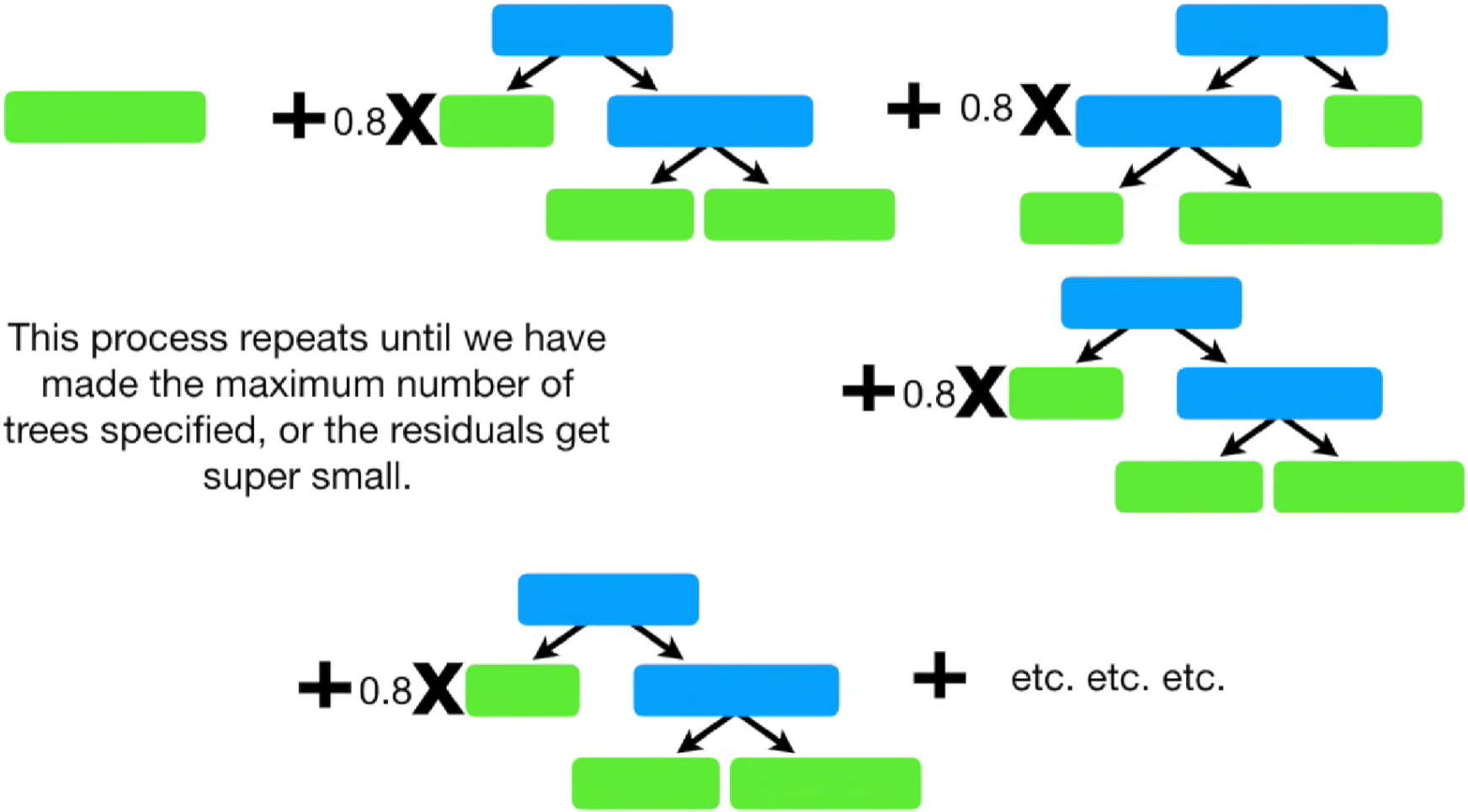


Likes Popcorn	Age	Favorite Color	Loves Troll 2	Predicted Prob.	Residual
Yes	12	Blue	Yes	0.9	0.1
Yes	87	Green	Yes	0.5	0.5
No	44	Blue	No	0.5	-0.5
Yes	19	Red	No	0.1	-0.1
No	32	Green	Yes	0.9	0.1
No	14	Blue	Yes	0.9	0.1

Gradient Boosting for Classification



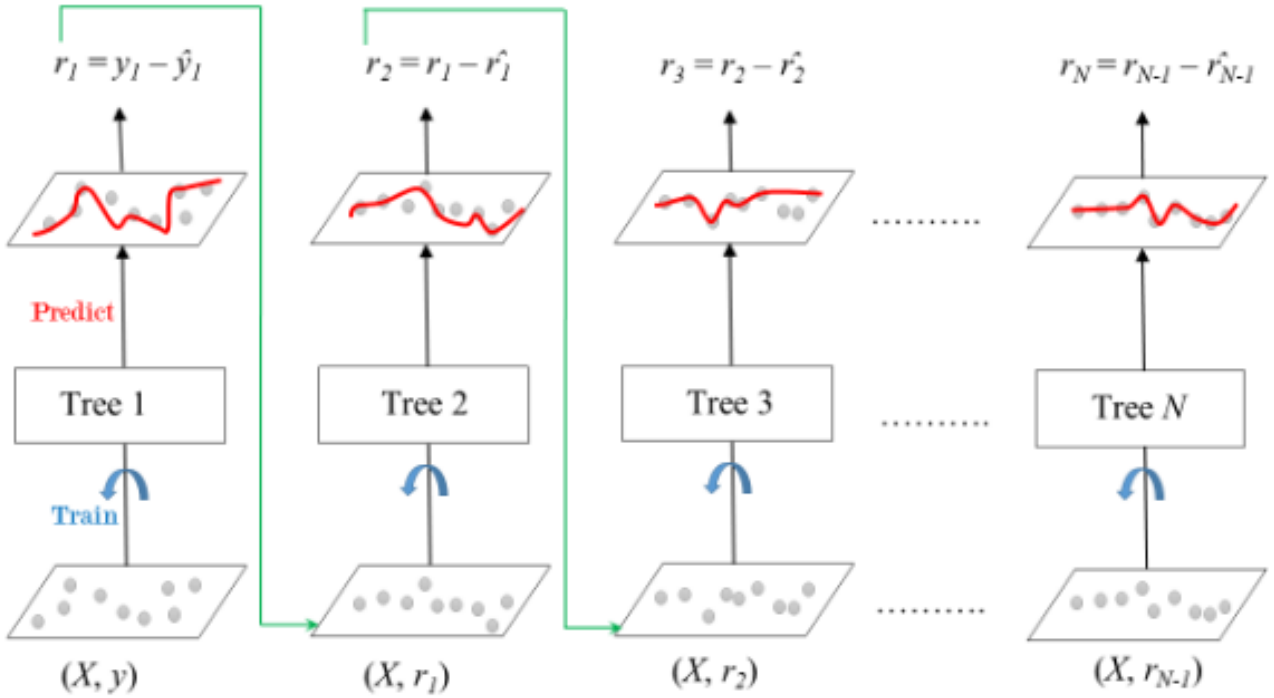
Gradient Boosting for Classification



Gradient boost uses Regression Trees for both regression and classification.

In both cases, the trees are fit to residuals, which are continuous and why we use regression trees

Gradient Boosting as Gradient Descent



ใช้ X เดิม แต่ target คือ residual

Tree ใหม่ที่เราฝึก --> กำลังประมาณ gradient (หรือ residual)

เราอัปเดต prediction โดย เดินไปในทิศทางที่ลด loss ได้มากที่สุด --> นั่นคือ Gradient Descent

Gradient Boosting as Gradient Descent

- Gradient Boosting has three main components:
  - Loss Function (or Error functions) - The role of the loss function is to estimate how good the model is at making predictions with the given data. This could vary depending on the problem at hand.
  - Weak Learner
  - Additive Model - This is the iterative and sequential approach of adding the trees (weak learners) one step at a time. After each iteration, we need to be closer to our final model. In other words, each iteration should reduce the value of our loss function

Gradient Boosting as Gradient Descent

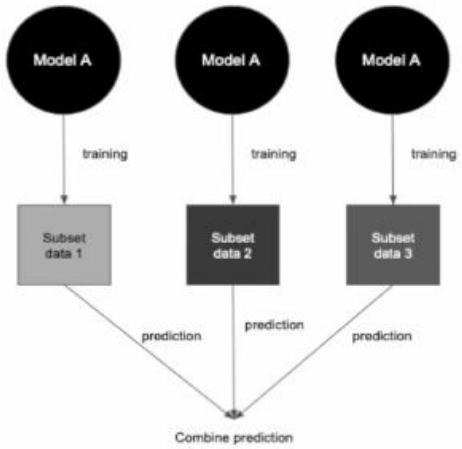
- GRADIENT BOOSTING IS GRADIENT DESCENT + BOOSTING
- To summarize, gradient boosting combines gradient descent and boosting:
  - Like AdaBoost, gradient boosting trains a weak learner to fix the mistakes made by the previous weak learner. AdaBoost uses example weights to focus learning on misclassified examples, while gradient boosting uses example residuals to do.
  - Like gradient descent, gradient boosting updates the current model with gradient information. Gradient descent uses the negative gradient directly, while gradient boosting trains a weak learner over the negative residuals to approximate the gradient.

Sequential ensembles: Gradient boosting

- AdaBoost trains a new weak learner to fix the misclassifications of the previous weak learner by maintaining and adaptively updating weights on training examples
- An alternative to weights on training examples to convey misclassification information to a base-learning algorithm for boosting: loss function gradients.
- We use loss functions to measure how well a model fits each training example in the data set.

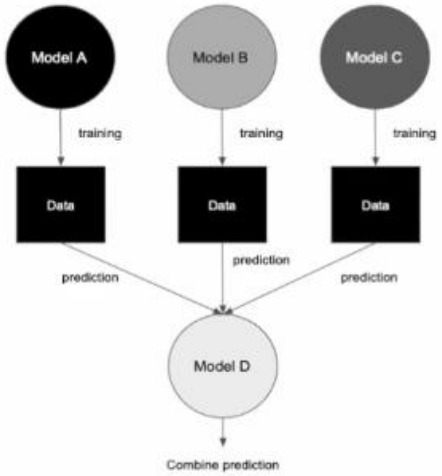
Loss Function	Applicability to Classification	Applicability to Regression
Mean Square Error (MSE) / L2 Loss	✗	✓
Mean Absolute Error (MAE) / L1 Loss	✗	✓
Binary Cross-Entropy Loss / Log Loss	✓	✗
Categorical Cross-Entropy Loss	✓	✗
Hinge Loss	✓	✗
Huber Loss / Smooth Mean Absolute Error	✗	✓
Log Loss	✓	✗

Bagging



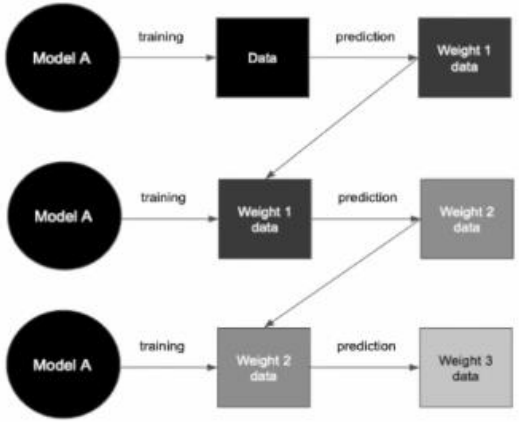
Train same model in parallel on different subset of dataset using Bootstrap sampling technique

Stacking



Train different model in parallel on same dataset and use model to combine model predictions

Boosting



Train same model in sequential manner where data is bias to put more focus on mistake previous model made